

UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

DIPARTIMENTO DI INGEGNERIA E SCIENZE DELL'INFORMAZIONE
E MATEMATICA

Tesi di Laurea in Ingegneria dell'Informazione

Studio di tecniche di riconoscimento di cloud botnet e realizzazione di una pipeline di triage assistito da LLM

Relatori:

Prof. Walter Tiberti

Prof. Andrea Marotta

Correlatore:

Prof. Piergiuseppe Di Marco

Candidato:

Elisabetta Casciano

Mat. 279569

ANNO ACCADEMICO 2025/2026

Abstract

This work analyzes anomaly detection techniques applied to system-level telemetry and develops a pipeline in which anomaly detection is followed by Large Language Model (LLM)-assisted triage. The experimental analysis is conducted on the BETH dataset, collected in a cloud environment and including, in its test set, the activity of a compromised host associated with an attempted botnet-node deployment.

In the main experimental configuration, kernel events are aggregated into 30-second time windows and represented through the relative frequencies of the observed event types. A sensitivity analysis across window sizes of 5, 10, 30, 60, and 120 seconds further shows that detector performance depends on the temporal granularity adopted: shorter windows yield higher SUS ROC-AUC values in the considered setting, whereas longer windows substantially reduce the number of available observations and make the resulting estimates less stable. Four different techniques are employed foranomaly detection: Isolation Forest, Local Outlier Factor, One-Class SVM, and Autoencoder. Their anomaly scores are also combined through an Ensemble.

Two different architectures are then compared: the detector + LLM pipeline and the LLM-only pipeline.

Among the individual detectors, the one based on the One-Class SVM achieved the best results with respect to the main SUS label, with a ROC-AUC of 0.690 and an F1-score of 0.667. The ensemble achieved a ROC-AUC of 0.679 and the same F1-score, without improving over the best individual detector. The detector + LLM pipeline identified 5 out of 7 SUS windows, whereas the LLM-only configuration identified 2 out of 7. The hybrid approach therefore achieved higher coverage of suspicious windows, although with a larger number of false positives, and reduced the number of LLM calls from 13 to 9.

The results highlight the different roles that anomaly detectors and LLMs can play within a security analysis pipeline. However, given the limited size of the test set, the findings should be interpreted as specific to the experimental setting considered in this work.

Indice

Indice **4**

1 Introduzione **7**

2 Background **10**

2.1 Anomaly detection e modalità di apprendimento 10

2.2 Principali metodi di anomaly detection 13

2.2.1 Isolation Forest 13

2.2.2 Local Outlier Factor 15

2.2.3 One-Class SVM 17

2.2.4 Autoencoder 18

2.3 Metriche di valutazione e soglia decisionale 20

2.4 Telemetria a livello di sistema ed eBPF 25

2.5 Large Language Model e triage di sicurezza 26

2.6 Rischi e limiti nell'impiego degli LLM 28

3 Il dataset BETH **31**

3.1 Origine e caratteristiche generali 31

3.2 Raccolta dei dati e architettura di monitoraggio 32

3.3 Struttura dei log kernel-process 33

3.4 Etichette `sus` ed `evil` 33

3.5 Suddivisione del dataset e sottoinsieme utilizzato 34

3.6 Scenario di attacco nel test set 35

3.7 Lavori correlati 37

4 Metodologia **42**

4.1 Obiettivo della metodologia 42

4.2 Dataset BETH e suddivisione dei dati 43
4.3 Preprocessing e costruzione delle finestre temporali 44
4.4 Feature engineering 46
4.5 Tecniche di anomaly detection 47
4.5.1 Isolation Forest 48
4.5.2 Local Outlier Factor 49
4.5.3 One-Class SVM 50
4.5.4 Autoencoder 51
4.6 Selezione delle soglie operative 54
4.7 Ensemble 55
4.8 Pipeline detector + LLM 56
4.9 Pipeline LLM-only 58
4.10 Protocollo di confronto 60

5 Risultati sperimentali **62**
5.1 Setup sperimentale e criteri di valutazione 62
5.2 Analisi esplorativa della rappresentazione mediante PCA e UMAP 63
5.2.1 Analisi mediante PCA 64
5.2.2 Interpretazione delle componenti principali 66
5.2.3 Analisi mediante UMAP 67
5.2.4 Confronto tra PCA e UMAP 68
5.3 Risultati dei singoli modelli di anomaly detection 70
5.3.1 Analisi di sensibilità alla durata delle finestre temporali . 74
5.4 Risultati dell'Ensemble e analisi delle soglie operative 79
5.5 Confronto con la letteratura 83
5.6 Risultati del triage LLM e confronto tra le due pipeline 87
5.6.1 Risultati della pipeline detector + LLM 87
5.6.2 Confronto con la pipeline LLM-only 89
5.6.3 Analisi descrittiva rispetto all'etichetta EVIL 91
5.7 Analisi dei tempi e delle risorse computazionali 91
5.7.1 Costo computazionale dei detector locali 92
5.7.2 Costo dello stadio locale della pipeline detector + LLM . 92
5.7.3 Latenza del triage LLM nella pipeline detector + LLM . 93

5.7.4 Confronto temporale tra le due pipeline 94
5.8 Discussione complessiva dei risultati 95

6 Conclusioni e sviluppi futuri **98**
6.1 Conclusioni 98
6.2 Limiti del lavoro 102
6.3 Sviluppi futuri 103

Bibliografia **105**

Capitolo 1

Introduzione

Negli ambienti informatici complessi, e in particolare nei sistemi cloud, la telemetria raccolta a livello di sistema consente di osservare in modo dettagliato l'attività svolta dagli host e può fornire informazioni utili per individuare comportamenti anomali. In ambito cybersecurity, tali comportamenti possono essere associati a tentativi di intrusione o ad attività che si discostano dal normale funzionamento del sistema. A causa dell'enorme quantità di dati da analizzare e alla difficoltà nel disporre di esempi etichettati rappresentativi delle diverse attività malevole, riuscire a riconoscere comportamenti che si discostano dalla normalità risulta essere complesso.

In questo contesto, le tecniche di anomaly detection consentono di individuare osservazioni che presentano caratteristiche differenti rispetto al comportamento atteso. In particolare, gli approcci non supervisionati risultano interessanti quando le informazioni disponibili sulle anomalie sono limitate, poiché permettono di costruire un modello del comportamento osservato senza richiedere una classificazione completa delle possibili tipologie di attacco.

L'individuazione di una possibile anomalia rappresenta tuttavia soltanto una prima fase dell'analisi. Gli alert generati dai detector devono successivamente essere esaminati per interpretare il comportamento osservato, individuare gli elementi che richiedono maggiore attenzione e stabilirne la priorità. Alla fase di anomaly detection segue quindi un processo di triage, finalizzato a supportare l'analisi delle segnalazioni prodotte.

Tradizionalmente, il triage degli alert di sicurezza viene svolto dagli analisti dei Security Operations Center, eventualmente con il supporto di regole, si-

stemi SIEM e strumenti di automazione. Sono stati inoltre proposti approcci basati su tecniche di machine learning per supportare attività come la classificazione degli alert. Più recentemente, i Large Language Model hanno aperto nuove possibilità anche per attività di questo tipo, grazie alla capacità di elaborare informazioni eterogenee e produrre descrizioni strutturate del contesto analizzato.

L'interesse verso queste tematiche è nato anche dalla partecipazione a un seminario professionalizzante offerto da Leonardo S.p.A. nell'ambito di AI4Security, che ha rappresentato un primo punto di contatto con l'applicazione di tecniche di intelligenza artificiale alla cybersecurity. Da questo spunto è nato l'approfondimento sviluppato nel presente lavoro.

L'obiettivo della tesi è studiare tecniche di anomaly detection non supervisionata applicate a dati di telemetria a livello kernel e valutare l'impiego di un Large Language Model come supporto alla successiva fase di triage. L'analisi viene condotta sul dataset BETH (BPF-Extended Tracking Honeypot), raccolto in ambiente cloud e progettato per lo studio dell'anomaly detection in ambito cybersecurity.

A differenza della maggior parte dei lavori svolti su questo dataset, gli eventi kernel vengono aggregati in finestre temporali e rappresentati attraverso le frequenze relative dei diversi tipi di evento osservati. Su questa rappresentazione vengono applicati quattro modelli di anomaly detection basati sulle tecniche di: Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder. Gli anomaly score prodotti dai singoli modelli vengono inoltre combinati attraverso un ensemble.

A partire da questa fase di rilevazione vengono considerate due differenti modalità di utilizzo del Large Language Model. Nella prima configurazione, indicata come pipeline detector + LLM, i modelli di anomaly detection costituiscono uno stadio iniziale di selezione: soltanto le finestre segnalate come anomale vengono successivamente inoltrate all'LLM. In questo caso il modello linguistico non deve svolgere nuovamente la detection, ma viene utilizzato per interpretare il comportamento osservato, assegnare una priorità all'alert e fornire indicazioni utili per il successivo approfondimento.

La seconda configurazione, denominata LLM-only, elimina invece lo stadio

preliminare di anomaly detection. Tutte le finestre del test set vengono fornite direttamente al modello linguistico, al quale viene richiesto sia di stabilire se il comportamento osservato debba essere considerato anomalo, sia di effettuare il successivo triage. Il confronto tra le due configurazioni permette quindi di valutare il diverso ruolo dell'LLM quando viene utilizzato come componente successiva a un sistema di rilevazione oppure direttamente come strumento di analisi della telemetria.

Il dataset utilizzato permette di analizzare il comportamento dei modelli in presenza di uno scenario di compromissione reale. Il test set di BETH contiene infatti l'attività registrata su un singolo host compromesso e comprende una sequenza di eventi riconducibile al tentativo di configurare l'host come nodo di una botnet. Il lavoro mira a studiare se tecniche di anomaly detection applicate alla telemetria di sistema siano in grado di evidenziare finestre associate a comportamenti sospetti o malevoli all'interno di tale scenario.

La valutazione sperimentale considera innanzitutto il comportamento dei singoli detector e dell'ensemble rispetto alle etichette disponibili nel dataset. Successivamente vengono confrontate le due pipeline, considerando sia la capacità di individuare le finestre annotate come sospette sia alcuni aspetti operativi, tra cui il numero di chiamate effettuate al modello linguistico e la latenza osservata.

La tesi è organizzata come segue. Il Capitolo 2 introduce i principali concetti relativi all'anomaly detection, i modelli utilizzati nella parte sperimentale, le metriche di valutazione, la telemetria a livello di sistema e l'impiego dei Large Language Model nel triage di sicurezza. Il Capitolo 3 descrive il dataset BETH, la struttura dei log, le etichette disponibili e lo scenario di attacco presente nel test set. Il Capitolo 4 presenta la metodologia adottata, dalla costruzione delle finestre temporali alla definizione dei detector, dell'ensemble e delle due pipeline considerate. Il Capitolo 5 riporta i risultati sperimentali e il confronto tra le configurazioni analizzate. Infine, il Capitolo 6 raccoglie le conclusioni, i limiti del lavoro e i possibili sviluppi futuri.

Capitolo 2

Background

Questo capitolo introduce i principali concetti teorici e tecnologici necessari per comprendere la metodologia adottata nel presente lavoro. Vengono innanzitutto richiamati i principi dell'anomaly detection e le principali modalità di apprendimento utilizzate per l'individuazione di comportamenti anomali. Successivamente vengono descritti i modelli di anomaly detection impiegati nella parte sperimentale, le metriche utilizzate per valutarne le prestazioni e il ruolo delle soglie decisionali. Il capitolo presenta inoltre i concetti di telemetria a livello di sistema ed eBPF, rilevanti per la raccolta dei dati BETH, e introduce infine l'impiego dei Large Language Model nel triage di sicurezza, insieme ai principali rischi associati al loro utilizzo.

2.1 Anomaly detection e modalità di apprendimento

L'*anomaly detection* ha l'obiettivo di individuare osservazioni o pattern che non risultano conformi al comportamento atteso dei dati. Tali osservazioni vengono generalmente indicate come anomalie o outlier e possono assumere significati differenti a seconda del dominio applicativo [5]. In ambito cybersecurity, un comportamento anomalo può essere associato, ad esempio, a un'attività di intrusione o a una deviazione rispetto al normale funzionamento di un sistema. L'anomaly detection risulta particolarmente rilevante per l'intrusion detection, poiché un'attività malevola può manifestarsi attraverso comportamenti diffe-

renti da quelli normalmente osservati. In questo ambito emergono tuttavia diverse difficoltà. I dati da analizzare possono essere molto numerosi e generati continuamente, mentre anche una percentuale relativamente ridotta di falsi positivi può produrre un numero elevato di segnalazioni da esaminare. Inoltre, ottenere esempi accuratamente etichettati di tutte le possibili attività malevole è complesso, sia perché le anomalie sono generalmente meno frequenti dei comportamenti normali, sia perché possono comparire nuove tipologie di attacco non presenti nei dati utilizzati durante lo sviluppo del sistema.

Negli Intrusion Detection System che impiegano tecniche di machine learning, l'individuazione delle anomalie dipende anche dal tipo di informazione disponibile durante l'addestramento del modello. La disponibilità delle etichette durante l'apprendimento permette di distinguere differenti modalità di anomaly detection. Nella tassonomia proposta da Chandola et al. vengono considerate tre principali modalità: supervisionata, semi-supervisionata e non supervisionata.

Nell'*anomaly detection supervisionata* il training set contiene istanze etichettate sia come *normali* (classe normale) sia *anomale* (classe anomala). Il problema può quindi essere affrontato costruendo un modello predittivo che apprende come distinguere le due classi e assegna successivamente le nuove osservazioni ad una di esse. Questo approccio richiede tuttavia la disponibilità di un insieme di dati già etichettato sufficientemente rappresentativo. Come anticipato, nel caso dell'anomaly detection, tale requisito può essere difficile da soddisfare, soprattutto per quanto riguarda le osservazioni anomale [5]. Anche in studi recenti sull'intrusion detection, la necessità di grandi quantità di dati etichettati viene indicata come una delle principali differenze operative tra approcci supervisionati e non supervisionati [6].

Nella modalità *semi-supervisionata*, secondo la classificazione utilizzata da Chandola et al., il training set contiene esempi esplicitamente etichettati appartenenti alla sola classe normale. Il modello dispone quindi dell'informazione che tali osservazioni rappresentano il comportamento normale e utilizza questi dati come riferimento per individuare, nei nuovi campioni, eventuali deviazioni [5]. Più in generale, la letteratura comprende anche approcci semi-supervisionati che integrano quantità limitate di informazione etichettata con

dati non etichettati, con l'obiettivo di mantenere la capacità di individuare anomalie non osservate in precedenza [7].

Gli approcci *non supervisionati* non utilizzano invece etichette di classe per apprendere direttamente la distinzione tra comportamento normale e anomalo. In applicazioni pratiche, il modello può essere costruito a partire dalla struttura dei dati non etichettati e identificare come potenzialmente anomale le osservazioni che risultano maggiormente differenti dai pattern appresi. Un'assunzione frequentemente associata a questi metodi è che il comportamento normale sia prevalente rispetto alle anomalie [5]. In studi applicativi recenti, il termine *unsupervised* viene utilizzato anche per modelli addestrati su un insieme di dati di training dal quale le etichette sono escluse [6].

Questa impostazione è di particolare interesse in cybersecurity, dove non è possibile assumere che tutte le modalità di attacco che verranno osservate in futuro siano già presenti nel training set. Görnitz et al. evidenziano come, nell'ambito della network intrusion detection, nuove minacce possano determinare una distribuzione dei dati differente da quella osservata durante l'addestramento. Un modello fortemente dipendente dalle classi di attacco note può quindi avere difficoltà nell'individuare tipologie di anomalia non precedentemente osservate [7]. Analogamente, Segurola-Gil et al. sottolineano il ruolo degli approcci non supervisionati nell'identificazione di minacce nuove, poiché tali metodi non richiedono informazioni etichettate sugli attacchi per costruire il proprio modello del comportamento [8].

L'assenza di supervisione non elimina tuttavia le difficoltà del problema. Metodi di anomaly detection non supervisionata possono produrre falsi positivi, segnalando come anomale attività legittime, oppure falsi negativi, non individuando alcune attività realmente anomale [8]. Le prestazioni dipendono quindi sia dal modello utilizzato sia dalla rappresentazione dei dati e dal criterio con cui gli *anomaly score* vengono successivamente trasformati in decisioni operative.

Nel presente lavoro viene adottata un'impostazione non supervisionata. Durante l'addestramento, i detector utilizzano esclusivamente le feature associate alle osservazioni del training set e non ricevono le relative etichette di classe. Le annotazioni del dataset vengono mantenute separatamente e utilizzate nelle

fasi successive per la selezione delle configurazioni e delle soglie operative sul validation set e per la valutazione finale sul test set.

2.2 Principali metodi di anomaly detection

Nel presente lavoro vengono considerati quattro metodi di anomaly detection basati su principi differenti: Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder.La scelta consente di confrontare approcci che individuano le anomalie secondo principi differenti e che trovano già applicazione nella letteratura sull'anomaly detection. In particolare, Liu et al. confrontano Isolation Forest con metodi quali Local Outlier Factor e One-Class SVM, mentre Chen et al. studiano l'impiego dell'Autoencoder per la network anomaly detection [9, 13].Le sezioni seguenti ne descrivono i principi di funzionamento, con particolare attenzione agli aspetti rilevanti per il successivo utilizzo nella metodologia sperimentale.

2.2.1 Isolation Forest

Isolation Forest (iForest) è un algoritmo di machine learning non supervisionato basato sul principio di isolamento delle osservazioni. L'idea proposta da Liu et al. parte dal fatto che le anomalie tendono a essere poco numerose e a presentare valori delle feature differenti rispetto alla maggior parte dei dati. Per questa ragione possono essere isolate più facilmente rispetto alle osservazioni normali [9].

Il funzionamento dell'algoritmo può essere descritto attraverso la costruzione degli *isolation tree* (*iTree*). Per ciascun albero viene considerato un sottoinsieme casuale dei dati X. A ogni nodo viene scelta casualmente una feature q e, per tale feature, un valore di separazione p compreso tra valore minimo e massimo osservati. Le istanze vengono quindi suddivise in due gruppi a seconda che il valore della feature sia inferiore oppure superiore alla soglia scelta.

$$X_l = \{x \in X : x_q < p\}, \qquad X_r = \{x \in X : x_q \geq p\}.$$

La stessa procedura viene ripetuta ricorsivamente sui gruppi ottenuti, fino a isolare progressivamente le singole osservazioni. Un'osservazione che presenta

caratteristiche molto diverse dalle altre tende a essere separata dopo poche partizioni; un punto appartenente a una regione contenente molte osservazioni simili richiede invece un numero maggiore di suddivisioni prima di essere isolato [23].

Il numero di archi attraversati dalla radice dell'albero fino al nodo terminale raggiunto dall'osservazione x prende il nome di *path length* ed è indicato con $h(x)$. Una path length breve costituisce quindi un'indicazione di possibile anomalia. Poiché ogni isolation tree viene costruito mediante scelte casuali, la path length di una stessa osservazione può variare da un albero all'altro. Per questo motivo Isolation Forest considera la lunghezza media dei percorsi ottenuti sull'insieme degli alberi. Tale quantità viene indicata con $E[h(x)]$, dove $E[\cdot]$ rappresenta il valore atteso. Nel caso della foresta, esso viene stimato come la media delle path length osservate nei diversi iTree [23, 9].

Indicando con t il numero di alberi della foresta, il concetto può essere espresso come

$$E[h(x)] \simeq \frac{1}{t} \sum_{k=1}^{t} h_k(x),$$

dove $h_k(x)$ rappresenta la path length dell'osservazione x nel k-esimo albero.

La sola path length media non è però sufficiente per ottenere uno score normalizzato. Gli autori introducono quindi una lunghezza media di riferimento, indicata con $c(\psi)$, dove ψ rappresenta il numero di osservazioni utilizzate per costruire ciascun isolation tree. Per $\psi > 2$ essa è definita come

$$c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}.$$

Il termine $H(i)$ indica l'i-esimo numero armonico, definito come

$$H(i) = \sum_{r=1}^{i} \frac{1}{r} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{i}.$$

Per valori sufficientemente grandi di i, esso può essere approssimato come

$$H(i) \simeq \ln(i) + 0.5772156649.$$

La quantità $c(\psi)$ rappresenta quindi una lunghezza media di riferimento per un albero binario casuale costruito a partire da ψ osservazioni e viene utilizzata per normalizzare la path length media.
A partire da queste quantità, l'anomaly score teorico di una osservazione x viene definito come

$$s(x, \psi) = 2^{-\frac{E[h(x)]}{c(\psi)}}.$$

La formula mostra che lo score aumenta quando la path length media diminuisce. Se un'osservazione viene isolata molto rapidamente, $E[h(x)]$ è piccolo e lo score tende a 1, indicando un'elevata anomalia. Se invece $E[h(x)]$ è vicino alla lunghezza media di riferimento $c(\psi)$, lo score tende a 0.5. Path length più elevate producono infine score più bassi e sono maggiormente associate a osservazioni normali [9].

2.2.2 Local Outlier Factor

Il Local Outlier Factor (LOF) è un metodo di outlier detection basato sull'analisi della densità locale delle osservazioni. L'idea proposta da Breunig et al. nasce dall'osservazione che una valutazione globale della distanza può non essere sufficiente quando il dataset contiene regioni con densità differenti. Un'osservazione può infatti risultare anomala rispetto al proprio vicinato anche senza essere particolarmente distante dall'insieme dei dati considerato nel suo complesso. LOF introduce quindi una misura del grado di anomalia di ciascuna osservazione rispetto alla densità delle osservazioni circostanti [10].
Il primo passo consiste nella definizione del vicinato locale di un'osservazione p. Dato un numero k di vicini, la *k-distance* di p corrisponde alla massima distanza tra p e i suoi k punti più prossimi. Il *k-distance neighborhood* contiene quindi le osservazioni la cui distanza da p non supera tale valore. Nel seguito del metodo questo numero di vicini viene indicato attraverso il parametro *MinPts*. Quando *MinPts* è piccolo, LOF è molto sensibile a piccole variazioni; viceversa, quando *MinPts* è grande, le anomalie locali potrebbero essere meno evidenti [10]. È necessario pensare a una densità locale perché nel momento in cui un punto risulta essere distante ma anche i suoi vicini sono distanti

tra loro, allora non si tratta necessariamente di un'anomalia. Se un punto ha una densità simile a quella dei suoi vicini, il valore del Local Outlier Factor sarà LOF ≈ 1 ed il punto può essere considerato normale. Al contrario, se il punto si trova in una regione caratterizzata da una densità significativamente inferiore rispetto a quella dei suoi vicini, si avrà LOF > 1 indicando un comportamento potenzialmente anomalo. Per stimare la densità locale, [10] introduce la *reachability distance*. Data un'osservazione p e un suo vicino o, essa è definita come

$$\text{reach-dist}_k(p, o) = \max\{k\text{-distance}(o), d(p, o)\}$$

dove $d(p, o)$ rappresenta la distanza effettiva tra i due punti. Se p è sufficientemente lontano da o, viene quindi utilizzata la loro distanza reale. Se invece i due punti sono molto vicini, la distanza viene sostituita dalla k-distance di o. Questa operazione introduce un limite inferiore alle distanze considerate e riduce l'influenza delle piccole fluttuazioni tra osservazioni molto vicine appartenenti allo stesso vicinato.

A partire dalle reachability distance viene calcolata la *local reachability density* (LRD). Essa rappresenta, in modo intuitivo, l'inverso della reachability distance media tra p e i suoi vicini. Di conseguenza, se le distanze rispetto al vicinato sono mediamente piccole, la densità locale risulta elevata; se invece sono grandi, la densità locale risulta più bassa.

Il Local Outlier Factor viene ottenuto confrontando la densità locale di p con quelle dei suoi vicini. In particolare, il LOF di p corrisponde alla media dei rapporti tra la local reachability density dei vicini e quella di p:

$$LOF(p) = \frac{1}{|N_{\text{MinPts}}(p)|} \sum_{o \in N_{\text{MinPts}}(p)} \frac{lrd(o)}{lrd(p)}.$$

Se p si trova in una regione con densità simile a quella del proprio vicinato, il suo LOF tende ad assumere valori prossimi a 1. Se invece la densità locale di p è inferiore rispetto a quella dei suoi vicini, il rapporto aumenta e il punto assume un LOF maggiore, indicando un grado più elevato di anomalia locale.

Il comportamento del metodo dipende dal parametro *MinPts*, che determina la dimensione del vicinato utilizzato nel confronto delle densità locali. Breunig et al. mostrano che il valore del LOF non varia necessariamente in modo

monotono al crescere di *MinPts*: modificare il numero di vicini può infatti cambiare la struttura locale rispetto alla quale un'osservazione viene valutata. La scelta del parametro determina quindi la scala del vicinato considerato nell'analisi [10].

2.2.3 One-Class SVM

La One-Class Support Vector Machine (One-Class SVM) è una variante delle Support Vector Machine progettata per descrivere una singola classe di osservazioni e individuare i punti che non risultano conformi ad essa. A differenza di una SVM tradizionale, che apprende una separazione tra due classi, la One-Class SVM costruisce una frontiera decisionale utilizzando le osservazioni disponibili come riferimento [11].

Il funzionamento del metodo si basa sulla trasformazione dei dati in uno spazio delle feature mediante una funzione $\phi(\cdot)$ definita implicitamente da un kernel. Il kernel è una funzione che permette alla SVM di confrontare i punti come se fossero stati trasformati in uno spazio più complesso, senza dover effettuare esplicitamente tale trasformazione. In tale spazio, l'algoritmo ricerca un iperpiano che separi la maggior parte delle osservazioni dall'origine, massimizzando la separazione tra i dati e quest'ultima. Una piccola frazione delle osservazioni può trovarsi dall'altro lato della frontiera ed essere quindi considerata anomala [11].

La frontiera decisionale può essere espressa attraverso la funzione

$$g(x) = w^T \phi(x) - \rho,$$

dove w determina l'orientamento dell'iperpiano e ρ determina dove viene posizionato e $\phi(x)$ rappresenta il punto trasformato nello spazio delle feature. La superficie per cui $g(x) = 0$ rappresenta la frontiera decisionale. Il segno della funzione permette quindi di distinguere le osservazioni considerate appartenenti alla regione modellata da quelle che si trovano al di fuori di essa:

$$f(x) = \text{sgn}(g(x)).$$

Nella formulazione presente in [11], un valore positivo corrisponde a un'osservazione considerata normale, mentre un valore negativo indica un punto posto dall'altro lato della frontiera e quindi classificato come outlier.
L'impiego di un kernel consente di ottenere frontiere decisionali non lineari rispetto allo spazio originale delle feature. In particolare, gli autori considerano il kernel gaussiano, attraverso il quale i dati vengono rappresentati implicitamente in uno spazio in cui può essere costruito l'iperpiano di separazione. Questo aspetto è rilevante quando la distribuzione delle osservazioni non può essere descritta adeguatamente mediante una semplice separazione lineare.
Un parametro importante della One-Class SVM è ν, che interviene nella regolarizzazione del modello e controlla il compromesso tra la separazione dall'origine e la possibilità di ammettere alcune osservazioni oltre la frontiera decisionale. Nella formulazione riportata da Amer et al., ν costituisce un limite superiore alla frazione di outlier e un limite inferiore alla frazione di support vector [11]. Pertanto, il parametro influenza direttamente quanto restrittiva risulta la regione appresa dal modello, senza determinare in modo rigido una percentuale esatta di anomalie. Valori maggiori di ν consentono quindi al modello di tollerare una frazione maggiore di osservazioni di training al di fuori della regione appresa.
La One-Class SVM viene tradizionalmente utilizzata in un'impostazione semi-supervisionata, nella quale il modello viene addestrato utilizzando osservazioni considerate normali. Resta possibile, tuttavia, anche il suo utilizzo in contesti non supervisionati, nei quali i dati impiegati per costruire la frontiera non dispongono di etichette. In questo caso, eventuali anomalie presenti durante l'apprendimento possono influenzare la posizione della frontiera decisionale, rendendo la qualità del modello dipendente anche dalla composizione dei dati utilizzati [11].

2.2.4 Autoencoder

Un Autoencoder è una rete neurale non supervisionata il cui obiettivo è apprendere una rappresentazione dei dati che permetta di ricostruire in uscita il vettore fornito in ingresso. Sakurada e Yairi ne propongono l'impiego come tec-

nica di riduzione dimensionale non lineare applicata al problema dell'anomaly detection [12].

Dal punto di vista architetturale, un Autoencoder è costituito da due componenti principali: un *encoder* e un *decoder*. Indicando con $x \in \mathbb{R}^d$ il vettore di ingresso, l'encoder lo trasforma in una rappresentazione interna composta da m unità, con $m < d$. In questo modo l'informazione originariamente descritta da d variabili viene rappresentata in uno spazio a dimensionalità inferiore. Il decoder effettua successivamente l'operazione inversa, utilizzando tale rappresentazione interna per produrre un vettore ricostruito $\hat{x}$ nello spazio originale [13].

Durante l'addestramento, i parametri dell'encoder e del decoder vengono ottimizzati in modo da minimizzare la differenza tra il vettore di ingresso x e la corrispondente ricostruzione $\hat{x}$. La rete è quindi spinta ad apprendere, nella rappresentazione a dimensionalità ridotta, le informazioni necessarie per ricostruire i dati di ingresso [13].

All'interno dei livelli della rete, ai valori calcolati dai neuroni può essere applicata una funzione di attivazione. L'impiego di funzioni non lineari consente all'Autoencoder di rappresentare relazioni non lineari tra le feature [13].

L'impiego dell'Autoencoder per l'anomaly detection si basa sull'idea che il modello apprenda le relazioni che caratterizzano il comportamento normale dei dati. Una nuova osservazione coerente con tali relazioni tende a essere ricostruita con un errore contenuto. Al contrario, un'osservazione che presenta caratteristiche differenti rispetto a quelle apprese può produrre una ricostruzione meno accurata e, di conseguenza, un errore maggiore [12].

Sakurada e Yairi utilizzano quindi l'errore di ricostruzione come *anomaly score*. Per un'osservazione $x^{(i)}$, esso viene espresso come

$$Err(i) = \sqrt{\sum_{j=1}^{D} \left(x_j^{(i)} - \hat{x}_j^{(i)} \right)^2},$$

dove $x_j^{(i)}$ rappresenta il valore originale della j-esima variabile e $\hat{x}_j^{(i)}$ il corrispondente valore ricostruito. Un errore di ricostruzione contenuto indica quindi una maggiore coerenza con le caratteristiche apprese dal modello, mentre valori più elevati costituiscono un'indicazione di possibile anomalia [12].

Come detto prima,un vantaggio dell'Autoencoder è la possibilità di modellare relazioni non lineari tra le feature. Un termine di confronto è rappresentato dalla Principal Component Analysis (PCA), una tecnica di riduzione della dimensionalità che rappresenta i dati mediante un numero ridotto di componenti principali. La trasformazione effettuata dalla PCA è lineare: i dati vengono quindi proiettati in uno spazio di dimensione inferiore attraverso combinazioni lineari delle variabili originali.
Chen et al. osservano che questa caratteristica può limitarne la capacità di rappresentare relazioni non lineari tra le feature. Un Autoencoder, invece, può introdurre trasformazioni non lineari attraverso le funzioni di attivazione dei suoi livelli e apprendere quindi rappresentazioni più complesse [13]. Anche Sakurada e Yairi mostrano sperimentalmente come l'uso di una riduzione dimensionale non lineare possa essere utile nell'individuazione delle anomalie [12].

2.3 Metriche di valutazione e soglia decisionale

Le tecniche di anomaly detection descritte nelle sezioni precedenti producono, per ciascuna osservazione, un punteggio continuo che esprime il grado di anomalia attribuito dal detector. La valutazione di tali punteggi può essere effettuata senza fissare immediatamente una soglia per ottenere una decisione binaria, analizzando la capacità del modello di distinguere le osservazioni appartenenti alle due classi al variare della soglia decisionale.
In questo contesto, l'analisi mediante curva *Receiver Operating Characteristic* (ROC) rappresenta uno degli strumenti comunemente utilizzati per la valutazione dei problemi di classificazione binaria. La curva ROC descrive il comportamento del classificatore considerando differenti valori della soglia decisionale. Ciascun punto della curva corrisponde quindi a una specifica configurazione operativa del modello, determinata dal valore della soglia utilizzata [15].
Una sintesi numerica della curva ROC è fornita dall'*Area Under the Curve* (AUC). A differenza delle metriche calcolate in corrispondenza di una singola soglia, l'AUC considera il comportamento del modello sull'insieme delle so-

glie decisionali. Essa costituisce pertanto una misura globale della capacità discriminante degli score prodotti dal classificatore [15].
Secondo l'interpretazione tradizionale della ROC-AUC, un valore pari a 0.5 corrisponde a una capacità di discriminazione equivalente a quella ottenibile mediante un ordinamento casuale, mentre un valore pari a 1 corrisponde a una discriminazione perfetta tra le due classi [15].
Una misura globale come l'AUC non sostituisce tuttavia la scelta di una soglia quando il sistema deve produrre una decisione binaria. Gli anomaly score sono infatti valori continui, mentre in una configurazione operativa è necessario stabilire quali osservazioni debbano essere considerate normali e quali anomale. La soglia determina quindi il punto operativo del classificatore e influenza le metriche calcolate sulle decisioni risultanti. Metriche quali accuracy, sensitivity, specificity e F1-score descrivono infatti il comportamento del modello in corrispondenza di una particolare scelta della soglia, mentre l'AUC considera l'intera curva ROC [15].
Nel presente lavoro vengono pertanto mantenuti distinti i due livelli di valutazione. La ROC-AUC viene utilizzata per descrivere la capacità degli anomaly score di separare le osservazioni positive da quelle negative senza fissare una soglia specifica. Le decisioni binarie vengono invece ottenute successivamente applicando una soglia operativa agli score prodotti dai detector. La curva ROC descrive il comportamento del classificatore considerando differenti valori della soglia decisionale. Ciascun punto della curva corrisponde quindi a una specifica configurazione operativa del modello, determinata dal valore della soglia utilizzata [15]. La Figura 2.1 mostra graficamente come differenti valori della soglia corrispondano a differenti punti operativi lungo la curva ROC. L'asse orizzontale della curva ROC rappresenta il *False Positive Rate* (FPR), definito come

$$FPR = \frac{FP}{FP + TN} = 1 - \text{specificity}, \tag{2.1}$$

dove FP indica il numero di falsi positivi e TN il numero di veri negativi. Il FPR rappresenta quindi la proporzione di osservazioni negative che vengono erroneamente classificate come positive.

L'asse verticale rappresenta invece il *True Positive Rate* (TPR), detto anche *sensitivity* o *recall*, definito come

$$TPR = \frac{TP}{TP + FN}, \tag{2.2}$$

dove TP indica il numero di veri positivi e FN il numero di falsi negativi. Il TPR rappresenta quindi la proporzione di osservazioni positive correttamente identificate dal modello. Nella rappresentazione proposta da Carrington et al., i cerchi individuano esempi di singoli punti operativi, ciascuno associato a una specifica soglia decisionale. Le croci indicano invece esempi di soglie che gli autori considerano irrealistiche o indesiderabili nel contesto applicativo. Tale caratterizzazione non è universale, poiché l'interesse per una determinata regione della curva dipende dalle condizioni operative considerate. L'AUC include comunque l'intera curva ROC e quindi anche tali regioni [15]. La figura evidenzia pertanto la differenza tra una misura globale quale l'AUC e le metriche valutate in corrispondenza di uno specifico punto operativo. Un valore di AUC maggiore indica una migliore prestazione complessiva del classificatore secondo la misura ROC-AUC. Tale confronto globale non implica tuttavia che il classificatore con AUC più elevata presenti prestazioni migliori in ogni regione dello spazio ROC. Fawcett mostra infatti che un classificatore con AUC complessivamente maggiore può risultare meno performante di un classificatore con AUC inferiore in una specifica regione della curva [16].

Questo aspetto deriva dal fatto che l'AUC sintetizza in un unico valore il comportamento del classificatore lungo l'intera curva ROC. Carrington et al. osservano inoltre che tale misura considera tutte le soglie decisionali, comprese soglie che possono risultare irrealistiche o indesiderabili nel contesto applicativo considerato [15].

Una volta fissata una soglia decisionale, gli anomaly score vengono trasformati in una classificazione binaria. Il confronto tra la classe reale e quella assegnata dal modello può essere rappresentato mediante una *matrice di confusione*, dalla quale derivano diverse metriche di valutazione [16].

In un problema binario si distinguono quattro possibili risultati:

- *True Positive* (TP): un'osservazione positiva viene correttamente classificata come positiva;

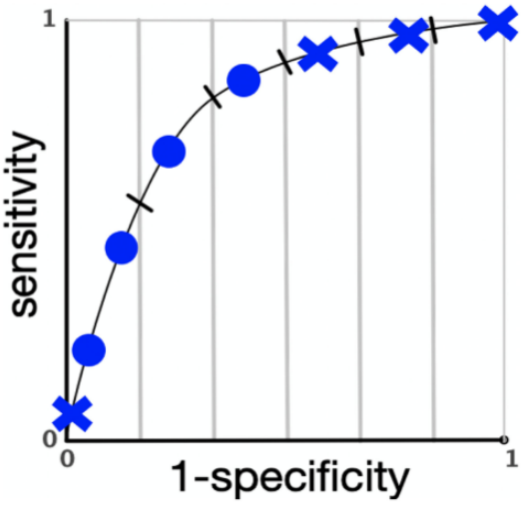


Figura 2.1: Rappresentazione schematica di una curva ROC. Ciascun punto della curva corrisponde a una diversa soglia decisionale; l'area sottesa dalla curva rappresenta la ROC-AUC. Elaborazione basata sui concetti descritti da Carrington et al. [15].

- *False Positive* (FP): un'osservazione negativa viene erroneamente classificata come positiva;
- *True Negative* (TN): un'osservazione negativa viene correttamente classificata come negativa;
- *False Negative* (FN): un'osservazione positiva viene erroneamente classificata come negativa.

Nel contesto dell'anomaly detection, considerando la classe anomala come positiva, un vero positivo corrisponde quindi a un'anomalia correttamente rilevata, mentre un falso positivo rappresenta un'osservazione normale segnalata erroneamente come anomala. Un falso negativo corrisponde invece a un'anomalia non individuata dal sistema.
La matrice di confusione può essere rappresentata come segue:

Tabella 2.1: Matrice di confusione per un problema di classificazione binaria.

	Classe reale positiva	**Classe reale negativa**
Predetto positivo	TP	FP
Predetto negativo	FN	TN

A partire dai quattro valori della matrice di confusione è possibile calcolare diverse metriche. La *precision* misura la proporzione di veri positivi tra tutte le osservazioni classificate come positive:

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{2.3}$$

Nel caso dell'anomaly detection, essa indica quindi quale frazione degli alert prodotti dal sistema corrisponda effettivamente a osservazioni appartenenti alla classe positiva.

La *recall*, equivalente al True Positive Rate e indicata anche come *sensitivity*, misura invece la proporzione di osservazioni positive correttamente individuate:

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{2.4}$$

Una recall elevata indica pertanto che il modello riesce a individuare una quota elevata delle osservazioni positive presenti nei dati. Precision e recall descrivono aspetti differenti del comportamento del classificatore: la prima considera l'affidabilità delle segnalazioni positive, mentre la seconda considera la capacità di recuperare le osservazioni effettivamente positive [16].

Per sintetizzare congiuntamente precision e recall viene utilizzato l'*F1-score*, definito come la loro media armonica:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{2.5}$$

L'F1-score assume valori elevati quando entrambe le componenti risultano elevate e diminuisce quando una delle due assume un valore ridotto. Per questa ragione consente di considerare contemporaneamente la capacità del modello di individuare i positivi e la precisione delle segnalazioni prodotte.

Un'ulteriore metrica è l'*accuracy*, che rappresenta la proporzione complessiva di osservazioni classificate correttamente:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{2.6}$$

L'accuracy considera quindi sia i veri positivi sia i veri negativi. La sua interpretazione richiede tuttavia attenzione quando la distribuzione delle classi

è sbilanciata. Fawcett osserva infatti che metriche quali accuracy, precision e F-score dipendono dalla distribuzione delle classi, mentre le curve ROC sono costruite a partire da TPR e FPR, calcolati separatamente sulle osservazioni positive e negative [16].

Le metriche appena descritte dipendono dalla decisione binaria prodotta dal classificatore e, di conseguenza, dalla soglia utilizzata per trasformare gli score continui in etichette. Modificando la soglia possono infatti cambiare i valori di TP, FP, TN e FN e quindi precision, recall, F1-score e accuracy. Questo distingue tali misure dalla ROC-AUC, che considera invece il comportamento degli score sull'insieme dei possibili valori della soglia [15].

Nel presente lavoro questa distinzione viene mantenuta anche nella valutazione sperimentale. La ROC-AUC viene utilizzata per analizzare la capacità degli anomaly score di separare le osservazioni appartenenti alle due classi senza fissare uno specifico punto operativo. Precision, recall, F1-score e accuracy vengono invece calcolate dopo l'applicazione di una soglia decisionale. La procedura adottata per la selezione delle soglie operative dei singoli detector viene descritta nel 4.

2.4 Telemetria a livello di sistema ed eBPF

L'analisi di sicurezza di un sistema può basarsi sull'osservazione di eventi generati a livello del sistema operativo, come l'attività dei processi, le chiamate di sistema e gli eventi di rete. Una tecnologia che consente di raccogliere informazioni di questo tipo è eBPF (*extended Berkeley Packet Filter*), che permette di eseguire programmi all'interno del kernel e di strumentarne il comportamento senza modificarne il codice sorgente. eBPF consente di associare piccoli programmi a specifici eventi del sistema, come l'esecuzione di una chiamata di sistema. Quando l'evento si verifica, il programma eBPF viene eseguito e può raccogliere informazioni su di esso. Prima del caricamento, il programma viene controllato dal *verifier*, che verifica che possa essere eseguito in sicurezza. I programmi eBPF possono essere associati a specifici punti di aggancio del kernel, detti *hook*, e vengono eseguiti quando si verifica l'evento corrispondente. Tra gli eventi ai quali è possibile collegare un programma eBPF rientrano, ad

esempio, le chiamate di sistema, l'ingresso o l'uscita da una funzione, i tracepoint del kernel e gli eventi di rete [18]. In questo modo è possibile osservare l'attività del sistema in corrispondenza di eventi specifici.
Un altro elemento dell'architettura eBPF è costituito dalle *BPF maps*, strutture dati che permettono di conservare e condividere informazioni. Le mappe possono essere utilizzate sia da più programmi eBPF sia per lo scambio di dati tra il kernel e applicazioni eseguite in user space [18, 17]. Le informazioni raccolte dai programmi eBPF possono quindi essere rese disponibili in user space per successive attività di elaborazione e analisi.
Tra gli eventi di particolare interesse per il monitoraggio a livello di sistema rientrano le *system call*. Attraverso i tracepoint è possibile attivare programmi eBPF all'ingresso o all'uscita di specifiche chiamate di sistema e osservare le operazioni effettuate dai processi. Soldani et al. riportano, ad esempio, il monitoraggio di operazioni sul file system mediante chiamate quali `open`, `close`, `read` e `write`. La possibilità di intercettare questo tipo di eventi rende eBPF utilizzabile per attività di tracing, osservabilità, intrusion detection e runtime security [18].
Nel contesto del presente lavoro, l'interesse per eBPF deriva dalla possibilità di raccogliere eventi a livello kernel e di utilizzarli come telemetria per successive attività di analisi. Il dataset BETH, descritto nel Capitolo 3, contiene informazioni ottenute mediante monitoraggio a livello di sistema [1]; le modalità specifiche di raccolta e la struttura dei log utilizzati nella parte sperimentale vengono approfondite nel capitolo successivo.

2.5 Large Language Model e triage di sicurezza

Nei Security Operations Center (SOC), una delle prime attività svolte dagli analisti è il triage degli alert di sicurezza. Il triage consiste nella valutazione delle segnalazioni ricevute e nella determinazione della loro priorità, con l'obiettivo di distinguere gli eventi che richiedono attenzione da quelli che possono essere considerati meno rilevanti. L'aumento del numero di alert da analizzare può però rendere questa attività particolarmente onerosa e contribuire al

fenomeno dell'*alert fatigue*, cioè alla condizione in cui gli analisti risultano sovraccaricati dalla quantità di segnalazioni da esaminare [19].

In questo contesto è stato studiato anche l'impiego dei Large Language Model (LLM) come supporto alle attività svolte dagli analisti. Gli LLM sono modelli linguistici di grandi dimensioni e basati su reti neurali, in grado di elaborare informazioni espresse in linguaggio naturale [19]. Il loro utilizzo in un SOC può riguardare, tra le altre attività, la classificazione degli alert e la loro prioritizzazione.

Il triage non consiste infatti soltanto nello stabilire se una segnalazione sia associata o meno a un'effettiva minaccia. Nella fase iniziale di analisi è necessario anche valutare la criticità dell'alert, inserirlo nel relativo contesto e stabilire con quale priorità debba essere trattato. Queste operazioni costituiscono tipicamente parte delle attività svolte dagli analisti di primo livello di un SOC [19].

Rieger et al. hanno analizzato l'impiego degli LLM proprio per queste attività, considerando un insieme di 178 alert etichettati manualmente. Otto LLM general-purpose sono stati utilizzati per classificare gli alert come *true positive* o *false positive* e per assegnare loro uno dei quattro livelli di priorità considerati nello studio: *low*, *medium*, *high* e *critical*. Per la classificazione binaria gli autori hanno inoltre effettuato un confronto con modelli di machine learning supervisionato, tra cui Logistic Regression, Random Forest e Linear SVM. I risultati mostrano che diversi LLM riescono a ottenere valori elevati di recall nella classificazione degli alert. Tuttavia, i modelli di machine learning più leggeri considerati dagli autori ottengono prestazioni competitive e, in alcuni casi, superiori nella classificazione binaria. Nello studio, in particolare, la Linear SVM raggiunge il valore complessivo di F1-score più elevato [19]. L'impiego di un LLM non implica quindi necessariamente prestazioni di classificazione superiori rispetto a modelli più semplici.

La prioritizzazione degli alert risulta invece più difficile per gli LLM analizzati. Rieger et al. osservano che, pur ottenendo in alcuni casi una recall elevata per gli alert ad alta severità, la precision rimane ridotta, con un conseguente aumento del numero di segnalazioni erroneamente prioritizzate. Gli autori concludono quindi che gli LLM possono fornire un supporto al triage iniziale e

all'analisi contestuale, ma presentano ancora limiti nell'assegnazione affidabile della priorità agli alert [19].

Nel presente lavoro l'impiego dell'LLM viene considerato proprio in questa prospettiva di supporto al triage. Nella pipeline ibrida, la fase di anomaly detection viene eseguita localmente dai detector descritti nelle sezioni precedenti, mentre l'LLM interviene successivamente sulle finestre selezionate per produrre una descrizione del comportamento osservato e assegnare un livello di priorità. Questa impostazione viene confrontata anche con una pipeline nella quale l'LLM riceve direttamente tutte le finestre da analizzare. La struttura delle due pipeline e le informazioni fornite al modello linguistico sono descritte nel Capitolo 4.

2.6 Rischi e limiti nell'impiego degli LLM

L'impiego dei Large Language Model in attività di sicurezza presenta alcuni limiti che devono essere considerati quando le risposte prodotte dal modello vengono utilizzate per supportare il processo di analisi. Tra questi rientrano la generazione di informazioni non corrette, gli attacchi di prompt injection e i problemi legati alla riservatezza delle informazioni elaborate.

Uno dei principali limiti degli LLM è rappresentato dalle *hallucinations*. Con questo termine si indicano risposte generate dal modello che possono apparire plausibili, ma che contengono informazioni non supportate dai fatti o dal contesto disponibile [20]. La forma convincente e linguisticamente corretta delle risposte può rendere questi errori difficili da riconoscere, soprattutto quando l'output viene utilizzato come supporto a una decisione.

Huang et al. distinguono due categorie principali di allucinazione. Le *factuality hallucinations* riguardano contenuti in contrasto con fatti verificabili oppure affermazioni che non possono essere verificate. Le *faithfulness hallucinations* si verificano invece quando la risposta non è coerente con le istruzioni ricevute, con il contesto fornito al modello o con il contenuto della risposta stessa [20].

Nel contesto del triage di sicurezza, questo problema è rilevante perché l'LLM può ricevere informazioni relative a eventi, processi e risultati prodotti dai detector e generare a partire da essi una descrizione o una priorità. Una risposta

non coerente con i dati forniti potrebbe quindi portare a una rappresentazione errata dell'evento analizzato. Per questo motivo, l'output dell'LLM deve essere considerato come un supporto all'analisi e non come una fonte indipendente di evidenza.

Un ulteriore rischio riguarda gli attacchi di *prompt injection*. In questo caso, un contenuto malevolo può introdurre istruzioni che deviano l'LLM dal compito che dovrebbe svolgere, modificando il comportamento o l'output previsto dall'applicazione.

Greshake et al. analizzano in particolare la *indirect prompt injection*, che può verificarsi quando un LLM è integrato in un'applicazione e utilizza dati provenienti da fonti esterne. Un attaccante può inserire istruzioni malevole all'interno di dati che verranno successivamente recuperati ed elaborati dal modello. In questo modo, il contenuto ricevuto come dato può essere interpretato dall'LLM anche come un'istruzione, rendendo meno netta la separazione tra dati e comandi. Gli autori mostrano che questo tipo di attacco può alterare il comportamento dell'applicazione e influenzare il contenuto prodotto dal modello. Tra gli effetti analizzati rientrano, ad esempio, la generazione di riassunti volutamente errati, la sottrazione di informazioni e, nei sistemi in cui il modello può interagire con strumenti o API, l'esecuzione di azioni non previste [21].

Un ulteriore aspetto da considerare riguarda la riservatezza dei dati elaborati dagli LLM. Nel contesto della cybersecurity, infatti, i dati possono avere natura sensibile o proprietaria. Habibzadeh et al. osservano che il loro trattamento mediante servizi LLM esterni può introdurre problemi di privacy e sicurezza, soprattutto quando le informazioni vengono trasmesse a servizi di terze parti attraverso API pubbliche. In questi casi, tra i problemi evidenziati rientrano il rischio di *data leakage* e di accesso non autorizzato alle informazioni. Gli autori sottolineano inoltre che l'impiego di servizi LLM commerciali può porre ulteriori difficoltà legate alla gestione e alla localizzazione dei dati e alla trasparenza del loro trattamento [22].

Per gli ambienti nei quali la riservatezza dei dati rappresenta un requisito particolarmente importante, Habibzadeh et al. suggeriscono di preferire, quando possibile, soluzioni eseguite *on-premises* o in infrastrutture cloud controllate

rispetto all'utilizzo di servizi LLM di terze parti tramite API, con l'obiettivo di ridurre il rischio di esposizione dei dati [22].

Capitolo 3

Il dataset BETH

Il dataset utilizzato in questo lavoro è BETH, acronimo di *BPF-Extended Tracking Honeypot*. Il dataset è stato presentato da Highnam et al. per lo studio dell'anomaly detection non supervisionata e dell'analisi *out-of-distribution* in ambito cybersecurity [1].
In questo capitolo vengono descritte le principali caratteristiche di BETH, le modalità con cui i dati sono stati raccolti, la struttura dei log e il significato delle etichette disponibili. Le operazioni di preprocessing su di un suo sottoinsieme e la successiva costruzione delle finestre temporali sono invece descritte nel Capitolo 4.

3.1 Origine e caratteristiche generali

La raccolta completa descritta dagli autori comprende 8 004 918 eventi ottenuti da 23 honeypot indipendenti, eseguiti complessivamente per circa cinque ore non contigue su un'infrastruttura cloud. Ciascun host contiene attività benigna e, al più, un singolo attacco. Questa organizzazione rende più semplice distinguere il comportamento ordinario del sistema da quello associato a una compromissione [1].
Rispetto ad alcuni dataset di cybersecurity precedentemente utilizzati nella letteratura,come KDD Cup 99, NSL-KDD e UNSW-NB15, BETH contiene attività host più recente e dati raccolti in un ambiente cloud. Il dataset comprende inoltre sia log relativi ai processi a livello kernel sia log di rete, consentendo di osservare aspetti differenti dell'attività svolta sugli host [1].

Per i benchmark, gli autori non utilizzano l'intera raccolta di oltre otto milioni di eventi, ma selezionano un sottoinsieme dei log relativi ai processi. Tale sottoinsieme contiene 1 141 078 eventi provenienti da 13 host ed è suddiviso in training, validation e testing.

3.2 Raccolta dei dati e architettura di monitoraggio

Per la raccolta dei dati, gli autori di BETH hanno utilizzato una serie di host configurati come honeypot, cioè sistemi predisposti per attirare e registrare attività potenzialmente malevola. Gli host utilizzati nella raccolta eseguono Ubuntu 18.04 e includono un container Docker nel quale è integrato il sistema di monitoraggio.

Il monitoraggio è basato su eBPF (*extended Berkeley Packet Filter*), una tecnologia utilizzata dagli autori per realizzare due sensori distinti. Il primo registra eventi relativi all'attività dei processi, in particolare chiamate associate alla creazione, alla clonazione e alla terminazione dei processi. Il secondo registra invece le query e le risposte DNS generate dai processi presenti sull'host, compresi quelli eseguiti all'interno dei container Docker.

Gli eventi raccolti vengono trasmessi a un nodo separato utilizzato come server di raccolta e successivamente memorizzati in log distinti ma correlati, relativi rispettivamente all'attività dei processi e al traffico DNS [1].

Nel presente lavoro, così come nel lavoro originale, viene utilizzata esclusivamente la componente relativa ai log dei processi. In particolare, l'analisi sperimentale si basa sui tre file etichettati labelled_training_data.csv, `labelled_validation_data.csv` e `labelled_testing_data.csv`
che contengono il sottoinsieme kernel-process suddiviso in training, validation e test. I log DNS disponibili nella raccolta BETH non vengono invece impiegati nell'analisi sperimentale.

3.3 Struttura dei log kernel-process

I log relativi ai processi rappresentano la componente di BETH utilizzata nel presente lavoro. Ogni evento è descritto da 14 caratteristiche e da due etichette binarie, `sus` ed `evil` [1].

Tra le informazioni disponibili è presente innanzitutto il `timestamp`, che indica il tempo trascorso dall'avvio del sistema. Sono inoltre presenti diversi identificativi relativi al contesto del processo, tra cui `processId`, `threadId`, `parentProcessId` e `userId`. Il campo `mountNamespace` identifica invece il namespace di montaggio nel quale opera il processo, mentre `hostName` identifica l'host dal quale proviene il log.

L'evento registrato è descritto attraverso i campi `eventId` ed `eventName`. Il primo contiene l'identificativo numerico associato all'evento, mentre il secondo ne contiene il nome. Nel dataset originale esiste una corrispondenza univoca tra `eventName` ed `eventId`. Sono inoltre disponibili il nome del processo (`processName`), il numero di argomenti associati all'invocazione del processo (`argsNum`) e il loro valore (`args`), il valore restituito (`returnValue`) e gli indirizzi dello stack (`stackAddresses`).

Le ultime due colonne, `sus` ed `evil`, non rappresentano caratteristiche dell'evento utilizzate per descriverne il comportamento, ma etichette aggiunte dagli autori per distinguere attività sospetta e attività riconducibile a una presenza malevola. Il loro significato viene approfondito nella sezione successiva.

3.4 Etichette `sus` ed `evil`

I record di BETH sono associati a due etichette binarie, `sus` ed `evil`, introdotte dagli autori per supportare l'analisi dei risultati ottenuti dai modelli.

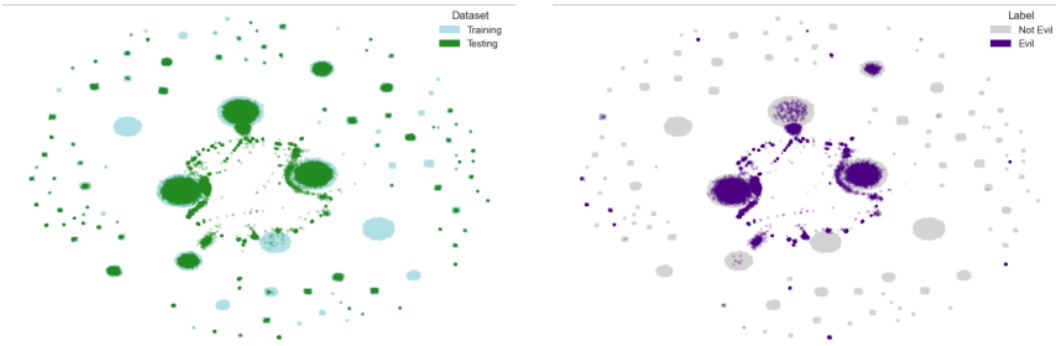
L'etichetta `sus` identifica eventi considerati sospetti, cioè attività insolite o poco frequenti rispetto al comportamento normalmente osservato nel sistema. Tra gli esempi riportati dagli autori rientrano l'utilizzo di determinati processi da parte di utenti esterni, chiamate poco frequenti effettuate da processi di sistema e operazioni di chiusura di processi la cui creazione non era stata precedentemente osservata.

L'etichetta `evil` identifica invece eventi associati a una presenza malevola esterna al sistema. Gli autori riportano, ad esempio, comandi utilizzati per ottenere informazioni sulla macchina, modificare l'accesso SSH o estrarre file precedentemente caricati. Gli eventi marcati come `evil` vengono considerati *out-of-distribution*, poiché derivano da un comportamento non presente nei dati utilizzati per l'addestramento [1].
Le due etichette sono state assegnate manualmente. Gli stessi autori specificano che tale annotazione è stata effettuata da un singolo revisore e che, di conseguenza, non dovrebbe essere considerata da sola come riferimento definitivo per sistemi destinati a contesti operativi reali.

3.5 Suddivisione del dataset e sottoinsieme utilizzato

Come detto in precedenza, nei benchmark gli autori hanno selezionato un sottoinsieme dei log relativi ai processi, suddividendolo in training, validation e testing. La suddivisione è approssimativamente pari a 60/20/20 ed è definita tenendo conto degli host, della quantità di eventi registrati e dell'attività osservata.
Il training set contiene 763 144 eventi provenienti da 8 host, mentre il validation set comprende 188 967 eventi provenienti da 4 host. Il test set contiene anch'esso 188 967 eventi, ma proviene da un singolo host. Complessivamente, il sottoinsieme utilizzato per il benchmark è quindi costituito da 1 141 078 eventi raccolti da 13 host.
Gli autori specificano che i dati di training e validation provengono da host nei quali non sono stati osservati segni di compromissione. L'attività registrata in questi insiemi è quindi considerata rappresentativa del comportamento normale del sistema. Il test set, invece, contiene l'attività di un singolo host compromesso e include anche eventi associati ad attività malevola [1].
Nel presente lavoro vengono utilizzati gli stessi tre file etichettati forniti per questa suddivisione: training, validation e testing. La successiva trasformazione di tali eventi in finestre temporali e la costruzione delle feature utilizzate dai modelli vengono descritte nel Capitolo 4.

Gli autori rappresentano inoltre i dati di training e testing mediante UMAP, una tecnica che consente di proiettare dati caratterizzati da molte variabili in uno spazio bidimensionale. La visualizzazione mostra che una parte dei dati di test presenta caratteristiche simili a quelle osservate nel training, mentre alcuni eventi etichettati come `evil` occupano aree differenti della rappresentazione.



(a) Fitted on Training and transformed Testing

(b) Same as (a), coloured with `evil` labels

Figura 3.1: Visualizzazione UMAP dei dati di training e testing di BETH, ottenuta dagli autori utilizzando il proprio preprocessing. Riprodotta da Highnam et al. [1], licenza CC BY 4.0.

3.6 Scenario di attacco nel test set

Il test set contiene l'attività registrata su un singolo honeypot compromesso. Secondo l'analisi riportata dagli autori, la sequenza osservata è riconducibile al tentativo di configurare la macchina come nodo di una botnet.

Una botnet è una rete composta da dispositivi compromessi e controllati da un soggetto esterno. I singoli dispositivi infetti, spesso indicati come *bot*, possono ricevere istruzioni da un'infrastruttura di *Command and Control* (C2). Il C2 rappresenta quindi il punto attraverso il quale l'attaccante può coordinare l'attività delle macchine compromesse, inviando comandi o ricevendo informazioni da esse.

Nel caso descritto dagli autori, dopo l'accesso iniziale al server vengono eseguite alcune operazioni di configurazione dell'ambiente e vengono osservate comunicazioni riconducibili al C2. Successivamente viene utilizzato SFTP (*SSH File Transfer Protocol*), un protocollo per il trasferimento di file attraverso una con-

nessione SSH, per scaricare sul sistema un archivio denominato `dota3.tar.gz`, identificato dagli autori come malware associato a una botnet.

Prima dell'esecuzione del malware vengono inoltre eseguiti alcuni comandi tipici di una fase di ricognizione del sistema. Per esempio, in un sistema Unix/Linux-based, il comando `whoami` permette di conoscere l'utente con cui è in esecuzione la sessione, `ls` consente di visualizzare il contenuto di una directory, mentre `cat /proc/cpuinfo` permette di ottenere informazioni relative alla CPU della macchina. Queste operazioni consentono quindi di raccogliere informazioni sull'ambiente compromesso prima di procedere con le fasi successive dell'attacco.

Dopo alcuni minuti, l'archivio `dota3.tar.gz` viene estratto e il programma contenuto al suo interno viene eseguito. Questa operazione segue una fase preparatoria nella quale erano già state osservate comunicazioni riconducibili al Command and Control, il trasferimento dell'archivio tramite SFTP e alcuni comandi di ricognizione del sistema, come `whoami`, `ls` e `cat /proc/cpuinfo`. Con l'esecuzione del programma il comportamento dell'host cambia sensibilmente: vengono generati oltre cento thread che tentano di stabilire connessioni verso server differenti. La creazione di un numero così elevato di thread e i numerosi tentativi di connessione coincidono con un forte incremento dell'attività registrata nella parte finale della sequenza temporale. Rispetto alle operazioni precedenti, prevalentemente orientate alla configurazione dell'ambiente, al trasferimento del file e alla raccolta di informazioni sul sistema, questa fase corrisponde all'esecuzione attiva del malware.

Nel complesso, la sequenza osservata passa quindi dall'accesso e dalla preparazione dell'host al trasferimento e all'avvio del programma malevolo. Gli autori riconducono questo comportamento al tentativo di configurare la macchina compromessa come nodo di una botnet [1].

La sequenza temporale dell'attacco è riportata in Figura 3.2. Il grafico mostra l'evoluzione del numero di eventi nel tempo: dopo l'accesso iniziale e le operazioni di configurazione sono presenti intervalli con minore attività, seguiti da nuove interazioni con il sistema. Nella parte finale della sequenza si osserva invece un aumento dell'attività associato all'esecuzione del malware e ai numerosi tentativi di connessione verso sistemi differenti.

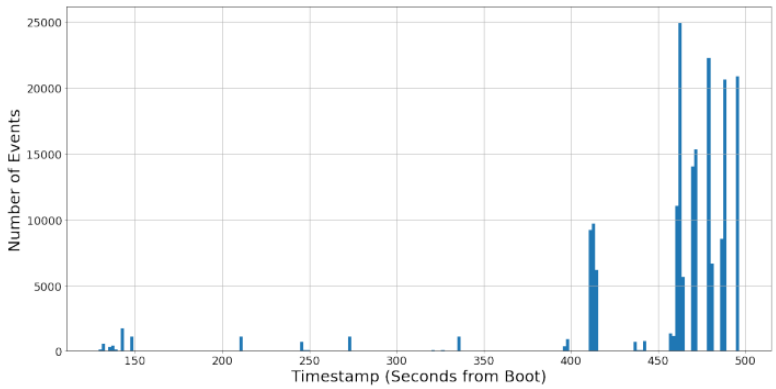


Figura 3.2: Timeline dell'attacco osservato nel test set di BETH. Riprodotta da Highnam et al. [1], licenza CC BY 4.0.

3.7 Lavori correlati

Il lavoro originale di Highnam et al. introduce BETH e propone una prima valutazione di tecniche di anomaly detection non supervisionata sui log kernel-process del dataset. Gli autori confrontano Robust Covariance, One-Class SVM, Isolation Forest e un approccio basato su Variational Autoencoder e Density of States Estimation, utilizzando l'AUROC come metrica di valutazione e un ensemble di cinque modelli per ciascun metodo [1].

Il dataset dispone di due etichette binarie, `sus` ed `evil`. La prima identifica eventi considerati sospetti o anomali, mentre la seconda è associata ad attività riconducibile a una presenza malevola esterna. Nel benchmark riportato dagli autori, tuttavia, il confronto quantitativo principale viene effettuato rispetto all'etichetta `sus`, distinguendo gli eventi sospetti da quelli benigni. In questo scenario, Isolation Forest ottiene il valore di AUROC più elevato, pari a 0.850, seguito dall'approccio VAE+DoSE-SVM con 0.698, dalla One-Class SVM con 0.605 e dalla Robust Covariance con 0.519.

L'etichetta `evil` è comunque presente nel dataset e identifica eventi considerati out-of-distribution; tali eventi compaiono nel test set, ma il lavoro originale non riporta, nella tabella dei benchmark, una AUROC separata calcolata specificamente rispetto a questa etichetta.

Il dataset è stato successivamente utilizzato anche per finalità diverse dalla valutazione diretta di algoritmi di anomaly detection. Doğanay et al. studiano BETH nell'ambito della visualizzazione di dati di cybersecurity, applicando la

Principal Component Analysis (PCA) a un sottoinsieme del test set [3].
Gli autori costruiscono un campione di 800 eventi a partire dal file `labelled_testing_data.csv`. In particolare, vengono selezionati 200 eventi associati al processo `dpkg` con `sus`=0 ed `evil`=0, 300 eventi `sshd` con `sus`=1 ed `evil`=1 e 300 eventi `systemd` con `sus`=1 ed `evil`=0. La scelta di utilizzare 800 campioni riprende un precedente studio di visualizzazione condotto sul dataset KDD99, utilizzato dagli autori come termine di confronto.
Prima dell'applicazione della PCA, i dati numerici vengono standardizzati mediante `StandardScaler`, così da ottenere variabili con media pari a zero e varianza unitaria. La PCA viene quindi utilizzata per rappresentare i dati mediante due componenti principali, indicate come PC1 e PC2, consentendone la visualizzazione su un piano bidimensionale. Gli autori utilizzano colori e simboli differenti per evidenziare i tre gruppi selezionati e riportano una separazione visiva netta tra essi. La PCA riduce quindi i dati a due componenti principali, PC1 e PC2, utilizzate come coordinate per la rappresentazione bidimensionale. Nel sottoinsieme considerato, la prima componente rappresenta il 49.09% della varianza dei dati, mentre la seconda ne rappresenta il 32.84%. Complessivamente, le due componenti utilizzate nella visualizzazione rappresentano quindi circa l'81.9% della varianza totale. Il lavoro mostra quindi un utilizzo di BETH orientato principalmente all'analisi esplorativa e alla visualizzazione dei dati tramite una riduzione della dimensionalità, piuttosto che alla costruzione e valutazione di un modello di anomaly detection [3].
Più recentemente, Alatawi e Albalawi hanno ripreso il sottoinsieme kernel-process di BETH con l'obiettivo di migliorare le prestazioni ottenute dall'Isolation Forest nel lavoro originale. Lo studio mantiene la suddivisione del dataset in training, validation e testing [4].
Prima dell'addestramento, gli autori conducono un'analisi esplorativa dei dati per esaminare la distribuzione degli eventi e delle etichette nei tre insiemi. Vengono analizzati, tra gli altri aspetti, la sequenza temporale degli eventi nel test set, la distribuzione delle combinazioni di `sus` ed `evil` e le correlazioni tra le caratteristiche numeriche. Tale analisi evidenzia una differenza rilevante tra gli insiemi: training e validation non contengono eventi marcati contemporaneamente come `sus`=1 ed `evil`=1, mentre nel test set è presente una quantità

consistente di eventi appartenenti a questa categoria. Gli autori sottolineano pertanto la forte differenza nella distribuzione delle classi tra i dati utilizzati per l'addestramento e quelli impiegati nella valutazione finale.

Per l'addestramento dell'Isolation Forest vengono selezionate sette caratteristiche dei log: `processId`, `parentProcessId`, `userId`, `mountNamespace`, `eventId`, `argsNum` e `returnValue`. Le caratteristiche vengono sottoposte a una fase di trasformazione e successivamente standardizzate mediante `StandardScaler`. I parametri di standardizzazione vengono stimati esclusivamente sul training set e quindi riutilizzati per trasformare validation e test. Il modello è implementato mediante la classe `IsolationForest` di scikit-learn, utilizzando 100 alberi, un valore di `contamination` pari a 0.01 e `randomState`=42.

Dopo l'addestramento sul training set, il modello viene verificato su un insieme di validation separato. Gli autori utilizzano un valore minimo di AUROC pari a 0.55 come criterio per procedere alla successiva valutazione sul test set. Le prestazioni finali vengono quindi analizzate attraverso AUROC, precision, recall, F1-score, accuracy e matrice di confusione. A differenza del lavoro originale, che presenta principalmente l'AUROC dei modelli considerati, questo studio riporta quindi anche metriche dipendenti da una decisione binaria sulle anomalie.

Come etichetta di riferimento per la valutazione viene utilizzata `sus`. Nel test set, pertanto, vengono considerati positivi tutti gli eventi con `sus`=1, compresi quelli per i quali anche `evil`=1. Lo studio non presenta una valutazione separata rispetto alla sola etichetta `evil` [4].

Per quanto riguarda la capacità discriminante del modello, gli autori riportano un AUROC pari a 0.917 e confrontano tale risultato con il valore di 0.850 ottenuto dall'Isolation Forest nel benchmark originale di BETH. Il lavoro viene quindi presentato dagli autori come un miglioramento del risultato iniziale ottenuto sul dataset. Il confronto deve tuttavia essere interpretato considerando le differenti scelte sperimentali adottate, tra cui la selezione e trasformazione delle caratteristiche, la standardizzazione dei dati e la specifica configurazione dell'Isolation Forest. I due valori costituiscono pertanto riferimenti utili per valutare l'evoluzione degli esperimenti condotti su BETH, ma non derivano da due procedure sperimentali completamente identiche.

Un'altra direzione di ricerca è rappresentata dall'elaborazione dei log BETH come flusso di eventi. Eremin analizza dieci algoritmi di anomaly detection online o semi-online, utilizzando esclusivamente i log relativi ai processi e non la componente DNS del dataset. Tra i metodi considerati sono presenti IForestASD, Incremental Local Outlier Factor, una versione stocastica di One-Class SVM, KitNet, RS-Hash, xStream e altri algoritmi progettati per l'elaborazione di dati in streaming. La valutazione viene effettuata sul test set separatamente rispetto alle etichette `sus` ed `evil`, utilizzando come metriche ROC-AUC e tempo di elaborazione [2].

Tra le configurazioni sperimentate, KitNet raggiunge i valori di ROC-AUC più elevati, pari a 0.990 rispetto all'etichetta `sus` e a 0.994 rispetto all'etichetta `evil`. Anche altri metodi, tra cui Incremental LOF, RS-Hash e xStream, ottengono valori elevati in alcune configurazioni. Gli esperimenti mostrano tuttavia che le prestazioni possono cambiare in funzione dell'ordine con cui vengono presentati gli eventi e delle caratteristiche utilizzate. In particolare, l'ordinamento per timestamp e host e l'arricchimento delle feature producono effetti differenti a seconda del modello e dell'etichetta considerata [2].

Questi risultati evidenziano come, anche utilizzando lo stesso dataset, il protocollo di elaborazione e la rappresentazione dei dati possano influire in modo significativo sulle prestazioni ottenute. I valori riportati da Eremin non sono pertanto direttamente confrontabili con quelli del presente lavoro, che adotta una rappresentazione basata su finestre temporali e frequenze relative degli eventi.

I lavori considerati mostrano differenti modalità di utilizzo di BETH. Doğanay et al. impiegano un sottoinsieme del test set per la visualizzazione mediante Principal Component Analysis [3]; Alatawi e Albalawi utilizzano invece i log kernel-process per valutare un approccio di anomaly detection basato su Isolation Forest [4]. Eremin considera infine BETH in uno scenario di elaborazione a flusso, confrontando dieci algoritmi di anomaly detection online e semi-online [2].

Il presente lavoro adotta una rappresentazione differente dei log kernel-process: gli eventi vengono aggregati in finestre temporali e rappresentati attraverso le frequenze relative dei diversi tipi di evento. Su tale rappresentazione vengo-

no confrontati più modelli di anomaly detection e un ensemble; alla fase di rilevazione viene inoltre affiancata una successiva fase di triage assistita da LLM.

Capitolo 4

Metodologia

4.1 Obiettivo della metodologia

L'obiettivo della metodologia proposta è analizzare eventi di telemetria kernel reale mediante tecniche di anomaly detection non supervisionata e valutare l'impiego di un Large Language Model (LLM) come supporto alla fase di analisi e triage delle finestre considerate anomale. Il flusso sperimentale parte dai dati del dataset BETH, che vengono trasformati da una sequenza di eventi kernel a una rappresentazione aggregata per finestre temporali. Per ogni finestra viene costruito un vettore di feature basato sulla frequenza relativa dei diversi tipi di evento osservati. Tale rappresentazione consente di applicare modelli di anomaly detection a dati originariamente organizzati come sequenze di eventi. La rilevazione delle anomalie viene effettuata mediante quattro approcci: Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder. I punteggi di anomalia prodotti dai singoli modelli vengono inoltre combinati attraverso un ensemble, dopo una fase di normalizzazione che consente di rendere confrontabili score appartenenti a scale differenti. Le soglie operative vengono definite utilizzando i dati di training e validation, mantenendo il test set esclusivamente per la valutazione finale. La metodologia comprende inoltre due differenti modalità di utilizzo dell'LLM. Nella prima, indicata come pipeline *detector + LLM*, i modelli tradizionali eseguono la fase iniziale di anomaly detection e soltanto le finestre selezionate vengono successivamente inviate all'LLM, che svolge un compito di triage e contestualizzazione dell'alert. Nella seconda configurazione, indicata come pipeline *LLM-only*, l'LLM riceve diret-

tamente la rappresentazione comportamentale di tutte le finestre del test set e deve svolgere autonomamente sia la decisione di anomalia sia la successiva prioritizzazione. In questo caso non vengono fornite al modello informazioni provenienti dai detector tradizionali, né le etichette utilizzate successivamente per la valutazione. Il confronto tra le due pipeline ha quindi lo scopo di analizzare due aspetti distinti: da un lato la capacità di individuare le finestre annotate come sospette nel dataset, dall'altro il costo operativo delle due architetture in termini di latenza e risorse computazionali osservabili. In questo modo è possibile valutare il ruolo dell'LLM sia come componente successivo a un sistema di anomaly detection tradizionale, sia come strumento utilizzato direttamente per l'analisi della telemetria.

4.2 Dataset BETH e suddivisione dei dati

L'analisi sperimentale è stata condotta utilizzando il dataset BETH, già descritto nel capitolo precedente. Ai fini della metodologia sono stati utilizzati i tre file etichettati forniti dal dataset: `labelled_training_data.csv`, `labelled_validation_data.csv` e `labelled_testing_data.csv`. La suddivisione originale in training, validation e test è stata mantenuta durante l'intero esperimento, evitando di ricampionare o riassegnare gli eventi tra i diversi insiemi. Il training set contiene 763 144 eventi, mentre validation e test set contengono entrambi 188 967 eventi. Ciascun record è descritto da 16 attributi, comprendenti informazioni temporali, identificativi di processo e thread, utente, namespace, nome del processo e dell'host, tipo di evento, argomenti e valore di ritorno. Sono inoltre presenti le due annotazioni `sus` ed `evil`, utilizzate dal dataset per identificare rispettivamente eventi sospetti e malevoli. Le etichette non vengono utilizzate come feature di ingresso dei modelli di anomaly detection. I detector vengono infatti addestrati secondo un'impostazione non supervisionata, mentre le annotazioni vengono conservate separatamente per la selezione delle soglie operative sul validation set e per la successiva valutazione delle decisioni prodotte sul test set. La distribuzione delle etichette non è uniforme nei tre insiemi. Nel training set sono presenti 761 875 eventi con `sus`=0 e `evil`=0 e 1 269 eventi con `sus`=1, mentre non sono presenti eventi

marcati come `evil`. Anche il validation set non contiene eventi `evil`: 188 181 eventi non presentano nessuna delle due annotazioni e 786 risultano `sus`. Il test set contiene invece sia eventi sospetti sia eventi malevoli, permettendo di effettuare la valutazione finale della pipeline. Questa caratteristica della suddivisione ha una conseguenza metodologica importante. L'etichetta `sus` può essere utilizzata sul validation set per selezionare una soglia operativa senza accedere alle etichette del test set. Non è invece possibile definire con la stessa procedura una soglia specifica per `evil`, poiché nel validation set non sono presenti esempi positivi per tale etichetta. Per questo motivo, nel seguito della metodologia, `sus` viene utilizzata come riferimento principale per la selezione delle soglie e per il confronto tra le pipeline, mentre `evil` viene mantenuta per un'analisi descrittiva a posteriori.

4.3 Preprocessing e costruzione delle finestre temporali

Prima della costruzione delle feature, gli eventi di ciascuno split sono stati ordinati in base al campo `timestamp`, in modo da preservarne la sequenza temporale. Training, validation e test sono stati trattati come registrazioni indipendenti e, di conseguenza, il processo di suddivisione in finestre è stato applicato separatamente a ciascun insieme.

Per ogni split è stato calcolato un tempo relativo rispetto al primo evento osservato e gli eventi sono stati successivamente raggruppati in finestre temporali non sovrapposte. Questa scelta consente di trasformare il flusso originale di eventi kernel in una sequenza di intervalli temporali utilizzabili come singole osservazioni dai modelli di anomaly detection.

La durata della finestra è stata scelta dopo aver esplorato diverse granularità temporali, pari a 5, 10, 30, 60 e 120 secondi. Finestre molto brevi permettono di ottenere un numero maggiore di osservazioni, ma tendono a frammentare maggiormente l'attività del sistema. Al contrario, finestre più ampie aggregano un numero elevato di eventi ma riducono sensibilmente il numero di campioni disponibili.

Nel training set, ad esempio, una finestra di 60 secondi produce 64 osservazioni, mentre una finestra di 120 secondi ne produce soltanto 32. La durata di 30 secondi è stata quindi scelta come compromesso tra granularità temporale e numerosità del campione, producendo 128 finestre nel training set.

Applicando la stessa procedura a validation e test sono state ottenute rispettivamente 22 e 13 finestre. Il risultato del preprocessing è quindi una rappresentazione in cui ogni riga corrisponde a una finestra temporale di 30 secondi, sulla quale vengono successivamente costruite le feature utilizzate dai modelli.

Come verifica supplementare, è stata inoltre effettuata un'analisi di sensibilità rispetto alla durata delle finestre temporali. In particolare, sono state considerate finestre non sovrapposte di 5, 10, 30, 60 e 120 secondi. Tale analisi non è stata utilizzata per modificare retrospettivamente la configurazione principale dell'esperimento, fissata a 30 secondi, ma per osservare quanto il comportamento dei detector dipenda dalla granularità temporale adottata.

Per ciascuna durata è stata applicata la stessa procedura di costruzione delle feature, mantenendo i 32 tipi di `eventName` definiti sul training set e rappresentando ogni finestra mediante le corrispondenti frequenze relative. Le configurazioni dei detector sono state mantenute invariate rispetto all'esperimento principale. Per ogni granularità, la standardizzazione è stata nuovamente adattata esclusivamente sul training set e applicata successivamente a validation e test.

Poiché la distribuzione degli anomaly score varia al modificarsi della rappresentazione temporale, le soglie numeriche non sono state mantenute fisse. Per ciascuna durata sono stati invece ricalcolati sul training set i percentili candidati 90, 95, 97 e 99, applicando quindi lo stesso protocollo dell'esperimento principale: la soglia operativa è stata selezionata sul validation set utilizzando l'F1-score rispetto all'etichetta SUS. Il test set è rimasto escluso dalla selezione delle soglie ed è stato utilizzato esclusivamente per la valutazione finale.

L'analisi di sensibilità è stata limitata ai quattro detector individuali, Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder. L'Ensemble e gli stadi basati su LLM non sono stati rieseguiti per le diverse granularità, poiché l'obiettivo di questa verifica è isolare l'effetto della durata della finestra sul comportamento dei detector di base.

Le etichette `sus` ed `evil`, originariamente associate ai singoli eventi, sono state aggregate a livello di finestra. Una finestra viene marcata come `sus_window` o `evil_window` quando contiene almeno un evento con la corrispondente annotazione positiva. Tali variabili non vengono utilizzate come input dei modelli, ma vengono mantenute separatamente per la selezione delle soglie e per la successiva valutazione sperimentale.

4.4 Feature engineering

A partire dalle finestre temporali costruite nella fase precedente, gli eventi kernel sono stati trasformati in una rappresentazione numerica adatta ai modelli di anomaly detection.

La rappresentazione scelta è basata sul campo `eventName`. La scelta di utilizzare eventName è motivata dall'obiettivo di rappresentare la composizione dell'attività osservata in ciascuna finestra e di verificare se la presenza o la maggiore frequenza di specifici tipi di evento sia associata a comportamenti anomali. Per questo motivo, ciascun tipo di evento osservato nel training set viene utilizzato come feature della rappresentazione. Nel training set sono presenti 32 tipi distinti di evento e ciascuno di essi è stato utilizzato come una feature. Ogni finestra temporale viene quindi rappresentata attraverso il numero di occorrenze dei diversi tipi di evento osservati al suo interno, ottenendo una matrice in cui le righe corrispondono alle finestre e le colonne ai diversi `eventName`.

L'insieme delle feature è stato definito esclusivamente a partire dal training set e successivamente mantenuto invariato per validation e test. In questo modo tutti gli split condividono la stessa rappresentazione e non vengono introdotte nuove colonne sulla base di informazioni osservate soltanto nei dati utilizzati nelle fasi successive dell'esperimento. Eventuali tipi di evento presenti esclusivamente in validation o test non vengono quindi trasformati in nuove feature.

Dopo l'allineamento alle 32 feature definite sul training set, i conteggi di ciascuna finestra sono stati normalizzati rispetto alla somma dei conteggi delle feature mantenute. Le feature finali corrispondono quindi alle frequenze re-

lative dei 32 tipi di evento considerati nella rappresentazione della finestra. In questo modo la rappresentazione descrive principalmente la composizione dell'attività osservata, invece di dipendere soltanto dal volume degli eventi associati alle feature utilizzate. Per ogni finestra, la somma delle 32 frequenze relative è pari a 1.

Al termine di questa fase vengono ottenute tre matrici con la stessa struttura: 128 finestre e 32 feature per il training set, 22 finestre e 32 feature per il validation set e 13 finestre e 32 feature per il test set. Queste matrici costituiscono l'input utilizzato successivamente dai modelli di anomaly detection.

4.5 Tecniche di anomaly detection

Sui dati sono state applicate quattro tecniche di anomaly detection: Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder. La scelta permette di confrontare approcci basati su principi differenti, mantenendo tuttavia comune la rappresentazione dei dati in ingresso.

L'implementazione sperimentale è stata realizzata in Python. Per la standardizzazione dei dati e per i modelli Isolation Forest, Local Outlier Factor e One-Class SVM è stata utilizzata la libreria Scikit-Learn, mentre l'Autoencoder è stato implementato mediante TensorFlow e l'API Keras.

Prima dell'addestramento, le 32 feature sono state standardizzate mediante `StandardScaler`. Le feature rappresentano frequenze relative dei diversi tipi di evento e, pur essendo espresse nella stessa scala, possono presentare distribuzioni e variabilità differenti. Alcuni tipi di evento possono infatti essere molto frequenti, mentre altri compaiono soltanto in misura ridotta.

La standardizzazione viene applicata separatamente a ciascuna feature. Indicando con μ la media e con σ la deviazione standard calcolate sul training set, ogni valore x viene trasformato come

$$z = \frac{x - \mu}{\sigma}.$$

Il valore ottenuto esprime quindi lo scostamento della feature rispetto al proprio comportamento medio nel training set, in rapporto alla sua variabilità. In questo modo le diverse feature vengono riportate su una scala più omoge-

nea, evitando che differenze nella loro variabilità numerica influenzino in modo sproporzionato modelli basati sulle distanze, come LOF e One-Class SVM, o il calcolo dell'errore di ricostruzione dell'Autoencoder.
I parametri dello `StandardScaler` sono stati stimati esclusivamente sulle finestre del training set. Gli stessi valori di media e deviazione standard sono stati poi utilizzati per trasformare validation e test, senza effettuare un nuovo fitting.
I modelli vengono addestrati sulle feature del training set senza utilizzare le annotazioni `sus` ed `evil`. Per rendere più semplice il confronto tra algoritmi differenti, gli anomaly score sono inoltre orientati secondo una convenzione comune: valori più elevati indicano una maggiore anomalia della finestra.

4.5.1 Isolation Forest

Il primo algoritmo utilizzato è Isolation Forest, implementato tramite la classe `IsolationForest` di Scikit-Learn. Il numero di alberi è stato fissato a $n_{\text{estimators}} = 200$. Lo score prodotto da Isolation Forest dipende dalla lunghezza dei cammini osservati nei diversi alberi e viene calcolato aggregando le informazioni dell'intera foresta. Liu et al. osservano che tali valori tendono a stabilizzarsi all'aumentare del numero di alberi [9]. Nel presente esperimento è stato quindi utilizzato un insieme di 200 alberi come configurazione fissa del modello; tale valore non è stato selezionato mediante una ricerca sistematica dell'iperparametro.
Poiché la costruzione degli alberi comprende operazioni casuali, è stato impostato `random_state=42` per rendere riproducibile l'esperimento, mantenendo le stesse scelte casuali nelle diverse esecuzioni.
Il parametro `contamination` è stato mantenuto nella modalità `"auto"`. Tale parametro viene utilizzato da Scikit-Learn per stabilire la soglia interna con cui Isolation Forest distingue le osservazioni normali dagli outlier, senza richiedere di specificare manualmente una percentuale attesa di anomalie.
Nel presente lavoro, tuttavia, la decisione finale non è stata ottenuta direttamente da questa soglia interna. Sono stati utilizzati gli anomaly score continui prodotti dal modello e la soglia operativa è stata selezionata successivamente sul validation set.

Dopo il fitting, lo score è stato calcolato separatamente per training, validation e test mediante la funzione `decision_function`. Poiché la convenzione adottata dalla libreria assegna valori più bassi alle osservazioni maggiormente anomale, il segno dello score è stato invertito. In questo modo, anche per Isolation Forest, uno score crescente corrisponde a un maggiore grado di anomalia.

4.5.2 Local Outlier Factor

Il secondo modello ottenuto è stato realizzato utilizzando Local Outlier Factor (LOF), un algoritmo che individua le anomalie confrontando ciascuna osservazione con quelle presenti nel suo intorno. Nel nostro caso, ogni finestra temporale viene confrontata con finestre che presentano caratteristiche simili nello spazio delle 32 feature.

Il principale parametro del metodo è il numero di vicini utilizzati per definire il contesto locale dell'osservazione. Nel presente esperimento è stato impostato `n_neighbors=20`. Breunig et al. mostrano che la scelta della dimensione del vicinato influenza direttamente il valore del LOF e riportano, per i dataset da loro analizzati, valori compresi tra 10 e 20 come una scelta generalmente efficace [10]. Il valore 20 è stato quindi adottato come configurazione fissa del modello. Il parametro `contamination` è stato mantenuto nella configurazione `"auto"`.

È stato inoltre impostato `novelty=True`, in modo da poter utilizzare il modello, dopo il fitting sul training set, anche per assegnare uno score a osservazioni non utilizzate durante l'addestramento, appartenenti ai set di validation e test. LOF assegna a ciascuna finestra uno score che esprime quanto il suo comportamento locale differisca da quello delle osservazioni circostanti. Nel codice, lo score viene calcolato in modo diverso per il training e per i nuovi dati: per le osservazioni utilizzate durante il fitting viene impiegato `negative_outlier_factor_`, mentre per validation e test viene utilizzato `score_samples`.

Per mantenere la stessa convenzione adottata negli altri detector, il segno degli score viene infine invertito. Di conseguenza, anche per LOF, valori più elevati indicano finestre considerate maggiormente anomale.

4.5.3 One-Class SVM

Il terzo algoritmo utilizzato è una One-Class SVM con kernel RBF. La scelta di un kernel non lineare permette al modello di costruire una frontiera decisionale più flessibile rispetto a una separazione lineare nello spazio originale delle feature [5, 11].

Il kernel RBF dipende dal parametro `gamma`, che nel presente esperimento è stato mantenuto nella configurazione `scale`. Tale parametro non è stato sottoposto a una procedura di selezione dedicata; il confronto tra configurazioni è stato invece concentrato sul parametro ν.

Il parametro ν influenza la frontiera costruita dalla One-Class SVM e il grado di tolleranza verso osservazioni del training che rimangono al di fuori della regione considerata normale [11]. Per questo motivo sono state valutate più configurazioni del modello, utilizzando i valori

$$\nu \in \{0.01, 0.03, 0.05, 0.10, 0.15, 0.20\}.$$

L'insieme dei valori candidati è stato definito come parte della configurazione sperimentale e non deriva da una ricerca esaustiva dell'iperparametro.

La selezione è stata effettuata esclusivamente sul validation set, utilizzando come criterio la ROC-AUC rispetto all'etichetta SUS. La ROC-AUC permette infatti di valutare la capacità degli anomaly score di ordinare le osservazioni positive rispetto a quelle negative senza fissare in questa fase una specifica soglia decisionale [16]. La selezione della soglia operativa viene invece effettuata separatamente, una volta fissato il parametro ν ed addestrati i dati sul training set.

I valori di ν compresi tra 0.01 e 0.15 hanno prodotto sul validation set una ROC-AUC SUS pari a 0.500, mentre con $\nu = 0.20$ è stato ottenuto un valore pari a 0.600. Tra le configurazioni considerate è stato quindi selezionato $\nu = 0.20$. Il test set è rimasto escluso da questa procedura ed è stato utilizzato soltanto per la valutazione finale.

La scelta dell'iperparametro deve comunque essere interpretata con cautela, poiché il validation set contiene soltanto due finestre positive rispetto all'e-

tichetta SUS. Una volta selezionata la configurazione, il modello definitivo è stato addestrato esclusivamente sulle finestre del training set.
Gli anomaly score vengono ricavati dalla `decision_function`. Per mantenere la stessa convenzione adottata per gli altri detector, il segno dello score viene invertito: in questo modo valori più elevati corrispondono a finestre considerate maggiormente anomale.

4.5.4 Autoencoder

Come quarto approccio è stato utilizzato un Autoencoder, cioè una rete neurale addestrata a ricostruire i propri dati di ingresso. L'idea utilizzata per l'anomaly detection è che una finestra che differisce dai pattern appresi durante il training possa risultare più difficile da ricostruire e produrre quindi un errore maggiore. Nel presente lavoro l'Autoencoder riceve in ingresso le 32 feature associate a ciascuna finestra temporale. Considerata la ridotta numerosità del training set, costituito da 128 finestre, è stata adottata un'architettura compatta:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 32.$$

La riduzione da 32 a 16 e successivamente a 8 unità costituisce la fase di codifica. Il livello centrale da 8 unità rappresenta il *bottleneck*, cioè la rappresentazione a dimensionalità ridotta a partire dalla quale il decoder deve ricostruire le 32 feature originali.
I livelli nascosti utilizzano la funzione di attivazione ReLU. Il passaggio da 32 a 16 e poi a 8 dimensioni è determinato dal numero di neuroni dei diversi livelli, mentre la funzione di attivazione viene applicata ai valori prodotti da ciascun neurone. L'uso di una funzione ReLU permette all'Autoencoder di apprendere trasformazioni non lineari durante la costruzione della rappresentazione interna.
Il livello di uscita utilizza invece un'attivazione lineare. Anche prima dell'addestramento, le feature vengono trasformate mediante `StandardScaler`, che centra ciascuna feature rispetto alla media calcolata sul training set e la riscalata in funzione della deviazione standard. I valori così ottenuti non sono

vincolati a essere non negativi: valori inferiori alla media della corrispondente feature possono assumere un valore standardizzato negativo.

Poiché il decoder deve ricostruire le feature nello stesso spazio standardizzato utilizzato in ingresso, è stata utilizzata un'attivazione lineare nel livello di uscita. A differenza della ReLU, che pone a zero le uscite negative, l'attivazione lineare non impone un vincolo di positività ai valori ricostruiti.

Sul primo livello nascosto è stata inoltre applicata una regolarizzazione L2 con coefficiente 10^{-4}. Questa tecnica aggiunge alla funzione di perdita una penalizzazione associata alla grandezza dei pesi del livello, favorendo soluzioni con pesi più contenuti. La regolarizzazione è stata introdotta per limitare un adattamento eccessivo ai dati di training. Il valore del coefficiente è stato fissato nella configurazione sperimentale e non è stato selezionato mediante una ricerca sistematica degli iperparametri.

Come funzione di perdita è stato utilizzato il Mean Squared Error (MSE), coerentemente con l'obiettivo di ricostruzione dell'Autoencoder. Indicando con x_j la j-esima feature della finestra e con $\hat{x}_j$ il valore ricostruito, l'errore utilizzato è

$$\mathrm{MSE}(x, \hat{x}) = \frac{1}{32} \sum_{j=1}^{32} (x_j - \hat{x}_j)^2.$$

L'errore di ricostruzione viene successivamente utilizzato anche come anomaly score della finestra: valori maggiori corrispondono a una maggiore differenza tra il vettore osservato e quello ricostruito. L'utilizzo dell'errore di ricostruzione come misura di anomalia è adottato anche nei lavori di Sakurada e Yairi e di Chen et al. [12, 13].

L'ottimizzazione dei pesi è stata effettuata mediante Adam. L'ottimizzatore è stato mantenuto fisso nella configurazione sperimentale e non è stato confrontato con algoritmi di ottimizzazione alternativi.

L'addestramento è stato eseguito utilizzando gruppi di 16 finestre, indicati come *batch*. La rete elabora un batch alla volta: per ciascun gruppo calcola l'errore di ricostruzione e utilizza tale errore per aggiornare i propri pesi. Poiché il training set contiene 128 finestre, sono necessari 8 batch per elaborare una volta l'intero training set. Un passaggio completo su tutte le finestre costituisce un'epoca.

È stato impostato un limite massimo di 300 epoche. Per controllare l'andamento dell'addestramento è stato inoltre utilizzato un meccanismo di *Early Stopping*, che osserva la loss calcolata sul validation set. Con `patience=20`, l'addestramento può essere interrotto quando la validation loss non migliora per 20 epoche consecutive.

L'opzione `restore_best_weights=True` consente infine di ripristinare i pesi corrispondenti alla migliore validation loss osservata durante l'addestramento, invece di mantenere necessariamente quelli dell'ultima epoca eseguita. Nel presente esperimento la condizione di arresto anticipato non si è verificata e l'Autoencoder ha completato tutte le 300 epoche previste.

I valori relativi al batch size, al numero massimo di epoche, alla `patience` e al coefficiente di regolarizzazione sono stati fissati come parte della configurazione sperimentale e non sono stati ottenuti mediante una ricerca esaustiva degli iperparametri.

L'andamento della funzione di perdita durante l'addestramento è riportato in Figura 4.1. La funzione utilizzata dal modello è basata sull'errore quadratico medio, o Mean Squared Error (MSE), al quale si aggiunge il contributo della regolarizzazione L2 applicata al primo livello nascosto.

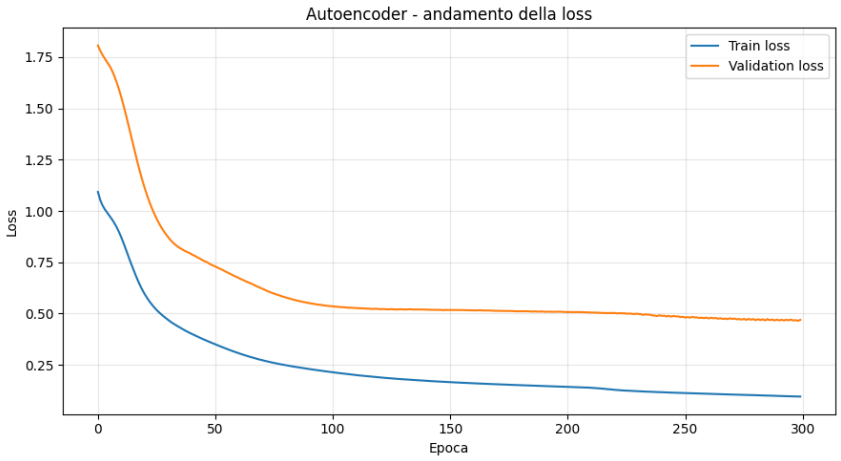


Figura 4.1: Andamento della loss sul training set e sul validation set durante le 300 epoche di addestramento dell'Autoencoder.

Nelle prime epoche sia la training loss che la validation loss diminuiscono rapidamente. Successivamente, la training loss continua a ridursi fino a un valore finale di circa 0.095, mentre la validation loss tende a stabilizzarsi e raggiun-

ge un valore minimo di circa 0.465. L'Autoencoder ha eseguito tutte le 300 epoche previste: l'Early Stopping, configurato con una patience pari a 20, non ha interrotto anticipatamente l'addestramento, poiché la validation loss ha continuato a mostrare piccoli miglioramenti.
Il progressivo divario tra training loss e validation loss indica che il modello ricostruisce meglio le finestre utilizzate per l'addestramento rispetto a quelle del validation set. Questo comportamento evidenzia un generalization gap, che deve essere tenuto in considerazione nell'interpretazione dei risultati.
Il validation set viene utilizzato in questa fase esclusivamente per monitorare l'errore di ricostruzione e controllare il processo di training; le etichette `sus` ed `evil` non vengono utilizzate per ottimizzare i pesi della rete.

4.6 Selezione delle soglie operative

Gli anomaly score prodotti dai modelli sono valori continui e non costituiscono direttamente una decisione binaria tra finestra normale e anomala. Per ottenere tale decisione è quindi necessario definire, per ciascun detector, una soglia operativa.
Per ciascun modello sono state inizialmente ricavate quattro soglie candidate, corrispondenti al 90°, 95°, 97° e 99° percentile della distribuzione degli anomaly score osservati sul training set. Una soglia più elevata rende il criterio di rilevamento più restrittivo, poiché soltanto le finestre con score particolarmente elevato vengono considerate anomale.
Le soglie candidate sono state successivamente applicate al validation set e confrontate utilizzando l'etichetta `sus_window`. Il criterio principale adottato per la scelta è stato l'F1-score, in modo da considerare congiuntamente precision e recall. Il test set è rimasto escluso da questa fase ed è stato utilizzato soltanto per la valutazione finale.
Per Isolation Forest, il miglior risultato sulla validation è stato ottenuto utilizzando il 95° percentile della distribuzione degli score del training, corrispondente a una soglia pari a circa 0.041385.
Per la One-Class SVM, le soglie corrispondenti al 97° e al 99° percentile hanno prodotto sulla validation lo stesso F1-score e le stesse predizioni. È stato quindi

adottato il 97° percentile, pari a una soglia di circa 0.988403, essendo il meno restrittivo tra i due valori equivalenti.
Nel caso dell'Autoencoder, il 90° percentile è risultato l'unico tra quelli considerati in grado di individuare almeno una delle finestre `sus` presenti nella validation. È stata quindi selezionata la soglia corrispondente, pari a circa 0.226207. La scelta deve essere interpretata con cautela, poiché il validation set contiene soltanto due finestre positive per `sus`.
Per Local Outlier Factor non è stato invece possibile individuare una soglia operativa mediante lo stesso criterio. Tutte le soglie candidate hanno infatti prodotto un F1-score pari a zero sulla validation. Per evitare una scelta arbitraria, nessuna delle quattro soglie è stata quindi adottata come soglia operativa per LOF. Gli score continui prodotti dal modello vengono comunque mantenuti e saranno successivamente utilizzati nella costruzione dell'ensemble.
Infine, non è stata selezionata una soglia specifica per l'etichetta `evil`. Come osservato in precedenza, il validation set non contiene finestre positive per tale etichetta e non permette quindi di effettuare una selezione supervisionata della soglia senza ricorrere impropriamente al test set.

4.7 Ensemble

Oltre ai singoli detector, è stato costruito un ensemble con l'obiettivo di combinare le informazioni prodotte da Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder.
Gli anomaly score generati dai quattro modelli non possono essere combinati direttamente, poiché appartengono a scale numeriche differenti. In una prima fase è stata quindi considerata una normalizzazione mediante Z-score, calcolando per ciascun modello media e deviazione standard esclusivamente sulla distribuzione degli score del training set.
Questa soluzione consente di riportare gli score su una scala comune, ma nell'analisi preliminare sono emersi valori standardizzati particolarmente elevati per alcuni modelli, in particolare per l'Autoencoder. In una combinazione basata sulla media, tali valori possono attribuire a un singolo detector un'influenza sproporzionata sul risultato finale.

Per questo motivo, nella configurazione definitiva è stata adottata una normalizzazione basata sul percentile empirico rispetto alla distribuzione degli score del training set. Per ogni modello viene calcolata la proporzione di score del training minori o uguali allo score della finestra considerata. Il valore ottenuto appartiene quindi all'intervallo tra 0 e 1: valori prossimi a 1 indicano che la finestra presenta uno score elevato rispetto alla maggior parte delle finestre osservate nel training.

La distribuzione utilizzata come riferimento per questa trasformazione viene costruita esclusivamente sul training set. La stessa distribuzione viene poi utilizzata per normalizzare gli score di training, validation e test, evitando quindi di ricavare parametri di normalizzazione dai dati utilizzati nelle fasi successive dell'esperimento.

Una volta normalizzati gli score, l'ensemble viene ottenuto calcolando la media dei valori prodotti dai quattro detector. A ciascun modello viene assegnato lo stesso peso, pari a 0.25. Non viene quindi privilegiato a priori nessuno dei quattro approcci sulla base delle prestazioni osservate sul test set.

Anche l'ensemble produce inizialmente uno score continuo. Per ottenere una decisione binaria sono state considerate, come per i singoli detector, soglie corrispondenti al 90°, 95°, 97° e 99° percentile della distribuzione degli ensemble score sul training set. La selezione è stata effettuata sulla validation utilizzando l'etichetta `sus_window` e l'F1-score come criterio principale.

La configurazione selezionata corrisponde al 90° percentile degli ensemble score del training, pari a una soglia di circa 0.86582. Anche in questo caso non è stata definita una soglia specifica per `evil`, poiché il validation set non contiene finestre positive per tale etichetta.

4.8 Pipeline detector + LLM

La prima configurazione analizzata combina i modelli di anomaly detection descritti nelle sezioni precedenti con un Large Language Model utilizzato per il triage delle finestre segnalate. In questa pipeline i due stadi svolgono quindi compiti distinti: i detector individuano le finestre da analizzare, mentre l'LLM interviene successivamente per interpretare e prioritizzare gli alert ricevuti.

La selezione delle finestre da sottoporre al triage viene effettuata utilizzando le soglie operative definite sulla validation. In particolare, vengono considerate le decisioni prodotte da Isolation Forest, One-Class SVM, Autoencoder ed Ensemble. Una finestra viene inoltrata all'LLM quando almeno uno di questi genera un alert. È stata quindi adottata una regola di unione, in modo da non escludere a priori una finestra segnalata da un solo detector.

Per ogni finestra viene inoltre calcolato il numero di segnali di alert attivi tra Isolation Forest, One-Class SVM, Autoencoder ed Ensemble, indicato come `consensus_count`. Tale valore viene utilizzato come informazione sintetica sul numero di criteri di rilevazione che hanno segnalato la finestra e non costituisce una nuova decisione di anomaly detection.

Local Outlier Factor non partecipa direttamente alla selezione binaria delle finestre, poiché nella fase di validation non è stato possibile definire una soglia operativa SUS. Il modello contribuisce tuttavia indirettamente alla pipeline attraverso il proprio anomaly score, utilizzato nella costruzione dell'Ensemble.

Prima dell'invio all'LLM, la telemetria associata a ciascuna finestra selezionata viene sintetizzata in una rappresentazione comportamentale. Invece di fornire direttamente tutti gli eventi kernel originali, vengono utilizzati il numero totale di eventi, i tipi di evento più frequenti con le relative frequenze, i processi maggiormente presenti e le associazioni processo-evento più frequenti. Questa rappresentazione riduce la quantità di informazioni inviate al modello mantenendo gli elementi ritenuti maggiormente utili per il triage.

Al contesto comportamentale vengono affiancate le informazioni prodotte dalla fase di anomaly detection. Per Isolation Forest, One-Class SVM, Autoencoder ed Ensemble vengono forniti lo score della finestra, la corrispondente soglia SUS e l'indicazione dell'eventuale superamento della soglia. Viene inoltre fornito il valore di `consensus_count`. Gli score dei diversi detector non vengono considerati direttamente confrontabili tra loro e la distanza tra uno score e la relativa soglia non viene interpretata come una misura della gravità dell'anomalia.

In questa configurazione l'LLM non svolge nuovamente il compito di anomaly detection. La finestra è già stata selezionata dallo stadio precedente e il modello viene utilizzato esclusivamente per produrre un report di triage. In particolare, deve assegnare una priorità tra `low`, `medium` e `high`, fornire una

breve descrizione del comportamento osservato, evidenziare gli elementi che meritano attenzione, indicare i processi da approfondire, suggerire eventuali verifiche successive ed esplicitare le incertezze della propria analisi.
Le annotazioni `sus` ed `evil` non vengono incluse nel contesto fornito all'LLM e vengono utilizzate soltanto successivamente per la valutazione sperimentale. Inoltre, il prompt specifica che il contenuto della telemetria deve essere trattato esclusivamente come dato non affidabile e non come insieme di istruzioni, limitando l'analisi alle informazioni effettivamente presenti nel contesto.
Per l'esperimento è stato utilizzato il modello `gemini-3.6-flash`, interrogato tramite API.La scelta del modello non deriva da un confronto sistematico tra differenti LLM, ma è legata alla continuità con l'esperienza maturata durante il seminario professionalizzante nell'ambito di AI4Security, nel quale era stato utilizzato un modello della stessa famiglia. Il presente lavoro mantiene quindi tale scelta come riferimento operativo per la realizzazione del prototipo, senza assumere che il modello selezionato rappresenti la soluzione migliore rispetto ad altre alternative disponibili. Lo stesso modello viene impiegato in entrambe le configurazioni considerate, in modo da mantenere invariata la componente LLM e concentrare il confronto sul diverso ruolo assegnato al modello nelle due pipeline. L'accesso tramite API consente inoltre di integrare direttamente le chiamate al modello nella procedura sperimentale.
L'output viene richiesto in formato strutturato JSON, così da ottenere per ogni finestra gli stessi campi e rendere più semplice la raccolta e il confronto dei risultati prodotti nelle diverse esecuzioni.

4.9 Pipeline LLM-only

La seconda configurazione sperimentale utilizza il Large Language Model senza una precedente fase di selezione da parte dei detector. In questo caso tutte le finestre del test set vengono analizzate direttamente dall'LLM, che deve svolgere sia il compito di anomaly detection sia la successiva fase di triage.
Per rendere il confronto con la pipeline detector + LLM il più possibile coerente, viene utilizzata la stessa rappresentazione comportamentale della telemetria. Per ogni finestra vengono quindi forniti il numero totale di eventi, i

tipi di evento più frequenti con le relative frequenze, i processi maggiormente presenti e le associazioni processo-evento più frequenti.

A differenza della pipeline precedente, non viene tuttavia fornita alcuna informazione proveniente dai modelli di anomaly detection. Gli anomaly score, le soglie operative, gli alert dei singoli detector, il valore di `consensus_count` e la decisione di selezione della finestra sono completamente esclusi dal contesto. Anche le annotazioni `sus` ed `evil` rimangono nascoste al modello e vengono utilizzate soltanto nella successiva fase di valutazione. L'LLM deve quindi formulare la propria decisione utilizzando esclusivamente le informazioni comportamentali contenute nella telemetria sintetizzata.

Per ogni finestra il modello deve innanzitutto stabilire se il comportamento osservato debba essere considerato anomalo, producendo una decisione booleana attraverso il campo `anomalous`. Successivamente assegna una priorità tra `low`, `medium` e `high` e genera un report contenente una descrizione del comportamento osservato, le evidenze che supportano la valutazione, i processi da approfondire, eventuali controlli successivi e un'indicazione delle incertezze presenti nell'analisi.

Il prompt specifica inoltre che la presenza di un numero elevato di eventi non deve essere considerata, da sola, una prova sufficiente di anomalia. Il modello viene invitato a basare la propria decisione esclusivamente sulle informazioni fornite e a non introdurre processi, eventi, comandi o possibili tecniche di attacco non supportati dalla telemetria disponibile. Anche in questa configurazione il contenuto della telemetria viene trattato come dato non affidabile e non come insieme di istruzioni.

Come nella pipeline detector + LLM, viene utilizzato il modello `gemini-3.6-flash` e la risposta viene richiesta in formato JSON strutturato. La differenza fondamentale tra le due configurazioni risiede quindi nel ruolo assegnato all'LLM: nella pipeline ibrida il modello interviene soltanto dopo la selezione effettuata dai detector, mentre nella configurazione LLM-only deve decidere autonomamente quali finestre presentino un comportamento anomalo e successivamente effettuare il relativo triage.

4.10 Protocollo di confronto

Le due pipeline vengono confrontate utilizzando le stesse finestre del test set, mantenendo separate la fase di configurazione dei modelli e quella di valutazione finale. Le etichette del test non vengono quindi utilizzate per modificare soglie, parametri o criteri di selezione definiti nelle fasi precedenti.

Il primo aspetto considerato riguarda la capacità delle due architetture di individuare le finestre annotate come sospette. Nella pipeline LLM-only la decisione di anomaly detection corrisponde direttamente al valore booleano `anomalous` prodotto dal modello linguistico per ciascuna finestra. Nella pipeline detector + LLM viene invece considerata la decisione prodotta dallo stadio di anomaly detection: una finestra è considerata selezionata quando almeno uno tra Isolation Forest, One-Class SVM, Autoencoder ed Ensemble genera un alert utilizzando la propria soglia SUS validata.

In entrambi i casi il riferimento principale è l'etichetta `sus_window`. Le decisioni ottenute vengono confrontate con la ground truth calcolando matrice di confusione, precision, recall, F1-score e accuracy. Data la dimensione ridotta del test set, tali valori vengono interpretati come risultati descrittivi dell'esperimento e non come stime generali delle prestazioni delle due architetture.

L'etichetta `evil_window` viene invece mantenuta per un'analisi descrittiva separata. Poiché nella validation non sono presenti finestre positive per `evil`, tale etichetta non è stata utilizzata per selezionare soglie operative e non viene impiegata per modificare retrospettivamente le decisioni prodotte dalle pipeline.

Un secondo asse di confronto riguarda il costo temporale osservato. Per ogni chiamata al modello linguistico viene misurata la latenza end-to-end dal momento immediatamente precedente all'invio della richiesta fino alla ricezione della risposta. Questa misura comprende quindi anche i contributi della comunicazione di rete e dell'infrastruttura remota e non rappresenta esclusivamente il tempo interno di inferenza dell'LLM. Oltre al tempo complessivo vengono considerate media, mediana, deviazione standard, minimo e massimo delle latenze osservate.

Nella pipeline LLM-only viene effettuata una chiamata LLM per ogni finestra del test set. Nella pipeline detector + LLM le finestre vengono invece prima elaborate localmente dallo stadio di anomaly detection e soltanto quelle selezionate vengono inviate al modello linguistico. Il tempo complessivo di questa seconda configurazione viene pertanto ottenuto considerando sia il costo dello stadio locale sia le latenze delle successive chiamate di triage.

Per stimare il costo dello stadio locale, l'intera fase di inferenza della pipeline di anomaly detection viene eseguita ripetutamente sul test set. Il benchmark comprende la standardizzazione delle feature, il calcolo degli anomaly score dei quattro detector, la normalizzazione necessaria all'Ensemble, il calcolo dello score combinato, l'applicazione delle soglie operative e la selezione delle finestre. L'esecuzione viene ripetuta 100 volte, in modo da ricavare una misura media del tempo richiesto per elaborare l'intero test set.

Per i componenti eseguiti localmente vengono inoltre osservati il tempo CPU e la memoria residente del processo Python. Quest'ultima viene monitorata tramite il valore RSS (*Resident Set Size*) durante l'esecuzione, rilevando l'incremento massimo osservato rispetto alla memoria già occupata dal runtime. Tale quantità non rappresenta quindi la memoria totale necessaria per caricare i modelli, ma soltanto la memoria aggiuntiva osservata durante l'inferenza nel processo già inizializzato.

Le risorse computazionali utilizzate da Gemini non possono invece essere misurate direttamente, poiché il modello viene eseguito su un servizio remoto accessibile tramite API. CPU, GPU e memoria utilizzate lato server non sono quindi incluse nel confronto. Per la componente LLM vengono considerati esclusivamente gli elementi osservabili dal client, in particolare la latenza end-to-end e il numero di richieste effettuate. Il confronto finale considera quindi congiuntamente la capacità di individuare le finestre `sus`, il numero di chiamate richieste al modello linguistico, la latenza osservata e il costo computazionale locale della componente di anomaly detection, tenendo conto delle differenti caratteristiche architetturali delle due pipeline.

Capitolo 5

Risultati sperimentali

5.1 Setup sperimentale e criteri di valutazione

Gli esperimenti principali sono stati condotti sulla rappresentazione a finestre temporali di 30 secondi descritta nel capitolo precedente. Dopo il preprocessing sono disponibili 128 finestre per il training set, 22 per il validation set e 13 per il test set, ciascuna rappresentata mediante le 32 feature corrispondenti alle frequenze relative dei diversi tipi di evento (`eventName`) osservati in ciascuna finestra.

A questa configurazione principale è stata affiancata un'analisi supplementare di sensibilità, nella quale i quattro detector individuali sono stati valutati anche utilizzando finestre di 5, 10, 60 e 120 secondi. Tale analisi ha finalità di robustezza e non viene utilizzata per ridefinire a posteriori la configurazione principale adottata nel resto della pipeline.

Il training set viene utilizzato per l'addestramento dei modelli e come riferimento per la costruzione delle soglie candidate e per la normalizzazione degli anomaly score dell'Ensemble. Il validation set viene utilizzato per la selezione delle configurazioni previste dall'esperimento e delle soglie operative, mentre il test set rimane escluso da queste scelte ed è utilizzato soltanto per la valutazione finale.

Nel test set sono presenti 7 finestre annotate come `sus` e 5 finestre annotate come `evil`. L'etichetta `sus` costituisce il riferimento principale per il confronto delle prestazioni, poiché può essere utilizzata durante la fase di validation per la selezione delle soglie. L'etichetta `evil` viene invece considerata principalmente

per un'analisi descrittiva, poiché nel validation set non sono presenti finestre positive che consentano di definire una soglia operativa specifica.
La valutazione dei singoli detector viene effettuata considerando innanzitutto la capacità degli anomaly score di ordinare le finestre rispetto alle annotazioni del dataset, attraverso la ROC-AUC. Per l'etichetta `sus`, agli score continui vengono inoltre applicate le soglie operative selezionate sulla validation, calcolando precision, recall e F1-score sulle decisioni binarie ottenute nel test set.
La stessa impostazione viene utilizzata per l'Ensemble, il cui score continuo viene valutato attraverso la ROC-AUC e successivamente trasformato in una decisione binaria mediante la soglia selezionata sul validation set.
La seconda parte dell'analisi riguarda il confronto tra la pipeline detector + LLM e la pipeline LLM-only. In questo caso vengono confrontate le decisioni di anomaly detection rispetto all'etichetta `sus`, utilizzando matrice di confusione, precision, recall, F1-score e accuracy.
Oltre alla capacità di rilevazione vengono considerate anche le caratteristiche operative delle due architetture. In particolare, vengono analizzati il numero di chiamate effettuate al modello linguistico, le latenze end-to-end osservate e, per la componente di anomaly detection eseguita localmente, il tempo di esecuzione, il tempo CPU e l'incremento di memoria RSS osservato durante l'inferenza.
Gli esperimenti relativi ai detector sono stati eseguiti nel runtime Google Colab, mentre il modello linguistico `gemini-3.6-flash` è stato utilizzato tramite API remota. Di conseguenza, il consumo di CPU, GPU e memoria dei server che eseguono il modello linguistico non è direttamente osservabile e non viene incluso nelle misure delle risorse locali.

5.2 Analisi esplorativa della rappresentazione mediante PCA e UMAP

Prima di analizzare le prestazioni dei detector, è stata effettuata un'analisi esplorativa della rappresentazione a finestre mediante due tecniche di ridu-

zione della dimensionalità: Principal Component Analysis (PCA) e Uniform Manifold Approximation and Projection (UMAP).
L'obiettivo di questa analisi non è introdurre ulteriori modelli di anomaly detection né utilizzare le proiezioni per modificare la configurazione dei detector, ma osservare in forma bidimensionale la struttura delle finestre rappresentate mediante le 32 frequenze relative degli `eventName`.
Prima dell'applicazione delle due tecniche, le feature sono state standardizzate mediante `StandardScaler`. I parametri della trasformazione sono stati stimati esclusivamente sul training set e successivamente utilizzati per trasformare validation e test. Analogamente, PCA e UMAP sono state adattate esclusivamente alle finestre del training set, mentre validation e test sono stati successivamente proiettati negli spazi ottenuti.
Le etichette `sus_window` ed `evil_window` non sono state utilizzate durante la costruzione delle rappresentazioni. Esse vengono considerate soltanto a posteriori per evidenziare la posizione delle finestre annotate nel dataset.

5.2.1 Analisi mediante PCA

La PCA realizza una trasformazione lineare delle feature originali individuando componenti ordinate rispetto alla variabilità rappresentata. Nel presente esperimento, la prima componente principale spiega il 32,38% della varianza del training set, mentre la seconda ne spiega il 18,01%. Le prime due componenti rappresentano quindi complessivamente il 50,39% della varianza osservata nelle feature standardizzate.
La Figura 5.1 riporta la proiezione delle finestre del training e del test sulle prime due componenti. Le osservazioni del training sono utilizzate come riferimento visivo, mentre le finestre del test vengono distinte tra non-SUS, SUS non-EVIL ed EVIL.
Nella proiezione alcune finestre EVIL occupano posizioni periferiche rispetto alla regione maggiormente popolata dalle finestre del training. La finestra 0 presenta in particolare un valore elevato lungo PC2, mentre le finestre 11 e 12 assumono valori elevati lungo PC1. Anche le finestre 9 e 10 risultano separate dalla concentrazione principale in direzioni differenti.

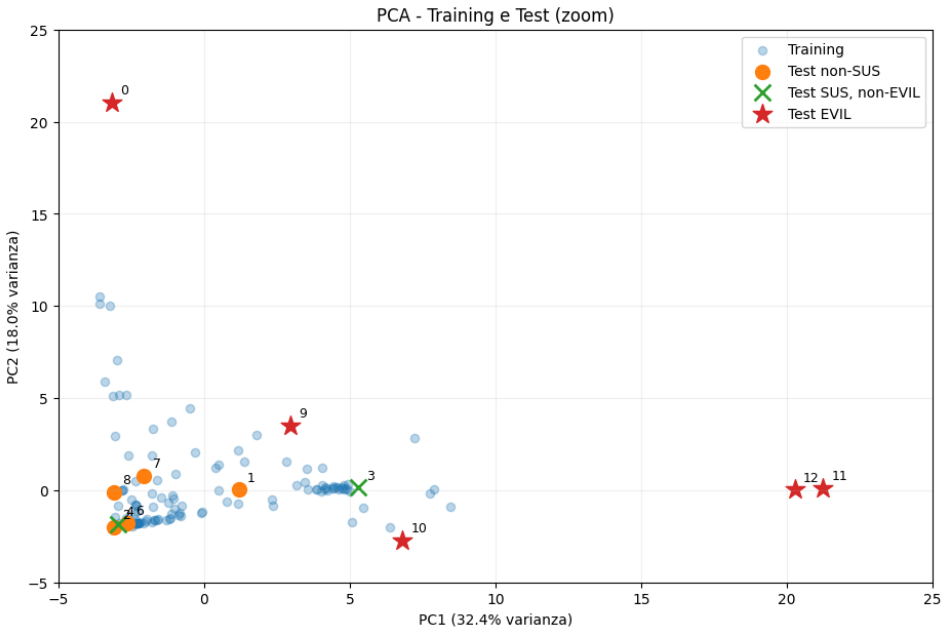


Figura 5.1: Proiezione PCA delle finestre di training e test. Le finestre del test sono distinte tra non-SUS, SUS non-EVIL ed EVIL. È riportato uno zoom della regione contenente la maggior parte delle osservazioni.

Una posizione periferica nella proiezione non coincide tuttavia necessariamente con un'annotazione SUS o EVIL. La finestra 5 del test set, annotata come non-SUS e non-EVIL, presenta infatti coordinate particolarmente elevate ($PC1 \approx 43.10$ e $PC2 \approx 48.23$). Per rendere maggiormente leggibile la regione contenente le altre osservazioni, tale finestra ricade al di fuori dello zoom mostrato in Figura 5.1.

Anche le finestre SUS non-EVIL non presentano una collocazione uniforme: la finestra 3 risulta spostata verso valori positivi di PC1, mentre la finestra 4 si trova in prossimità di osservazioni del training.

La visualizzazione non evidenzia pertanto una separazione lineare netta tra le tre categorie considerate. Tale risultato deve inoltre essere interpretato tenendo conto del fatto che il piano formato da PC1 e PC2 rappresenta il 50,39% della varianza complessiva delle feature standardizzate. La figura costituisce quindi una rappresentazione parziale e puramente esplorativa dello spazio originale a 32 dimensioni.

5.2.2 Interpretazione delle componenti principali

Per esaminare il contributo delle singole feature alle prime due componenti sono stati analizzati i corrispondenti loadings.

Per PC1, i coefficienti di maggiore entità tra quelli osservati sono associati agli eventi `dup`, `accept`, `setuid`, `setgid`, `dup2`, `connect`, `kill`, `cap_capable`, `getsockname` e `security_bprm_check`. I coefficienti mostrati per queste feature hanno segno positivo; pertanto, all'interno della rappresentazione standardizzata utilizzata nell'esperimento, un aumento congiunto di tali feature contribuisce alla direzione positiva della prima componente.

Per PC2, tra i coefficienti di maggiore entità positiva compaiono `access`, `lstat`, `unlink`, `fstat`, `dup3`, `unlinkat`, `security_inode_unlink`, `prctl` e `symlink`. La feature `security_file_open` presenta invece un coefficiente negativo di entità comparabile.

Questi coefficienti descrivono la costruzione matematica delle componenti e non costituiscono indicatori di attività malevola. La PCA è infatti calcolata senza utilizzare le annotazioni SUS ed EVIL; la relazione tra la posizione nello spazio PCA e tali annotazioni viene osservata soltanto successivamente. Le Figure 5.2 e 5.3 riportano le dieci feature con maggiore contributo in valore assoluto rispettivamente alla prima e alla seconda componente principale.

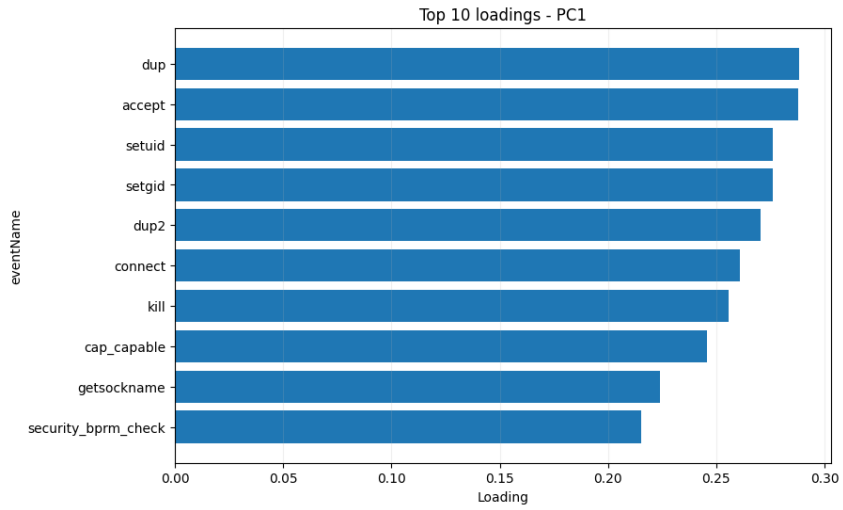


Figura 5.2: Dieci feature con maggiore contributo alla prima componente principale, selezionate in base al valore assoluto del loading.

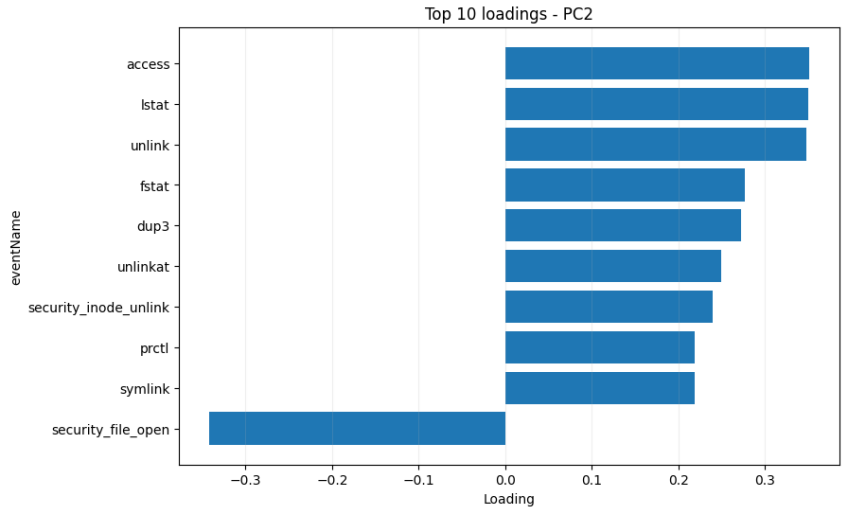


Figura 5.3: Dieci feature con maggiore contributo alla seconda componente principale, selezionate in base al valore assoluto del loading.

5.2.3 Analisi mediante UMAP

Come analisi complementare è stata utilizzata UMAP, una tecnica non lineare di riduzione della dimensionalità che costruisce una rappresentazione a bassa dimensionalità prestando particolare attenzione alle relazioni di vicinato tra le osservazioni.

Nel presente esperimento è stata utilizzata una configurazione con `n_neighbors=15`, `min_dist=0.1`, metrica euclidea e `random_state=42`. I valori di `n_neighbors` e `min_dist` corrispondono ai valori predefiniti della libreria `umap-learn`; la configurazione non è stata ottimizzata utilizzando le etichette SUS o EVIL.

Il modello UMAP è stato adattato esclusivamente al training set. Le finestre del test sono state successivamente trasformate nello stesso spazio bidimensionale.

Nella rappresentazione ottenuta, alcune finestre del test assumono posizioni molto vicine. In particolare, le finestre EVIL 9 e 10 risultano prossime tra loro, così come le finestre 11 e 12. La finestra EVIL 0 occupa invece una regione differente della proiezione.

Anche le due finestre SUS non-EVIL presentano collocazioni differenti: la finestra 3 si trova nella parte superiore della rappresentazione, in prossimità di alcune osservazioni EVIL e di training, mentre la finestra 4 si colloca nella regione destra della mappa, anch'essa in prossimità di osservazioni di training.

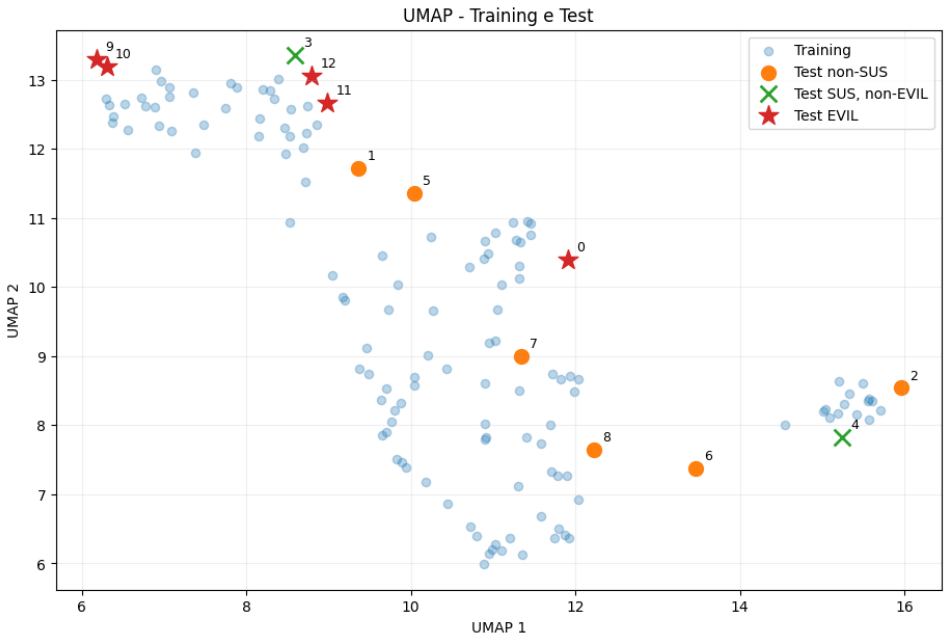


Figura 5.4: Proiezione UMAP delle finestre di training e test. Le finestre del test sono distinte tra non-SUS, SUS non-EVIL ed EVIL.

Le finestre non-SUS del test sono distribuite in diverse regioni della rappresentazione. Non emerge quindi una separazione completa tra finestre non-SUS, SUS non-EVIL ed EVIL.

La vicinanza osservata nella proiezione UMAP viene interpretata esclusivamente come evidenza esplorativa relativa alla rappresentazione ottenuta con la configurazione considerata. Essa non viene utilizzata come misura quantitativa di separazione delle classi né come risultato di clustering.

5.2.4 Confronto tra PCA e UMAP

PCA e UMAP offrono due visualizzazioni complementari della rappresentazione a finestre, ma i rispettivi spazi non devono essere interpretati nello stesso modo.

La PCA fornisce una trasformazione lineare per la quale è possibile quantificare la quota di varianza associata alle componenti. Nel presente esperimento, PC1 e PC2 spiegano complessivamente il 50,39% della varianza delle feature standardizzate. La proiezione mostra alcune finestre EVIL in posizioni periferiche, ma mostra anche che una marcata distanza dalla concentrazione principale non è esclusiva delle finestre annotate come sospette o malevole.

UMAP produce invece una rappresentazione non lineare nella quale `n_neighbors` e `min_dist` influenzano rispettivamente la scala del vicinato considerato e la disposizione dei punti nello spazio a bassa dimensionalità. Nella configurazione utilizzata emergono alcune vicinanze tra finestre EVIL, senza tuttavia ottenere una separazione completa rispetto alle altre osservazioni.

Nel complesso, le due visualizzazioni non mostrano una corrispondenza semplice tra le annotazioni SUS o EVIL e regioni completamente separate dello spazio bidimensionale. Il risultato viene considerato esclusivamente descrittivo della rappresentazione a finestre utilizzata nel presente esperimento.

Nessuna informazione ricavata da PCA o UMAP è stata utilizzata per scegliere gli iperparametri dei detector, definire le soglie operative o modificare il protocollo di valutazione.

Va inoltre osservato che le feature utilizzate sono frequenze relative e costituiscono pertanto una rappresentazione soggetta a vincolo di somma. La PCA standard viene qui impiegata esclusivamente come strumento esplorativo di visualizzazione e non come analisi formale di dati composizionali. Approcci specifici per dati composizionali, basati ad esempio su trasformazioni log-ratio, potrebbero costituire un'estensione dell'analisi.

L'uso di UMAP consente inoltre un confronto qualitativo con l'analisi esplorativa presentata nel lavoro originale su BETH. Highnam et al. applicano UMAP alle feature preprocessate dei singoli eventi, adattando la rappresentazione sul training set e proiettando successivamente il test nello stesso spazio. Gli autori osservano una significativa sovrapposizione tra training e test; tuttavia, colorando la stessa rappresentazione sulla base dell'etichetta `evil`, gli eventi malevoli risultano concentrati in alcune aree distinte dello spazio UMAP [1].

Nel presente lavoro si osserva un comportamento solo parzialmente analogo. Alcune finestre EVIL risultano localmente vicine, in particolare le coppie 9–10 e 11–12, mentre la finestra 0 occupa una regione differente. Non emerge tuttavia una regione unica e nettamente separata contenente tutte le finestre EVIL.

Il confronto deve essere interpretato esclusivamente in termini qualitativi, poiché nel lavoro originale ogni punto rappresenta un singolo evento kernel-process sottoposto al preprocessing definito dagli autori, mentre nel presente

lavoro ogni osservazione corrisponde a una finestra temporale di 30 secondi descritta mediante le frequenze relative di 32 tipi di `eventName`.

5.3 Risultati dei singoli modelli di anomaly detection

La prima analisi riguarda la capacità degli anomaly score prodotti dai quattro detector di separare le finestre annotate nel test set. La Tabella 5.1 riporta i valori di ROC-AUC ottenuti rispetto alle etichette `sus` ed `evil`.

Tabella 5.1: ROC-AUC dei singoli modelli sul test set.

Modello	**AUC SUS**	**AUC EVIL**
Isolation Forest	0.643	0.825
Local Outlier Factor	0.548	0.825
One-Class SVM	0.690	0.900
Autoencoder	0.667	0.875

I risultati mostrano una maggiore capacità di separazione rispetto all'etichetta `evil` rispetto a `sus` per tutti i modelli. Sulla label `sus`, la One-Class SVM presenta l'AUC più elevata, pari a 0.690, seguita dall'Autoencoder con 0.667 e da Isolation Forest con 0.643. LOF ottiene invece un valore pari a 0.548, vicino al comportamento di un ordinamento casuale. Per `evil`, la One-Class SVM raggiunge un'AUC di 0.900, mentre Autoencoder, Isolation Forest e LOF ottengono rispettivamente 0.875, 0.825 e 0.825.

La capacità di separazione di Isolation Forest rispetto alle etichette `SUS` ed `EVIL` è visualizzata anche attraverso le rispettive curve ROC, riportate in Figura 5.5. La curva relativa a `SUS` presenta una ROC-AUC pari a 0.643, indicando una capacità di separazione limitata ma superiore a quella corrispondente a un ordinamento casuale. La curva relativa a `EVIL` raggiunge invece una ROC-AUC pari a 0.825. Nel test set considerato, gli anomaly score prodotti da Isolation Forest separano quindi meglio le finestre EVIL dalle finestre non-EVIL rispetto a quanto avviene per le finestre SUS e non-SUS.

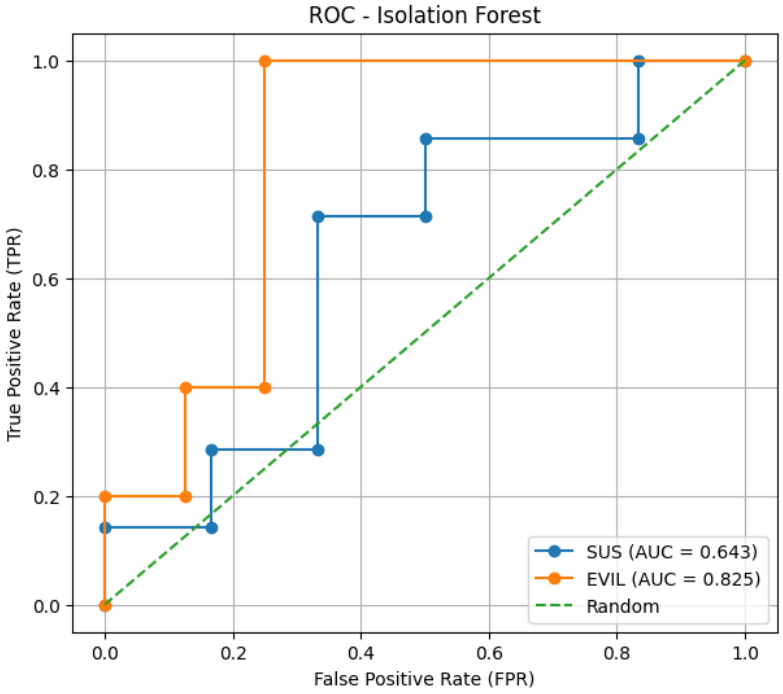


Figura 5.5: Curve ROC del modello Isolation Forest rispetto alle etichette `SUS` ed `EVIL`. La figura riporta anche la diagonale corrispondente a un ordinamento casuale.

La Figura 5.6 mostra le curve ROC ottenute dal modello Local Outlier Factor rispetto alle etichette `SUS` ed `EVIL`.

La curva relativa a `SUS` presenta una ROC-AUC pari a 0.548, solo lievemente superiore al valore casuale. Ciò indica che, nel test set considerato, gli score prodotti da LOF hanno una capacità molto limitata di separare le finestre SUS da quelle non-SUS. La curva relativa a `EVIL` raggiunge invece una ROC-AUC pari a 0.825, mostrando una capacità di separazione sensibilmente migliore. Anche in questo caso, quindi, gli anomaly score risultano più efficaci nel distinguere le finestre EVIL rispetto alle finestre SUS.

La Figura 5.7 mostra le curve ROC ottenute dal modello One-Class SVM rispetto alle etichette `SUS` ed `EVIL`.

Nel caso della One-Class SVM, la curva relativa a `SUS` presenta una ROC-AUC pari a 0.690, indicando una capacità di separazione discreta tra finestre SUS e non-SUS. La curva relativa a `EVIL` raggiunge invece una ROC-AUC pari a 0.900, evidenziando una capacità di separazione più elevata. Anche per questo detector, quindi, gli anomaly score risultano più efficaci nel distinguere

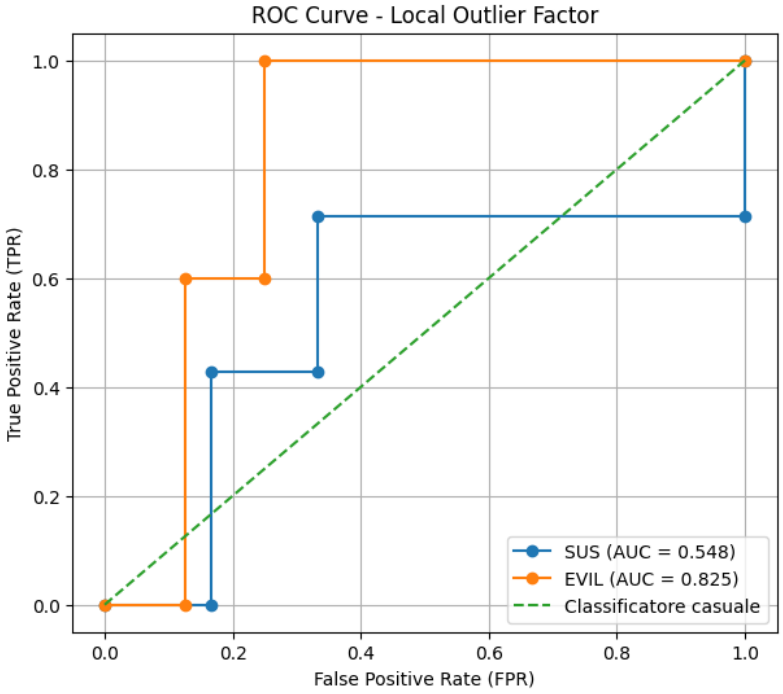


Figura 5.6: Curve ROC del modello Local Outlier Factor rispetto alle etichette `SUS` ed `EVIL`.

le finestre EVIL rispetto alle finestre SUS.

La Figura 5.8 riporta le curve ROC dell'Autoencoder rispetto alle etichette `SUS` ed `EVIL`.

L'AUC è pari a 0.667 per `SUS` e a 0.875 per `EVIL`. Gli anomaly score prodotti dall'Autoencoder mostrano quindi una capacità di separazione maggiore rispetto all'etichetta `EVIL` rispetto a `SUS`. La separazione rispetto a `SUS` rimane comunque superiore a quella corrispondente a un ordinamento casuale.

Per l'etichetta `sus` è possibile valutare anche le decisioni binarie ottenute applicando le soglie operative selezionate tramite il validation set. I risultati sono riportati nella Tabella 5.2.

Isolation Forest individua 2 delle 7 finestre `sus` presenti nel test set. La precision risulta pari a 0.667, mentre la recall è pari a 0.286: il modello genera quindi un numero contenuto di falsi positivi, ma non individua cinque delle sette finestre positive.

La One-Class SVM e l'Autoencoder mostrano invece una maggiore copertura delle finestre `sus`, identificandone entrambi 5 su 7 e ottenendo una recall pari a 0.714. La One-Class SVM presenta una precision di 0.625 e un F1-score di

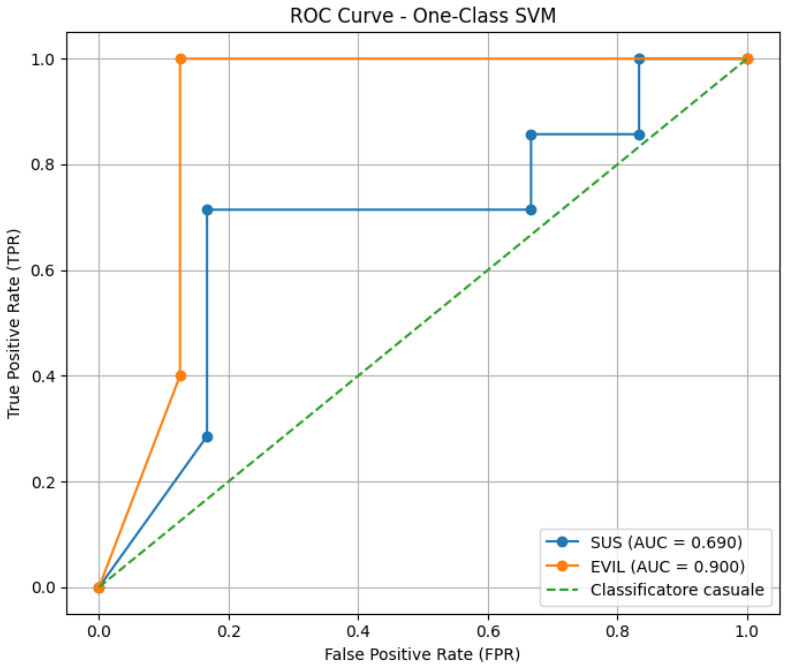


Figura 5.7: Curve ROC del modello One-Class SVM rispetto alle etichette `SUS` ed `EVIL`.

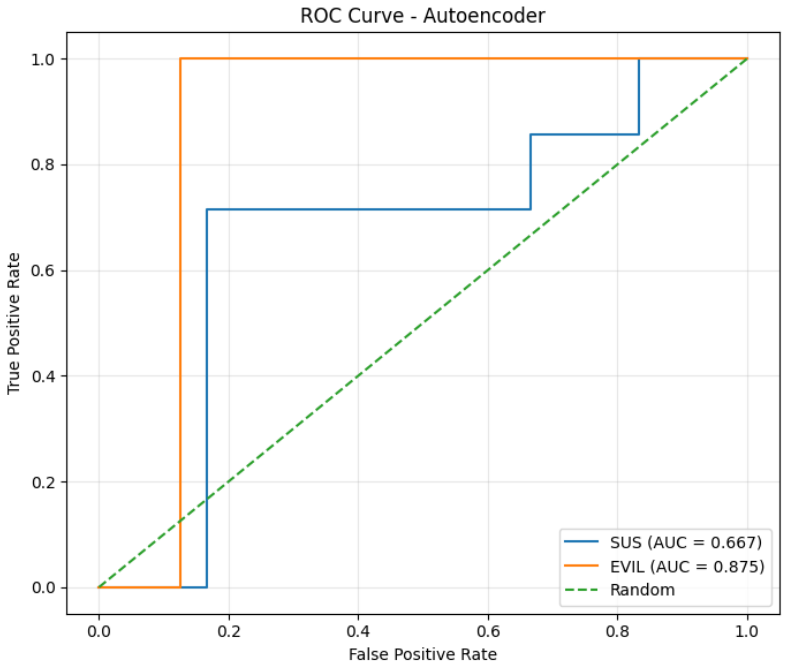


Figura 5.8: Curve ROC del modello Autoencoder rispetto alle etichette `SUS` ed `EVIL`.

Tabella 5.2: Prestazioni sul test set rispetto all'etichetta SUS utilizzando le soglie operative selezionate in validation.

Modello	**Precision**	**Recall**	**F1-score**
Isolation Forest	0.667	0.286	0.400
Local Outlier Factor	–	–	–
One-Class SVM	0.625	0.714	0.667
Autoencoder	0.556	0.714	0.625

0.667, mentre l'Autoencoder raggiunge una precision di 0.556 e un F1-score di 0.625.

Per LOF non vengono riportate precision, recall e F1-score, poiché nessuna delle soglie candidate ha prodotto un F1-score maggiore di zero sulla validation. Non è stata quindi definita una soglia operativa SUS da applicare al test set. Il modello rimane comunque confrontabile attraverso gli anomaly score continui e il relativo valore di ROC-AUC.

I risultati relativi all'etichetta `evil` richiedono una diversa interpretazione. La ROC-AUC viene calcolata sul test set direttamente a partire dagli anomaly score e permette di valutare la capacità dei modelli di ordinare le finestre EVIL rispetto a quelle non-EVIL. Poiché il validation set non contiene finestre EVIL positive, non è stato invece possibile selezionare una soglia operativa specifica per questa etichetta. Le quattro soglie candidate derivate dal training set vengono comunque applicate al test anche rispetto a `evil`, calcolando precision, recall e F1-score a scopo esplorativo. Tali risultati non vengono però utilizzati per scegliere retrospettivamente una soglia, ma esclusivamente per descrivere il comportamento dei detector rispetto alle finestre EVIL.

5.3.1 Analisi di sensibilità alla durata delle finestre temporali

La durata della finestra temporale determina sia il numero di osservazioni disponibili sia il livello di aggregazione dell'attività del sistema. Per valutare quanto tale scelta influenzi i risultati dei detector, è stata effettuata un'analisi

supplementare considerando finestre non sovrapposte di 5, 10, 30, 60 e 120 secondi.

Le configurazioni dei quattro detector sono state mantenute invariate rispetto all'esperimento principale. In particolare, non è stato effettuato un nuovo tuning degli iperparametri per ciascuna durata. Per ogni granularità sono state invece ricostruite le finestre e le relative feature, adattando lo `StandardScaler` esclusivamente sul corrispondente training set. Le soglie operative SUS sono state ricalcolate utilizzando i percentili 90, 95, 97 e 99 degli anomaly score del training e selezionate sul validation set secondo l'F1-score, come nel protocollo principale.

Questa impostazione consente di osservare il comportamento della configurazione dei detector adottata nella tesi al variare della granularità temporale, senza utilizzare le prestazioni del test per ottimizzare né gli iperparametri né la durata della finestra.

Tabella 5.3: Numerosità delle finestre e distribuzione delle etichette SUS ed EVIL al variare della durata. Training e validation non contengono finestre EVIL per nessuna delle granularità considerate.

Durata	N_{train}	$\mathbf{SUS}_{\text{train}}$	N_{val}	$\mathbf{SUS}_{\text{val}}$	N_{test}	$\mathbf{SUS}_{\text{test}}$	$\mathbf{EVIL}_{\text{test}}$
5 s	688	11	114	5	38	17	13
10 s	376	10	65	4	26	12	9
30 s	128	8	22	2	13	7	5
60 s	64	8	11	2	7	6	4
120 s	32	7	6	2	4	4	3

La Tabella 5.3 mostra che la durata della finestra modifica in modo sostanziale la numerosità del problema sperimentale. Con finestre di 5 secondi il test set contiene 38 osservazioni, mentre a 60 secondi ne rimangono soltanto 7 e a 120 secondi soltanto 4. Anche il numero di finestre positive varia, poiché le etichette degli eventi vengono aggregate mediante una regola *any*: una finestra è positiva quando contiene almeno un evento con la corrispondente annotazione.

Questo aspetto è particolarmente rilevante per l'interpretazione delle metriche. A 120 secondi tutte e quattro le finestre del test risultano SUS; in assenza di esempi negativi, la ROC-AUC rispetto a SUS non è definita. Anche le metriche

ottenute a 60 e 120 secondi devono essere interpretate con particolare cautela, poiché dipendono da un numero molto ridotto di osservazioni.

Tabella 5.4: Analisi di sensibilità di Isolation Forest rispetto alla durata delle finestre temporali. Precision, recall e F1-score sono riferiti all'etichetta SUS utilizzando la soglia selezionata sul validation set.

Durata	**AUC SUS**	**AUC EVIL**	**Percentile**	**Soglia**	**Precision**	**Recall**	**F1**
5 s	0.818	0.902	95°	-0.0136	0.833	0.294	0.435
10 s	0.792	0.928	95°	-0.0178	0.833	0.417	0.556
30 s	0.643	0.825	95°	0.0414	0.667	0.286	0.400
60 s	0.833	1.000	95°	0.1246	1.000	0.167	0.286
120 s	n.d.	1.000	90°	0.0692	1.000	0.500	0.667

Per Isolation Forest, le finestre di 5 e 10 secondi producono valori di ROC-AUC SUS superiori rispetto alla configurazione principale di 30 secondi, passando rispettivamente da 0.643 a 0.818 e 0.792. L'F1-score aumenta da 0.400 a 0.435 e 0.556, principalmente per effetto di una maggiore recall.
Il valore di ROC-AUC SUS pari a 0.833 osservato a 60 secondi non deve invece essere interpretato come evidenza di una superiorità di tale granularità. Il test comprende infatti soltanto 7 finestre, 6 delle quali positive per SUS. Coerentemente, la soglia selezionata produce precision pari a 1.000 ma recall pari a soltanto 0.167. A 120 secondi la ROC-AUC SUS non è definita, poiché tutte le osservazioni del test appartengono alla classe positiva.

Tabella 5.5: Analisi di sensibilità di Local Outlier Factor rispetto alla durata delle finestre temporali. Il simbolo – indica che nessuna delle soglie candidate ha prodotto un F1-score SUS maggiore di zero sul validation set.

Durata	**AUC SUS**	**AUC EVIL**	**Percentile**	**Soglia**	**Precision**	**Recall**	**F1**
5 s	0.754	0.877	–	–	–	–	–
10 s	0.696	0.863	–	–	–	–	–
30 s	0.548	0.825	–	–	–	–	–
60 s	0.833	1.000	97°	2.5291	1.000	0.667	0.800
120 s	n.d.	1.000	95°	1.7080	1.000	0.750	0.857

Anche per LOF la ROC-AUC SUS aumenta riducendo la durata da 30 a 10 e 5 secondi, raggiungendo rispettivamente 0.696 e 0.754. Tuttavia, per queste

tre granularità nessuna delle soglie candidate considerate produce un F1-score SUS maggiore di zero sul validation set. Non viene quindi definita una decisione binaria operativa, in coerenza con il criterio adottato nell'esperimento principale.

A 60 e 120 secondi vengono invece selezionate soglie operative e si osservano F1-score elevati. Anche in questo caso il risultato deve essere letto insieme alla forte riduzione del campione: il validation set contiene rispettivamente 11 e 6 finestre e il test soltanto 7 e 4. I valori ottenuti non costituiscono pertanto evidenza sufficiente per concludere che finestre più ampie migliorino in modo generale il comportamento di LOF.

Tabella 5.6: Analisi di sensibilità della One-Class SVM rispetto alla durata delle finestre temporali. La configurazione del modello è mantenuta invariata rispetto all'esperimento principale.

Durata	**AUC SUS**	**AUC EVIL**	**Percentile**	**Soglia**	**Precision**	**Recall**	**F1**
5 s	0.824	1.000	99°	14.9152	1.000	0.765	0.867
10 s	0.756	1.000	99°	8.6924	0.900	0.750	0.818
30 s	0.690	0.900	97°	0.9884	0.625	0.714	0.667
60 s	1.000	1.000	99°	0.3593	1.000	0.667	0.800
120 s	n.d.	1.000	90°	0.0000	1.000	1.000	1.000

La One-Class SVM mostra una sensibilità marcata alla durata della finestra. Con finestre di 5 e 10 secondi la ROC-AUC SUS è pari rispettivamente a 0.824 e 0.756, rispetto a 0.690 nella configurazione principale di 30 secondi. Anche l'F1-score aumenta, raggiungendo 0.867 a 5 secondi e 0.818 a 10 secondi.

A 60 secondi la ROC-AUC SUS risulta pari a 1.000. Tale valore deriva tuttavia da un test composto da sole 7 osservazioni, delle quali 6 positive, e non deve quindi essere interpretato come una stima robusta di separazione perfetta. Lo stesso principio vale, in misura ancora maggiore, per i 120 secondi: le quattro finestre del test sono tutte SUS e la ROC-AUC non può essere calcolata. Precision, recall e F1-score pari a 1.000 indicano che, in questo specifico campione, tutte le finestre vengono correttamente segnalate; non implicano tuttavia una capacità di distinguere esempi positivi e negativi, poiché nel test non sono presenti finestre SUS negative.

Tabella 5.7: Analisi di sensibilità dell'Autoencoder rispetto alla durata delle finestre temporali. Architettura e configurazione di addestramento sono mantenute invariate rispetto all'esperimento principale.

Durata	**AUC SUS**	**AUC EVIL**	**Percentile**	**Soglia**	**Precision**	**Recall**	**F1**
5 s	0.857	1.000	97°	0.4247	1.000	0.765	0.867
10 s	0.869	1.000	97°	0.3628	0.900	0.750	0.818
30 s	0.667	0.875	90°	0.2262	0.556	0.714	0.625
60 s	0.833	1.000	99°	0.2600	1.000	0.833	0.909
120 s	n.d.	1.000	95°	0.1434	1.000	1.000	1.000

L'Autoencoder mostra un miglioramento rispetto alla configurazione principale quando vengono utilizzate granularità più brevi. La ROC-AUC SUS passa infatti da 0.667 con finestre di 30 secondi a 0.857 con finestre di 5 secondi e a 0.869 con finestre di 10 secondi. Anche l'F1-score aumenta rispettivamente da 0.625 a 0.867 e 0.818.

Con finestre di 60 secondi si osservano una ROC-AUC SUS pari a 0.833 e un F1-score pari a 0.909, ma il risultato si basa nuovamente su sole 7 finestre di test. A 120 secondi precision, recall e F1-score sono pari a 1.000, mentre la ROC-AUC SUS non è definita. Come per la One-Class SVM, tali valori riflettono il comportamento su un test costituito esclusivamente da quattro finestre positive per SUS e non possono essere utilizzati come prova di una superiorità della granularità più ampia.

Nel complesso, l'analisi evidenzia che la granularità temporale non costituisce un parametro neutro della rappresentazione. Variando la durata della finestra cambiano contemporaneamente il numero di osservazioni disponibili, la composizione delle feature relative e l'aggregazione delle etichette; di conseguenza, cambiano anche le distribuzioni degli anomaly score e le prestazioni osservate. Nelle condizioni considerate, le finestre di 5 e 10 secondi producono per tutti e quattro i detector valori di ROC-AUC SUS superiori rispetto alla configurazione a 30 secondi. Il risultato mostra quindi che le prestazioni osservate sono sensibili alla granularità temporale adottata, senza tuttavia consentire di stabilire quale durata sia ottimale in termini generali.

Tale osservazione non consente tuttavia di individuare una durata "ottimale" della finestra. L'analisi è effettuata sul medesimo dataset e sul medesimo split,

mantenendo fisse le configurazioni dei detector, e non costituisce un nuovo processo di selezione degli iperparametri. Inoltre, al crescere della durata della finestra la numerosità di validation e test diminuisce rapidamente: i valori particolarmente elevati osservati a 60 e 120 secondi sono quindi fortemente sensibili al comportamento di poche singole osservazioni.

I risultati supportano pertanto l'interpretazione dei 30 secondi come una scelta di compromesso adottata nell'esperimento principale, e non come una granularità ottimizzata sulle prestazioni del test. Allo stesso tempo, mostrano che la durata della finestra rappresenta un parametro rilevante della pipeline e che una valutazione su registrazioni più estese sarebbe necessaria per stabilire con maggiore robustezza quale granularità sia preferibile.

I valori di ROC-AUC relativi all'etichetta EVIL sono riportati a scopo descrittivo. Il validation set non contiene infatti finestre EVIL positive per nessuna delle granularità considerate e non è quindi possibile definire una soglia operativa specifica per tale etichetta. Inoltre, alle granularità di 60 e 120 secondi il numero di finestre del test è molto ridotto; valori di ROC-AUC EVIL pari a 1.000 devono pertanto essere interpretati con particolare cautela.

5.4 Risultati dell'Ensemble e analisi delle soglie operative

Per rendere esplicito il processo di selezione delle soglie, la Tabella 5.8 riporta l'F1-score ottenuto sul validation set per ciascuno dei quattro percentili candidati. Poiché la selezione della soglia operativa è stata effettuata sulla base dell'F1-score rispetto all'etichetta `sus`, questi valori permettono di osservare direttamente quale configurazione ha determinato la scelta finale.

I risultati mostrano che, per Isolation Forest, il 95° percentile produce l'F1-score più elevato sulla validation. Per la One-Class SVM il 97° e il 99° percentile raggiungono entrambi un F1-score pari a 0.400 e producono le stesse predizioni; è stato quindi mantenuto il 97° percentile, meno restrittivo. Per l'Autoencoder e per l'Ensemble il 90° percentile è l'unica configurazione con F1-score maggiore di zero. LOF non individua invece nessuna delle due finestre SUS della

Tabella 5.8: F1-score sulla validation per le soglie candidate SUS.

Modello	**90°**	**95°**	**97°**	**99°**	**Selezionato**
Isolation Forest	0.400	0.667	0.000	0.000	95°
Local Outlier Factor	0.000	0.000	0.000	0.000	–
One-Class SVM	0.200	0.333	0.400	0.400	97°
Autoencoder	0.222	0.000	0.000	0.000	90°
Ensemble	0.250	0.000	0.000	0.000	90°

validation con le soglie considerate e non viene quindi associato a una soglia operativa SUS.

La selezione delle soglie operative è stata effettuata sul validation set rispetto all'etichetta `sus`, utilizzando come candidati il 90°, 95°, 97° e 99° percentile della distribuzione degli anomaly score osservati sul training set. La Tabella 5.9 riassume le configurazioni selezionate.

Tabella 5.9: Soglie operative SUS selezionate tramite validation.

Modello	**Percentile TRAIN**	**Soglia**
Isolation Forest	95°	0.041385
Local Outlier Factor	–	–
One-Class SVM	97°	0.988403
Autoencoder	90°	0.226207
Ensemble	90°	0.865820

Per Isolation Forest è stato selezionato il 95° percentile, mentre per la One-Class SVM il 97° e il 99° percentile hanno prodotto sulla validation lo stesso F1-score e le stesse predizioni. È stato quindi mantenuto il 97° percentile, corrispondente alla soluzione meno restrittiva tra le due equivalenti. Per l'Autoencoder è stato selezionato il 90° percentile. Nel caso di LOF, invece, tutte le soglie candidate hanno ottenuto un F1-score pari a zero sulla validation e non è stata quindi definita una soglia operativa SUS.

Per l'Ensemble, la soglia corrispondente al 90° percentile degli ensemble score del training ha ottenuto il miglior F1-score sulla validation. La soglia risultante è pari a circa 0.86582. È opportuno ricordare che il validation set contiene

soltanto due finestre `sus`; di conseguenza, la selezione della soglia dipende da un numero molto ridotto di esempi positivi.

Applicando la configurazione selezionata alle 13 finestre del test set, l'Ensemble identifica correttamente 5 delle 7 finestre `sus`. La matrice di confusione contiene 5 veri positivi, 2 falsi negativi, 3 falsi positivi e 3 veri negativi. Si ottengono quindi una precision pari a 0.625, una recall pari a 0.714 e un F1-score pari a 0.667. La ROC-AUC calcolata sugli ensemble score è invece pari a 0.679 per `sus` e a 0.888 per `evil`.

Il confronto con i detector individuali mostra che la combinazione dei quattro modelli non determina, in questo esperimento, un miglioramento rispetto al miglior detector singolo. La One-Class SVM raggiunge infatti una ROC-AUC pari a 0.690 per `sus` e 0.900 per `evil`, valori leggermente superiori a quelli dell'Ensemble, rispettivamente pari a 0.679 e 0.888. Anche considerando la decisione binaria su `sus`, One-Class SVM ed Ensemble ottengono lo stesso F1-score, pari a circa 0.667, con gli stessi valori di precision e recall.

L'Ensemble consente quindi di integrare in un unico score le informazioni provenienti dai quattro approcci, ma i risultati ottenuti non mostrano un vantaggio prestazionale rispetto alla One-Class SVM. Per questo motivo non vengono apportate ulteriori modifiche ai pesi o alla composizione dell'Ensemble sulla base del test set, poiché ciò comporterebbe un'ottimizzazione retrospettiva sui dati destinati alla valutazione finale.

Analisi esplorativa delle soglie rispetto a EVIL

Poiché il validation set non contiene finestre `evil`, non è possibile applicare lo stesso procedimento utilizzato per selezionare le soglie SUS. Le quattro soglie derivate dai percentili del training sono state comunque applicate al test per osservare, a scopo esplorativo, il comportamento dei singoli detector rispetto alle cinque finestre EVIL presenti.

La Tabella 5.10 riporta precision, recall e F1-score ottenuti applicando ciascuna delle quattro soglie candidate. I valori non rappresentano una selezione della soglia EVIL, ma descrivono esclusivamente il comportamento osservato sul test set.

Tabella 5.10: Analisi esplorativa delle soglie rispetto all'etichetta EVIL sul test set.

Modello	**Percentile**	**Precision**	**Recall**	**F1**
Isolation Forest	90°	0.600	0.600	0.600
	95°	0.667	0.400	0.500
	97°	0.667	0.400	0.500
	99°	0.500	0.200	0.286
LOF	90°	0.625	1.000	0.769
	95°	0.714	1.000	0.833
	97°	0.714	1.000	0.833
	99°	0.600	0.600	0.600
One-Class SVM	90°	0.500	1.000	0.667
	95°	0.625	1.000	0.769
	97°	0.625	1.000	0.769
	99°	0.625	1.000	0.769
Autoencoder	90°	0.556	1.000	0.714
	95°	0.556	1.000	0.714
	97°	0.625	1.000	0.769
	99°	0.625	1.000	0.769

L'analisi evidenzia che LOF, One-Class SVM e Autoencoder riescono, per diverse soglie, a intercettare tutte e cinque le finestre EVIL presenti nel test set. Isolation Forest mostra invece una recall più contenuta, che raggiunge al massimo 0.600 tra le soglie considerate.

LOF raggiunge, alle soglie del 95° e 97° percentile, precision pari a 0.714, recall pari a 1.000 e F1-score pari a 0.833. One-Class SVM e Autoencoder raggiungono anch'essi recall pari a 1.000 per diverse configurazioni, con valori di F1 fino a 0.769.

Tali risultati non devono tuttavia essere utilizzati per individuare quale percentile costituisca la soglia EVIL "migliore". La scelta sulla base delle metriche del test introdurrebbe infatti informazione proveniente dal set di valutazione nel processo di configurazione del sistema. Le metriche riportate nella tabella

hanno quindi esclusivamente valore esplorativo e descrittivo.
Per l'Ensemble non è stata effettuata un'analoga analisi esplorativa threshold-based rispetto a `evil`. La capacità di separazione dell'Ensemble rispetto a questa etichetta viene pertanto descritta attraverso la ROC-AUC calcolata sul test set, pari a 0.888.

5.5 Confronto con la letteratura

Il confronto tra i risultati ottenuti nel presente lavoro e quelli riportati nella letteratura su BETH richiede particolare cautela. Sebbene gli studi considerati utilizzino lo stesso dataset di origine, le procedure di preprocessing, le caratteristiche utilizzate dai modelli, l'unità di analisi e i protocolli di valutazione non coincidono. I valori di ROC-AUC riportati di seguito devono quindi essere interpretati come riferimenti utili per collocare i risultati sperimentali, e non come misure ottenute in condizioni direttamente equivalenti.
Una prima differenza riguarda l'unità sulla quale viene effettuata l'anomaly detection. Nel lavoro originale di Highnam et al. e negli studi di Alatawi e Albalawi ed Eremin, i modelli operano sui singoli eventi kernel-process, opportunamente sottoposti a preprocessing [1, 4, 2]. Nel presente lavoro, invece, gli eventi vengono prima raggruppati in finestre temporali non sovrapposte di 30 secondi. Ciascuna finestra viene successivamente rappresentata mediante le frequenze relative dei 32 tipi di `eventName` individuati nel training set. Le etichette vengono a loro volta aggregate: una finestra è considerata `SUS` o `EVIL` quando contiene almeno un evento con la corrispondente annotazione positiva. Questa trasformazione modifica in modo sostanziale il problema sperimentale. I 188 967 eventi presenti nel test set originale vengono infatti rappresentati nel presente lavoro mediante 13 finestre temporali, delle quali 7 sono etichettate come `SUS` e 5 come `EVIL`. La ROC-AUC calcolata in questo lavoro misura pertanto la capacità di ordinare correttamente finestre temporali aggregate, e non singoli eventi. Inoltre, la ridotta numerosità del test set a livello di finestra rende le stime ottenute particolarmente sensibili al comportamento osservato sui singoli campioni.

Nel benchmark originale di BETH, Highnam et al. confrontano Robust Covariance, One-Class SVM, Isolation Forest e un approccio VAE+DoSE-SVM. La valutazione quantitativa principale viene effettuata rispetto all'etichetta `sus`. Isolation Forest ottiene il valore di ROC-AUC più elevato, pari a 0.850, mentre One-Class SVM e VAE+DoSE-SVM raggiungono rispettivamente 0.605 e 0.698 [1]. Il confronto con i risultati del presente lavoro non è però diretto. Oltre alla diversa unità di analisi, la One-Class SVM utilizzata da Highnam et al. è una variante lineare addestrata mediante stochastic gradient descent, mentre nel presente lavoro viene utilizzata una One-Class SVM con kernel RBF. Analogamente, l'approccio VAE+DoSE-SVM del lavoro originale è metodologicamente differente dall'Autoencoder utilizzato in questa tesi.

Nel presente lavoro, rispetto all'etichetta `SUS`, Isolation Forest raggiunge una ROC-AUC pari a 0.643, la One-Class SVM 0.690 e l'Autoencoder 0.667. Il confronto numerico mostra quindi valori differenti rispetto al benchmark originale, ma tali differenze non possono essere attribuite esclusivamente alla qualità dei singoli algoritmi: cambiano infatti sia la rappresentazione in ingresso sia la granularità alla quale viene definita e valutata l'anomalia.

Un ulteriore riferimento è rappresentato dallo studio di Alatawi e Albalawi, che applica Isolation Forest al sottoinsieme kernel-process di BETH. Gli autori utilizzano sette caratteristiche dei singoli eventi, applicano una fase di preprocessing e standardizzazione e configurano Isolation Forest con 100 alberi e `contamination`=0.01. La valutazione utilizza `sus` come etichetta di riferimento e produce una ROC-AUC pari a 0.917, rispetto allo 0.850 del benchmark originale [4]. Lo stesso studio riporta inoltre una precision del 99.57%, una recall dell'84.97% e un F1-score del 91.69%.

Anche in questo caso il confronto con le metriche del presente lavoro deve essere effettuato con cautela. Alatawi e Albalawi classificano i singoli eventi del test set, mentre nel presente lavoro la valutazione viene effettuata sulle finestre temporali. Inoltre, la decisione binaria adottata nel loro esperimento deriva dalla configurazione dell'Isolation Forest con `contamination`=0.01, mentre in questa tesi le soglie operative vengono ricavate come percentili degli anomaly score del training set e selezionate successivamente attraverso il validation set. Di conseguenza, anche precision, recall e F1-score non rappresentano metriche

direttamente confrontabili tra i due esperimenti.

Eremin affronta invece BETH secondo un'impostazione differente, trattando gli eventi come un flusso e confrontando dieci algoritmi di anomaly detection online o semi-online [2]. A differenza del presente lavoro, il validation set non viene utilizzato e le metriche vengono calcolate sul test set separatamente rispetto alle etichette `SUS` ed `EVIL`. Lo studio analizza inoltre più configurazioni, modificando l'ordine degli eventi e l'arricchimento delle feature con informazioni relative al processo padre.

Tra i risultati riportati da Eremin, IForestASD raggiunge valori massimi di ROC-AUC pari a 0.706 per `SUS` e 0.719 per `EVIL`; Incremental LOF raggiunge rispettivamente 0.950 e 0.978, mentre la versione stocastica di One-Class SVM raggiunge valori massimi pari a 0.766 per `SUS` e 0.711 per `EVIL`. I risultati più elevati tra gli algoritmi esaminati sono ottenuti da KitNet, con una ROC-AUC massima pari a 0.990 rispetto a `SUS` e 0.994 rispetto a `EVIL` [2]. Questi massimi non corrispondono necessariamente alla medesima configurazione sperimentale: lo stesso studio mostra infatti che l'ordinamento degli eventi e l'arricchimento delle caratteristiche possono modificare le prestazioni dei diversi algoritmi.

La Tabella 5.11 riassume alcuni dei risultati maggiormente rilevanti per il confronto. I valori relativi a Eremin corrispondono al massimo riportato per ciascuna etichetta tra le configurazioni sperimentate.

I risultati mostrano che, nella rappresentazione a finestre adottata nel presente lavoro, la One-Class SVM è il detector individuale con la maggiore capacità discriminante sia rispetto a `SUS`, con ROC-AUC pari a 0.690, sia rispetto a `EVIL`, con ROC-AUC pari a 0.900. L'Ensemble raggiunge rispettivamente 0.679 e 0.888 e non determina quindi un miglioramento rispetto al miglior detector individuale.

È inoltre interessante osservare che, nel presente lavoro, tutti i detector ottengono una ROC-AUC più elevata rispetto a `EVIL` che rispetto a `SUS`. Tale comportamento indica che, sul test set considerato, la rappresentazione basata sulle frequenze degli eventi nelle finestre temporali produce anomaly score che separano più efficacemente le finestre contenenti attività marcata come `EVIL`. Anche questo risultato deve tuttavia essere interpretato considerando la

Tabella 5.11: Confronto tra alcuni risultati di ROC-AUC riportati nella letteratura su BETH e quelli ottenuti nel presente lavoro.

Studio	**Metodo**	**Unità di analisi**	**SUS**	**EVIL**
Highnam et al. [1]	One-Class SVM lineare	Evento	0.605	–
Highnam et al. [1]	Isolation Forest	Evento	0.850	–
Highnam et al. [1]	VAE+DoSE-SVM	Evento	0.698	–
Alatawi e Albalawi [4]	Isolation Forest	Evento	0.917	–
Eremin [2]	IForestASD	Evento, streaming	0.706	0.719
Eremin [2]	Incremental LOF	Evento, streaming	0.950	0.978
Eremin [2]	OCSVM stocastica	Evento, streaming	0.766	0.711
Eremin [2]	KitNet	Evento, streaming	0.990	0.994
Presente lavoro	Isolation Forest	Finestra 30 s	0.643	0.825
Presente lavoro	Local Outlier Factor	Finestra 30 s	0.548	0.825
Presente lavoro	One-Class SVM RBF	Finestra 30 s	0.690	0.900
Presente lavoro	Autoencoder	Finestra 30 s	0.667	0.875
Presente lavoro	Ensemble	Finestra 30 s	0.679	0.888

dimensione molto ridotta del test set a livello di finestra.
Nel complesso, il confronto con la letteratura evidenzia che i risultati del presente lavoro devono essere interpretati in relazione alla specifica rappresentazione adottata. In questo caso, l'unità analizzata non è il singolo evento kernel, ma una finestra temporale di 30 secondi descritta attraverso le frequenze relative dei diversi tipi di evento. Le prestazioni riportate misurano quindi l'efficacia dei detector su questa particolare rappresentazione dei dati. La fase di anomaly detection costituisce inoltre il primo stadio del sistema proposto, al quale segue il triage delle finestre selezionate mediante LLM.

5.6 Risultati del triage LLM e confronto tra le due pipeline

Dopo aver analizzato le prestazioni dei singoli detector e dell'Ensemble, vengono presentati i risultati relativi alle due pipeline descritte nelle Sezioni 4.8 e 4.9, secondo il protocollo di confronto definito nella Sezione 4.10.

5.6.1 Risultati della pipeline detector + LLM

Applicando le soglie operative definite sul validation set, lo stadio locale ha selezionato 9 delle 13 finestre presenti nel test set. La selezione è determinata dall'unione degli alert prodotti da Isolation Forest, One-Class SVM, Autoencoder ed Ensemble. LOF non produce direttamente una decisione binaria, poiché sul validation set non è stato possibile individuare una soglia SUS con F1 maggiore di zero; il suo contributo rimane tuttavia presente indirettamente nello score dell'Ensemble.
Rispetto all'etichetta SUS, le 9 finestre selezionate comprendono 5 delle 7 finestre positive presenti nel test set. La selezione operativa produce quindi 5 veri positivi, 4 falsi positivi, 2 falsi negativi e 2 veri negativi. Ne risultano una precision pari a 0.556, una recall pari a 0.714 e un F1-score pari a 0.625.
È importante distinguere questa valutazione dalla successiva attività svolta dall'LLM. Le metriche appena riportate misurano infatti la capacità dello stadio di detection di decidere quali finestre inoltrare al triage, mentre il modello

linguistico riceve soltanto le finestre selezionate e produce per ciascuna di esse un report interpretativo e un livello di priorità.
Le priorità assegnate dall'LLM risultano distribuite uniformemente tra i tre livelli previsti: 3 finestre sono state classificate come `low`, 3 come `medium` e 3 come `high`. Il confronto a posteriori con le ground truth evidenzia una differenza tra i tre gruppi, come riportato in Tabella 5.12.

Tabella 5.12: Distribuzione delle priorità prodotte dall'LLM e confronto descrittivo con le etichette del test set.

Priorità	**Finestre**	**SUS positive**	**EVIL positive**
Low	3	0	0
Medium	3	2	2
High	3	3	3

Le tre finestre con priorità `low` non risultano annotate né come SUS né come EVIL. Tra le finestre con priorità `medium`, due risultano invece positive per entrambe le etichette, mentre una è negativa. Infine, tutte le tre finestre classificate con priorità `high` risultano annotate sia come SUS sia come EVIL. Questa corrispondenza deve essere interpretata esclusivamente in modo descrittivo. I livelli `low`, `medium` e `high` rappresentano infatti categorie operative di triage e non sono stati calibrati utilizzando le ground truth del dataset. Inoltre, l'analisi riguarda soltanto 9 finestre e non permette quindi di attribuire alle priorità prodotte dal modello un significato statistico generalizzabile.
La pipeline presenta inoltre un limite strutturale legato alla selezione iniziale: le finestre SUS che non superano lo stadio di anomaly detection non raggiungono mai il modello linguistico. Nel test set, due delle sette finestre SUS non sono state selezionate e non hanno quindi ricevuto alcun triage. L'LLM può pertanto contribuire alla contestualizzazione e alla prioritizzazione degli alert ricevuti, ma non può recuperare una finestra esclusa a monte dallo stadio di detection.

5.6.2 Confronto con la pipeline LLM-only

Nella configurazione LLM-only tutte le 13 finestre del test set sono state analizzate direttamente dal modello linguistico, senza utilizzare anomaly score, soglie operative, alert dei detector o informazioni di consenso. In questo caso il campo `anomalous` prodotto dall'LLM costituisce direttamente la decisione binaria utilizzata per il confronto con l'etichetta SUS.

Il modello ha identificato come anomale soltanto 2 delle 7 finestre SUS presenti nel test set. Entrambe le segnalazioni corrispondono a finestre effettivamente positive, per cui non vengono prodotti falsi positivi. Si ottengono quindi 2 veri positivi, 0 falsi positivi, 5 falsi negativi e 6 veri negativi, con precision pari a 1.000, recall pari a 0.286 e F1-score pari a 0.444.

La Figura 5.9 mostra la matrice di confusione ottenuta dalla pipeline LLM-only rispetto all'etichetta `SUS`.

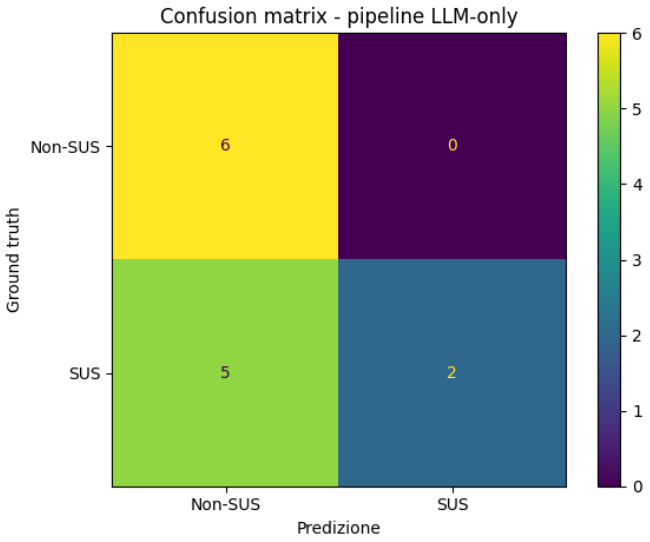


Figura 5.9: Matrice di confusione della pipeline LLM-only rispetto all'etichetta `SUS` sul test set.

La matrice evidenzia il comportamento selettivo della pipeline LLM-only: tutte le 6 finestre non-SUS vengono correttamente classificate come non anomale e non si osservano falsi positivi. Al contrario, soltanto 2 delle 7 finestre SUS vengono identificate come anomale, mentre le restanti 5 costituiscono falsi negativi. Questo comportamento è coerente con la precision pari a 1.000 e con la recall più contenuta, pari a 0.286.

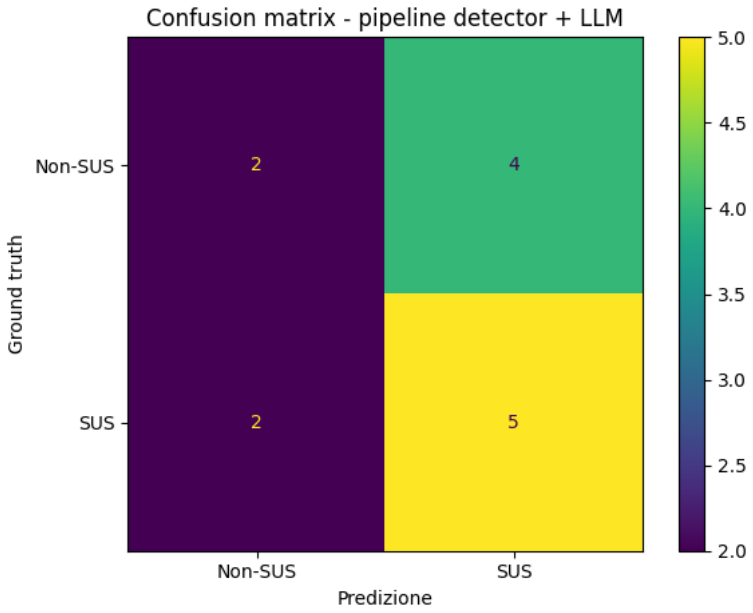


Figura 5.10: Matrice di confusione della pipeline detector + LLM rispetto all'etichetta SUS sul test set.

La Figura 5.10 mostra la matrice di confusione della pipeline detector + LLM rispetto all'etichetta SUS. In questo caso si osservano 5 veri positivi, 4 falsi positivi, 2 veri negativi e 2 falsi negativi. La pipeline ibrida riesce quindi a individuare una quota maggiore di finestre SUS rispetto alla configurazione LLM-only, ma al prezzo di un numero più elevato di falsi positivi. Questo comportamento è coerente con i valori di recall e precision osservati: la pipeline detector + LLM risulta più orientata alla copertura delle finestre sospette, mentre la pipeline LLM-only appare più conservativa.
La Tabella 5.13 riassume il confronto tra le decisioni di rilevazione delle due configurazioni.

Tabella 5.13: Confronto delle prestazioni rispetto all'etichetta `SUS` sul test set.

Pipeline	**TP**	**FP**	**FN**	**TN**	**Precision**	**Recall**	**F1**	**Accuracy**
LLM-only	2	0	5	6	1.000	0.286	0.444	0.615
Detector + LLM	5	4	2	2	0.556	0.714	0.625	0.538

Le due configurazioni mostrano quindi comportamenti differenti. La pipeline LLM-only segnala un numero ridotto di finestre e non produce falsi positivi nel campione considerato, ma non individua 5 delle 7 finestre SUS. La pipeline

detector + LLM presenta invece una maggiore copertura delle finestre SUS, con recall pari a 0.714, accompagnata però da un incremento dei falsi positivi. L'accuracy risulta leggermente superiore per la pipeline LLM-only, ma questo valore deve essere interpretato con cautela. Una strategia che classifica come non anomala gran parte delle finestre può infatti ottenere una buona accuracy pur presentando una recall ridotta sulle anomalie. Per questo motivo, nel presente contesto precision, recall, F1-score e matrice di confusione risultano più informative dell'accuracy considerata isolatamente.

5.6.3 Analisi descrittiva rispetto all'etichetta EVIL

Poiché il validation set non contiene finestre positive per l'etichetta EVIL, non è stata definita una soglia operativa specifica per questo target. Il confronto rispetto a EVIL viene pertanto mantenuto separato dalle metriche principali e utilizzato esclusivamente come analisi descrittiva.

Nel test set sono presenti 5 finestre EVIL. La pipeline LLM-only ne identifica come anomale 2, mentre lo stadio locale della pipeline detector + LLM include tutte e 5 le finestre EVIL tra le 9 selezionate per il triage.

Questo risultato non corrisponde a una recall EVIL validata. Le finestre della pipeline ibrida sono infatti state selezionate utilizzando soglie definite rispetto a SUS e non rispetto a EVIL. L'osservazione indica soltanto che, nello specifico test set considerato, tutte le finestre annotate come EVIL sono state incluse nell'insieme sottoposto al successivo triage LLM.

5.7 Analisi dei tempi e delle risorse computazionali

Oltre alle prestazioni di rilevazione, è stato analizzato il costo operativo delle due pipeline. Per la componente locale sono stati misurati il tempo reale di esecuzione (*wall time*), il tempo CPU e l'incremento della memoria residente del processo Python. Per le chiamate all'LLM, eseguite tramite API remota, è stata invece considerata la latenza end-to-end osservata dal client.

5.7.1 Costo computazionale dei detector locali

Il benchmark dei singoli detector è stato eseguito ripetendo 100 volte l'inferenza sull'intero test set, costituito da 13 finestre. Ogni esecuzione comprende la standardizzazione delle feature e il calcolo dell'anomaly score, utilizzando modelli e scaler già addestrati.
La Tabella 5.14 riporta il tempo medio necessario per elaborare le 13 finestre, il corrispondente tempo CPU medio e l'incremento massimo di memoria RSS osservato durante il benchmark.

Tabella 5.14: Costo computazionale locale dei singoli detector sul test set di 13 finestre. I tempi rappresentano la media ottenuta su 100 ripetizioni.

Modello	**Wall time [ms]**	**CPU time [ms]**	**Incremento RSS [MB]**
Isolation Forest	43.61	30.10	0.012
LOF	7.42	4.00	0.004
One-Class SVM	7.54	3.40	0.004
Autoencoder	283.41	138.40	0.090

Tra i quattro modelli, LOF e One-Class SVM presentano i tempi di esecuzione più contenuti, mentre Isolation Forest richiede un tempo maggiore. L'Autoencoder rappresenta il componente localmente più costoso, con un tempo medio di circa 283 ms per l'elaborazione delle 13 finestre.
Le misure di memoria devono essere interpretate con cautela. Il benchmark è stato eseguito in un processo Python nel quale librerie, dataset e modelli erano già caricati. L'incremento di RSS riportato in Tabella 5.14 non rappresenta quindi la memoria totale richiesta da ciascun modello, ma solamente l'aumento massimo della memoria residente osservato durante l'esecuzione dell'inferenza.

5.7.2 Costo dello stadio locale della pipeline detector + LLM

Separatamente dal benchmark dei singoli detector, è stato misurato anche il costo dell'intero stadio locale della pipeline detector + LLM considerato nel suo complesso.

In questo caso il benchmark comprende, in un'unica esecuzione, la standardizzazione delle feature, l'inferenza dei quattro detector, la normalizzazione degli anomaly score, la costruzione dell'Ensemble, l'applicazione delle soglie operative e la selezione finale delle finestre da inoltrare al triage.
L'intero stadio è stato ripetuto 100 volte sul test set di 13 finestre. Il tempo medio necessario per una singola elaborazione completa del test set è risultato pari a circa 327.6 ms di wall time e 174.4 ms di tempo CPU. L'incremento massimo di memoria RSS osservato durante il benchmark è pari a circa 1.13 MB.
Questa misura deve essere distinta dai tempi riportati per i singoli detector nella Tabella 5.14: in questo caso, infatti, viene misurato il costo complessivo della pipeline locale, comprendendo anche le operazioni successive all'inferenza dei modelli.
Il costo dello stadio locale risulta comunque ridotto rispetto alle latenze associate alle successive chiamate al servizio LLM, introducendo un overhead inferiore a un secondo sull'intero test set considerato.

5.7.3 Latenza del triage LLM nella pipeline detector + LLM

Dopo la selezione effettuata dallo stadio locale, 9 delle 13 finestre del test set sono state inoltrate all'LLM per il triage. Per ciascuna chiamata è stata misurata la latenza end-to-end osservata dal client, dal momento immediatamente precedente all'invio della richiesta fino alla ricezione della risposta.
Le 9 chiamate hanno richiesto complessivamente 94.696 s. La latenza media per chiamata è risultata pari a circa 10.522 s, con una mediana di 10.899 s. Il valore minimo osservato è pari a 7.673 s, mentre il massimo è pari a 11.678 s. Questi tempi comprendono non soltanto l'elaborazione effettuata dal modello remoto, ma anche la comunicazione di rete e gli altri fattori legati all'infrastruttura del servizio. Non devono pertanto essere interpretati come tempi di inferenza puri dell'LLM.
Considerando anche il costo dello stadio locale, pari a circa 0.328 s, il tempo end-to-end osservato per la pipeline detector + LLM sull'intero test set è pari

a circa 95.024 s.

5.7.4 Confronto temporale tra le due pipeline

La differenza principale tra le due configurazioni emerge considerando le chiamate effettuate all'LLM. Nella pipeline LLM-only tutte le 13 finestre del test set vengono inviate direttamente al modello. Nella pipeline detector + LLM, invece, lo stadio locale seleziona 9 finestre, limitando a queste ultime l'esecuzione del triage remoto.
La Tabella 5.15 riporta il confronto temporale complessivo osservato.

Tabella 5.15: Confronto dei tempi osservati nelle due pipeline sul test set.

Pipeline	**Finestre**	**Chiamate LLM**	**Detection locale [s]**	**Tempo end-to-end [s]**
LLM-only	13	13	0.000	373.940
Detector + LLM	13	9	0.328	95.024

Nella pipeline LLM-only, la somma delle latenze delle 13 chiamate è pari a 373.940 s. Nella pipeline detector + LLM, le 9 chiamate di triage richiedono complessivamente 94.696 s; aggiungendo circa 0.328 s per lo stadio locale, il tempo end-to-end osservato è pari a circa 95.024 s.
Nel campione analizzato, la pipeline ibrida presenta quindi un tempo complessivo sensibilmente inferiore. Tale differenza non può tuttavia essere attribuita in modo causale esclusivamente alla riduzione del numero di chiamate da 13 a 9. Le richieste al servizio remoto sono state infatti effettuate in momenti differenti e la loro latenza può dipendere anche da condizioni di rete, carico del provider e altri fattori esterni all'esperimento.
Le misure relative all'LLM rappresentano inoltre latenze end-to-end osservate dal client e non il solo tempo interno di inferenza del modello. Poiché Gemini viene eseguito su infrastruttura remota, non è possibile misurare direttamente CPU, GPU o memoria utilizzate lato server. Il confronto delle risorse computazionali rimane pertanto limitato alla componente locale della pipeline e alle latenze remote osservabili dal client.

5.8 Discussione complessiva dei risultati

I risultati dei detector possono essere letti da due punti di vista. La ROC-AUC valuta quanto bene gli anomaly score distinguono le finestre positive da quelle negative, senza fissare una soglia specifica. Precision, recall e F1-score descrivono invece il comportamento delle decisioni binarie ottenute dopo l'applicazione della soglia operativa selezionata sul validation set.

Sul target principale `SUS`, la One-Class SVM ha ottenuto la ROC-AUC più elevata tra i detector individuali, pari a 0.690, e ha raggiunto un F1-score di 0.667 dopo l'applicazione della soglia selezionata sul validation set. L'Autoencoder ha mostrato un comportamento simile in termini di copertura delle anomalie, con recall pari a 0.714, ma una precision inferiore e un F1-score pari a 0.625. Isolation Forest ha identificato soltanto 2 delle 7 finestre SUS presenti nel test set, ottenendo una precision pari a 0.667, una recall pari a 0.286 e un F1-score pari a 0.400, mentre per LOF non è stato possibile definire una soglia operativa SUS sulla base dei risultati ottenuti in validation.

La combinazione dei quattro detector mediante Ensemble non ha determinato, nel presente esperimento, un miglioramento rispetto al miglior modello individuale. L'Ensemble ha raggiunto una ROC-AUC SUS pari a 0.679 e un F1-score pari a 0.667, ottenendo quindi le stesse metriche binarie della One-Class SVM ma una capacità di ordinamento leggermente inferiore. Il vantaggio dell'Ensemble risiede pertanto principalmente nella possibilità di integrare in un unico score informazioni provenienti da modelli differenti, piuttosto che in un incremento misurabile delle prestazioni sul test set considerato.

Un altro risultato riguarda la diversa capacità di separazione osservata rispetto alle due etichette. Per tutti i detector, la ROC-AUC calcolata rispetto a `EVIL` è risultata superiore a quella ottenuta rispetto a `SUS`. Nel test set considerato, gli anomaly score hanno quindi distinto meglio le finestre EVIL dalle finestre non-EVIL rispetto a quanto abbiano distinto le finestre SUS dalle finestre non-SUS. Questo confronto rimane tuttavia descrittivo, poiché l'assenza di finestre EVIL positive nel validation set non ha consentito di definire una soglia operativa specifica per tale etichetta.

Il confronto tra le due pipeline mette inoltre in evidenza il diverso ruolo che

può essere assegnato all'LLM. Nella configurazione LLM-only, il modello ha adottato un comportamento particolarmente selettivo: le due finestre indicate come anomale risultano entrambe positive rispetto a `SUS`, determinando una precision pari a 1.000, ma cinque delle sette finestre SUS non vengono individuate. La recall risulta pertanto pari a 0.286 e l'F1-score a 0.444.

La pipeline detector + LLM presenta invece un comportamento orientato a una maggiore copertura. Lo stadio locale seleziona 5 delle 7 finestre SUS, ottenendo una recall pari a 0.714 e un F1-score pari a 0.625, a fronte della presenza di quattro falsi positivi. Nel contesto dell'esperimento, l'utilizzo dei detector come primo livello di selezione consente quindi di inoltrare al triage una quota maggiore delle finestre annotate come sospette, accettando però un aumento del numero di finestre non-SUS sottoposte all'analisi successiva.

Per quanto riguarda il triage, nelle 9 finestre analizzate dall'LLM si osserva una relazione tra il livello di priorità assegnato e le annotazioni del dataset. Le tre finestre con priorità `high` risultano tutte annotate sia come SUS sia come EVIL, mentre le tre finestre con priorità `low` non presentano nessuna delle due annotazioni. Tra le tre finestre con priorità `medium`, due risultano positive per entrambe le etichette.

Il risultato riguarda tuttavia un numero molto ridotto di finestre e viene quindi riportato come osservazione dello specifico esperimento svolto.

Dal punto di vista operativo, il costo della fase locale di anomaly detection risulta ridotto rispetto alle latenze associate alle chiamate al servizio LLM. Nel test considerato, la pipeline detector + LLM ha richiesto 9 chiamate remote e un tempo end-to-end complessivo di circa 95.0 s, mentre la configurazione LLM-only ha richiesto 13 chiamate e circa 373.9 s. La differenza osservata mostra il potenziale vantaggio di utilizzare uno stadio locale come filtro prima del triage remoto. Non è tuttavia possibile attribuire l'intera differenza temporale esclusivamente alla riduzione del numero di chiamate, poiché le latenze del servizio remoto hanno mostrato una variabilità significativa e possono dipendere da condizioni esterne all'esperimento.

Nel complesso, i risultati supportano l'uso dell'LLM come componente successiva a uno stadio di anomaly detection, con un ruolo distinto da quello dei detector. I modelli locali producono una decisione riproducibile basata sugli

anomaly score e sulle soglie definite durante la fase di validation, mentre l'LLM viene impiegato per interpretare e prioritizzare le finestre selezionate. La configurazione LLM-only rappresenta invece un'impostazione differente, nella quale allo stesso modello viene affidata sia la decisione di anomalia sia la successiva attività di triage.

Le conclusioni devono infine essere considerate alla luce delle dimensioni ridotte dei dati disponibili dopo la costruzione delle finestre temporali. Il validation set contiene soltanto 22 finestre, di cui due positive per `SUS`, mentre il test set comprende 13 finestre, di cui sette SUS e cinque EVIL. La selezione delle soglie risulta quindi inevitabilmente sensibile a un numero molto limitato di esempi positivi. Analogamente, il confronto tra le due pipeline e l'analisi delle priorità prodotte dall'LLM devono essere interpretati come risultati descrittivi dello specifico esperimento svolto e non come stime generalizzabili delle prestazioni su scenari differenti.

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Conclusioni

Il presente lavoro ha analizzato l'impiego di tecniche di anomaly detection non supervisionata su dati di telemetria a livello kernel, con l'obiettivo di valutare una pipeline nella quale la rilevazione delle anomalie viene affiancata da una successiva fase di triage assistita da Large Language Model. A questo scopo è stato utilizzato il dataset BETH, i cui eventi sono stati aggregati in finestre temporali e rappresentati mediante le frequenze relative dei diversi tipi di evento osservati.

Sulla rappresentazione ottenuta sono stati confrontati quattro modelli di anomaly detection basati su Isolation Forest, Local Outlier Factor, One-Class SVM e Autoencoder, scelti perché basati su criteri differenti per individuare le anomalie. Isolation Forest utilizza la facilità con cui un'osservazione può essere isolata rispetto al resto dei dati, mentre LOF valuta il comportamento di ciascun punto rispetto alla densità del proprio vicinato. La One-Class SVM costruisce invece una frontiera che descrive la regione occupata dai dati di riferimento e, attraverso l'impiego di kernel, può rappresentare anche frontiere non lineari. L'Autoencoder adotta infine un criterio basato sull'errore di ricostruzione e può modellare relazioni non lineari tra le feature.

Oltre ai singoli detector è stato considerato un ensemble costruito a partire dai rispettivi anomaly score. Le soglie operative sono state selezionate sul validation set e la valutazione finale è stata mantenuta separata sul test set, utilizzando principalmente l'etichetta SUS come riferimento per le decisioni

binarie.

Nel contesto sperimentale considerato, la One-Class SVM è risultata il detector individuale con la maggiore capacità discriminante rispetto all'etichetta SUS, raggiungendo una ROC-AUC pari a 0.690 e un F1-score pari a 0.667. Il risultato è coerente con il ruolo della One-Class SVM come metodo consolidato per l'anomaly detection: il modello apprende una frontiera che descrive la regione occupata dai dati di riferimento e, attraverso l'impiego di kernel come l'RBF, può rappresentare regioni decisionali non lineari [5]. L'Autoencoder ha ottenuto una ROC-AUC pari a 0.667 e una recall pari a 0.714, con un F1-score pari a 0.625. Isolation Forest ha raggiunto una ROC-AUC pari a 0.643, individuando 2 delle 7 finestre SUS presenti nel test set. LOF ha invece ottenuto una ROC-AUC pari a 0.548; per questo modello, nessuna delle soglie candidate ha individuato le due finestre SUS presenti nel validation set e tutti i valori di F1-score sono risultati pari a zero. Questo comportamento può essere messo in relazione con la natura locale del metodo: LOF valuta infatti ciascuna osservazione confrontandone la densità con quella del proprio vicinato, e il risultato dipende dalla struttura locale dei dati e dal numero di vicini considerato [10]. Nel presente esperimento, quindi, le finestre annotate come SUS nel validation set non hanno ricevuto score sufficientemente elevati rispetto alle soglie derivate dal training. Tale risultato deve inoltre essere interpretato con cautela, poiché il validation set contiene soltanto due finestre positive per SUS. L'Ensemble, introdotto per integrare in un unico score informazioni provenienti da criteri di rilevazione differenti, non ha determinato un miglioramento rispetto al miglior detector individuale. La One-Class SVM ha ottenuto una ROC-AUC SUS pari a 0.690, mentre l'Ensemble ha raggiunto 0.679; dopo l'applicazione delle soglie operative, entrambi hanno inoltre ottenuto precision pari a 0.625, recall pari a 0.714 e F1-score pari a 0.667. Nel presente esperimento, la combinazione degli score ha quindi fornito un'informazione aggregata sui diversi detector, senza tradursi in un incremento misurabile delle prestazioni.

Un ulteriore risultato riguarda il comportamento degli anomaly score rispetto all'etichetta EVIL, che nel dataset identifica attività malevola non presente nei dati di training. Sebbene non sia stato possibile definire una soglia operativa specifica per questa etichetta, a causa dell'assenza di esempi EVIL nel valida-

Modello	AUC SUS	AUC EVIL	Precision SUS	Recall SUS	F1 SUS
Isolation Forest	0.643	0.825	**0.667**	0.286	0.400
LOF	0.548	0.825	–	–	–
One-Class SVM	**0.690**	**0.900**	0.625	**0.714**	**0.667**
Autoencoder	0.667	0.875	0.556	**0.714**	0.625
Ensemble	0.679	0.888	0.625	**0.714**	**0.667**

Tabella 6.1: Sintesi delle prestazioni dei detector sul test set. Precision, Recall e F1-score sono riferiti all'etichetta SUS e alle soglie operative selezionate sul validation set. Per LOF non è stata definita una soglia operativa SUS.

tion set, la ROC-AUC consente comunque di valutare a posteriori la capacità degli score di assegnare valori più elevati alle finestre EVIL. Nel test set, tutti i detector hanno mostrato una ROC-AUC rispetto a EVIL superiore a quella ottenuta rispetto a SUS. Il risultato indica quindi che, nel campione analizzato, le finestre associate all'attività malevola risultano maggiormente separabili attraverso gli anomaly score. Tale osservazione rimane comunque descrittiva e non equivale alla definizione di un detector operativo specifico per EVIL.
I valori di ROC-AUC ottenuti non sono direttamente confrontabili con quelli riportati nei lavori precedenti su BETH, poiché differiscono la rappresentazione dei dati, l'unità di analisi e, in alcuni casi, le configurazioni dei modelli. Nel presente lavoro, infatti, la valutazione viene effettuata su finestre temporali aggregate di 30 secondi, mentre i principali studi di riferimento operano sui singoli eventi kernel-process. Il confronto con la letteratura deve pertanto essere interpretato come indicativo e utile a contestualizzare i risultati, piuttosto che come un confronto diretto delle prestazioni.
L'analisi di sensibilità effettuata sulla durata delle finestre ha inoltre mostrato che le prestazioni dei detector dipendono in misura non trascurabile dalla granularità temporale adottata. In particolare, le finestre di 5 e 10 secondi hanno prodotto, nel campione considerato, valori di ROC-AUC SUS superiori rispetto alla configurazione principale di 30 secondi per tutti i detector individuali. Le granularità di 60 e 120 secondi hanno invece determinato una forte

riduzione del numero di osservazioni disponibili, rendendo le metriche particolarmente sensibili ai singoli campioni e, nel caso dei 120 secondi, impedendo il calcolo della ROC-AUC SUS. La configurazione a 30 secondi deve quindi essere interpretata come il compromesso adottato per l'esperimento principale e non come una durata ottimale determinata empiricamente.

La seconda parte del lavoro ha riguardato il confronto tra due modalità di impiego dell'LLM. Nella pipeline detector + LLM, i detector locali svolgono una prima selezione delle finestre da sottoporre all'LLM, mentre nella configurazione LLM-only tutte le finestre vengono analizzate direttamente dall'LLM, al quale viene affidata anche la decisione di anomalia.

Nel test considerato, lo stadio locale della pipeline detector + LLM ha selezionato 5 delle 7 finestre SUS, con precision pari a 0.556, recall pari a 0.714 e F1-score pari a 0.625. La configurazione LLM-only ha invece segnalato come anomale soltanto 2 finestre, entrambe effettivamente SUS, ottenendo una precision pari a 1.000, una recall pari a 0.286 e un F1-score pari a 0.444.

La pipeline ibrida presenta tuttavia un limite legato alla selezione iniziale: le 2 finestre SUS non individuate dallo stadio di anomaly detection non vengono inoltrate all'LLM e non possono quindi essere recuperate nella fase successiva di triage. Il comportamento complessivo della pipeline dipende pertanto anche dalla capacità dei detector di non escludere a monte finestre effettivamente sospette.

Le due configurazioni mostrano quindi un comportamento differente: nel campione analizzato, la pipeline LLM-only è risultata più selettiva, mentre l'impiego dei detector come primo livello ha consentito di raggiungere una quota maggiore delle finestre annotate come sospette, al prezzo di un numero più elevato di falsi positivi.

Nella pipeline ibrida, l'LLM è stato utilizzato per interpretare e assegnare una priorità alle 9 finestre selezionate dallo stadio di anomaly detection. Nel campione analizzato emerge una relazione tra la priorità assegnata e le annotazioni del dataset: le tre finestre classificate come *high* risultano tutte positive sia per SUS sia per EVIL, mentre nessuna delle tre finestre classificate come *low* presenta tali annotazioni. Tra le tre finestre con priorità *medium*, due risultano positive per entrambe le etichette.

Questo andamento suggerisce che, nelle finestre effettivamente sottoposte al triage, il livello di priorità prodotto dall'LLM risulta coerente con la presenza di attività annotate come sospette o malevole. Il risultato deve tuttavia essere considerato esclusivamente descrittivo, poiché riguarda soltanto 9 finestre e i livelli *low*, *medium* e *high* non sono stati calibrati rispetto alle ground truth SUS ed EVIL.

Anche dal punto di vista dei tempi osservati emerge una differenza tra le due configurazioni. La pipeline detector + LLM ha richiesto 9 chiamate al servizio LLM e un tempo end-to-end complessivo di circa 95.0 secondi da aggiungere a 0.328 secondi richiesti dallo stadio locale della pipeline, mentre la configurazione LLM-only ha richiesto 13 chiamate e circa 373.9 secondi. La differenza nei tempi osservati non può tuttavia essere ricondotta unicamente al minor numero di chiamate effettuate, poiché la latenza del servizio remoto può essere influenzata anche da fattori esterni, come le condizioni di rete e il carico del provider.

6.2 Limiti del lavoro

La valutazione sperimentale è condizionata dalla ridotta numerosità dei dati ottenuti dopo la costruzione delle finestre temporali. Il validation set contiene 22 finestre, di cui soltanto due positive per SUS, mentre il test set comprende 13 finestre, sette SUS e cinque EVIL. Di conseguenza, la selezione delle soglie e il confronto tra le configurazioni considerate dipendono da un numero limitato di esempi e i risultati non possono essere considerati stime generalizzabili delle prestazioni.

Un secondo limite riguarda l'etichetta EVIL. Nel validation set non sono presenti finestre positive per questa classe e non è stato quindi possibile definire una soglia operativa specifica. I risultati relativi a EVIL devono pertanto essere considerati esclusivamente descrittivi.

La pipeline detector + LLM dipende inoltre dalla selezione effettuata dallo stadio di anomaly detection: una finestra non individuata dai detector non viene inoltrata all'LLM e non può quindi essere recuperata durante il triage.

Infine, il confronto dei tempi tra le due configurazioni basate su LLM deve essere interpretato con cautela. Le latenze delle chiamate al servizio remoto possono infatti dipendere anche da fattori esterni all'esperimento, come le condizioni di rete e il carico del provider; la differenza osservata non può quindi essere attribuita unicamente al diverso numero di chiamate effettuate.

6.3 Sviluppi futuri

Un primo sviluppo riguarda l'estensione della valutazione a un numero maggiore di host compromessi e a differenti scenari di attacco. Nel presente lavoro, infatti, il test set è relativo a un singolo host compromesso e a uno specifico scenario riconducibile all'installazione di un nodo di botnet. Una valutazione più ampia permetterebbe di verificare se il comportamento osservato dei detector e della pipeline rimane stabile anche in condizioni differenti.
La disponibilità di un validation set contenente anche esempi positivi per l'etichetta EVIL consentirebbe inoltre di definire una soglia operativa specifica per questa classe e di effettuare un confronto quantitativo più completo rispetto a quello descrittivo svolto nel presente lavoro.
L'analisi di sensibilità condotta sulla durata delle finestre ha evidenziato differenze rilevanti tra le granularità considerate, ma la ridotta estensione temporale dei dati rende poco affidabile il confronto alle durate maggiori, per le quali validation e test contengono un numero molto limitato di osservazioni. Uno sviluppo futuro potrebbe pertanto ripetere tale analisi su registrazioni più lunghe e su un numero maggiore di host e scenari di attacco, in modo da confrontare le diverse granularità mantenendo una numerosità sufficiente in tutti gli split.
Per quanto riguarda il triage assistito da LLM, sarebbe utile estendere l'analisi a un numero maggiore di finestre per verificare se l'assegnazione dei livelli di priorità *low*, *medium* e *high* rimane coerente su un campione più ampio. Ulteriori esperimenti potrebbero inoltre confrontare differenti modelli linguistici e differenti configurazioni di prompting, mantenendo invariata la rappresentazione fornita in ingresso.

Un ulteriore sviluppo potrebbe riguardare il confronto tra servizi LLM remoti e soluzioni eseguite localmente. Tale confronto permetterebbe di considerare congiuntamente qualità del triage, latenza, costo computazionale e requisiti di riservatezza dei dati, aspetti particolarmente rilevanti quando la pipeline viene applicata a telemetria di sicurezza.

Una ulteriore analisi potrebbe riguardare l'impiego di modelli di anomaly detection online o semi-online. In particolare, sarebbe interessante valutare approcci come IForestASD, già applicato in letteratura ai log BETH in uno scenario di elaborazione streaming, adattandoli alla rappresentazione a finestre temporali utilizzata nel presente lavoro. Ciò permetterebbe di confrontare i detector addestrati in modo statico con modelli in grado di aggiornarsi progressivamente al variare dei dati osservati, verificandone il comportamento sulle finestre e l'eventuale impatto sulle prestazioni della pipeline.

Bibliografia

[1] K. Highnam, K. Arulkumaran, Z. Hanif, N. R. Jennings, "BETH Dataset: Real Cybersecurity Data for Unsupervised Anomaly Detection Research", *Proceedings of the Conference on Applied Machine Learning for Information Security*, 2021.

[2] E. Eremin, "Unsupervised Anomaly Detection on Cybersecurity Data Streams: A Case with BETH Dataset", *International Journal of Open Information Technologies*, vol. 13, no. 6, pp. 107–113, 2025.

[3] H. A. Doğanay, A. Orman, M. Dener, "Big Data Visualization for Cyber Security: BETH Dataset", *El-Cezerî Journal of Science and Engineering*, vol. 9, no. 4, pp. 1572–1582, 2022. doi: 10.31202/ecjse.1209586.

[4] E. Alatawi, U. Albalawi, "Harnessing AI for Cyber Defense: Honeypot-Driven Intrusion Detection Systems", *Symmetry*, vol. 17, art. 628, 2025. doi: 10.3390/sym17050628.

[5] V. Chandola, A. Banerjee, V. Kumar, "Anomaly Detection: A Survey", *ACM Computing Surveys*, vol. 41, no. 3, art. 15, pp. 1–58, 2009. doi: 10.1145/1541880.1541882.

[6] T. Talaei Khoei, N. Kaabouch, "A Comparative Analysis of Supervised and Unsupervised Models for Detecting Attacks on the Intrusion Detection Systems", *Information*, vol. 14, art. 103, 2023. doi: 10.3390/info14020103.

[7] N. Görnitz, M. Kloft, K. Rieck, U. Brefeld, "Toward Supervised Anomaly Detection", *Journal of Artificial Intelligence Research*, vol. 46, pp. 235–262, 2013.

[8] L. Segurola-Gil, M. Moreno-Moreno, I. Irigoien, A. M. Florez-Tapia, "Unsupervised Anomaly Detection Approach for Cyberattack Identification", *International Journal of Machine Learning and Cybernetics*, vol. 15, pp. 5291–5302, 2024. doi: 10.1007/s13042-024-02237-w.

[9] F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation-Based Anomaly Detection", *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, art. 3, 2012. doi: 10.1145/2133360.2133363.

[10] M. M. Breunig, H.-P. Kriegel, R. T. Ng, J. Sander, "LOF: Identifying Density-Based Local Outliers", *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pp. 93–104, 2000. doi: 10.1145/342009.335388.

[11] M. Amer, M. Goldstein, S. Abdennadher, "Enhancing One-class Support Vector Machines for Unsupervised Anomaly Detection", *Proceedings of the ACM SIGKDD Workshop on Outlier Detection and Description (ODD)*, pp. 8–15, 2013. doi: 10.1145/2500853.2500857.

[12] M. Sakurada, T. Yairi, "Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction", *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, pp. 4–11, 2014. doi: 10.1145/2689746.2689747.

[13] Z. Chen, C. K. Yeo, B. S. Lee, C. T. Lau, "Autoencoder-based Network Anomaly Detection", *2018 Wireless Telecommunications Symposium (WTS)*, pp. 1–5, 2018. doi: 10.1109/WTS.2018.8363930.

[14] J. Huang, C. X. Ling, "Using AUC and Accuracy in Evaluating Learning Algorithms", *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 3, pp. 299–310, 2005.

[15] A. M. Carrington, D. G. Manuel, P. W. Fieguth, T. Ramsay, V. Osmani, B. Wernly, C. Bennett, S. Hawken, O. Magwood, Y. Sheikh, M. McInnes, A. Holzinger, "Deep ROC Analysis and AUC as Balanced Average Accuracy, for Improved Classifier Selection, Audit and Explanation", *IEEE*

Transactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 1, 2023. doi: 10.1109/TPAMI.2022.3145392.

[16] T. Fawcett, "An Introduction to ROC Analysis", *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.

[17] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller, "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel", *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '18)*, 2018. doi: 10.1145/3281411.3281443.

[18] D. Soldani, P. Nahi, H. Bour, S. Jafarizadeh, M. F. Soliman, L. Di Giovanna, F. Monaco, G. Ognibene, F. Risso, "eBPF: A New Approach to Cloud-Native Observability, Networking and Security for Current (5G) and Future Mobile Networks (6G and Beyond)", *IEEE Access*, vol. 11, 2023. doi: 10.1109/ACCESS.2023.3281480.

[19] M. Rieger, A. Shah, A. Alam, M. J. Hossain, "Possibilities and Limitations of Using Large Language Models (LLMs) for Alert Classification and Prioritisation in Security Operations Centers (SOCs)", *Expert Systems with Applications*, vol. 331, art. 133194, 2026. doi: 10.1016/j.eswa.2026.133194.

[20] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", *ACM Transactions on Information Systems*, vol. 43, no. 2, art. 42, 2025. doi: 10.1145/3703155.

[21] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 2023. doi: 10.1145/3605764.3623985.

[22] A. Habibzadeh, F. Feyzi, R. Ebrahimi Atani, "Large Language Models for Security Operations Centers: A Comprehensive Survey", *Journal of Electrical and Computer Engineering*, vol. 2026, art. 3383674, 2026. doi: 10.1155/jece/3383674.

[23] F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation Forest", *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422, 2008. doi: 10.1109/ICDM.2008.17.